DIPLOMARBEIT

Signature-Q-Learning: A Non-Markovian Reinforcement Learning Approach

Zur Erlangung des akademischen Grades

Diplom-Ingenieur

im Rahmen des Studiums

Finanz- und Versicherungsmathematik

am Institut für Stochastik und Wirtschaftsmathematik
der Fakultät für Mathematik und Geoinformation
TU Wien

eingereicht von

Felix Stubenvoll-Pretschuh, BSc. BSc.
Matrikelnummer 01114103

Betreuerin:
Univ.-Prof. Dr. Dipl.-Ing. Christa Cuchiero
Institut für Statistik und Operations Research, Universität Wien

In Zusammenarbeit mit:
Prof. Xin Guo, Coleman Fung Chair in Financial Modeling
Department of Industrial Engineering and Operations Research
University of California, Berkeley

Wien, 17. Dezember 2025

Verfasser

Betreuerin

MASTER THESIS

Signature-Q-Learning: A Non-Markovian Reinforcement Learning Approach

Submitted for the degree of

Master of Science

within the master's program

Financial and Actuarial Mathematics

Institute of Statistics and Mathematical Methods in Economics
Faculty of Mathematics and Geoinformation
Vienna University of Technology

by

Felix Stubenvoll-Pretschuh, BSc. BSc.

Matriculation number 01114103

Supervisor:

Univ.-Prof. Dr. Dipl.-Ing. Christa Cuchiero
Institute for Statistics and Operations Research, University of Vienna

In cooperation with:

Prof. Xin Guo, Coleman Fung Chair in Financial Modeling
Department of Industrial Engineering and Operations Research
University of California, Berkeley

Vienna, December 17, 2025

Author

Supervisor

Kurzfassung

Die meisten Methoden des Reinforcement Learning (bestärkendes Lernen) gehen davon aus, dass die zugrunde liegende Umgebung durch einen Markov-Entscheidungsprozess (MDP, Markov Decision Process) beschrieben werden kann, in dem der aktuelle Zustand des Prozesses ausreicht, um eine optimale Strategie zu erlernen. Viele reale Probleme weisen jedoch historienabhängige Dynamiken auf, bei denen Zustandsübergänge und Belohnungen von vergangenen Zuständen und Aktionen abhängen können, sodass MDP-basierte Lernmethoden sich als unzureichend erweisen. Diese Arbeit widmet sich dieser Problematik und entwickelt einen Rahmen für Reinforcement Learning in nicht-Markov'schen Umgebungen. Dazu werden historienbasierte Entscheidungsprozesse (HDP, History-based Decision Process) als formales Modell eingeführt sowie der Signature-Q-Learning-Algorithmus vorgestellt, der Pfadsignaturen nutzt, um optimale Strategien in solchen Umgebungen zu approximieren.

Der Hauptteil dieser Arbeit gliedert sich in drei Teile. Zunächst werden in Kapitel 4 historienbasierte Entscheidungsprozesse eingeführt, die den Rahmen der klassischen MDPs erweitern, um Umgebungen mit historienabhängigen Dynamiken zu modellieren. HDPs bieten einen nichtstationären Rahmen mit endlichem Zeithorizont, in dem die Strategie des Agenten nicht nur auf dem aktuellen Zustand, sondern auf der gesamten beobachteten Zustands- und Aktionshistorie beruht. In diesem Modell entwickeln wir ein dynamisches Programmierprinzip für die optimalen Wertfunktionen, in Analogie zu den Bellman-Gleichungen für MDPs, charakterisieren optimale Strategien als solche, die diesem Prinzip genügen, und beweisen deren Existenz.

Zweitens wird in Kapitel 5 der Signature-Q-Learning-Algorithmus vorgestellt, eine neue Reinforcement-Learning-Methode für historienabhängige Umgebungen. Dieser basiert auf den universellen Approximationseigenschaften von Pfadsignaturen, einem Konzept, das in der Theorie der rauen Pfade (rough paths) eine wichtige Rolle spielt. Der Algorithmus approximiert die optimale Q-Funktion als lineares Funktional der Pfadsignaturen von beobachteten Zustands-Aktions-Historien und ermöglicht so eine effiziente Repräsentation von zeitlichen Abhängigkeiten. Im Gegensatz zum klassischen Q-Learning in MDPs, das auf einzelnen Zustands-Aktions-Paaren basiert, nutzt Signature-Q-Learning die gesamte Historie und bleibt dabei rechnerisch effizient, indem es gekürzte Pfadsignaturen und eine inkrementelle Aktualisierung auf Basis der Chen-Identität verwendet.

Drittens evaluieren wir in Kapitel 6 den Algorithmus anhand zweier numerischer Experimente. Das Erste stellt eine nur teilweise beobachtbare Umgebung dar und zeigt, dass Signature-Q-Learning vergangene Zustände erfolgreich integriert, um das Problem zu lösen. Das Zweite wendet den Algorithmus auf das Problem der Liquidation einer Aktienposition unter Nutzung des ABIDES-Marktsimulators an, in dem historienabhängige Preiseffekte eine wesentliche Rolle spielen. In beiden Fällen lernt Signature-Q-Learning erfolgreich annähernd optimale Strategien, trotz der nicht-markovschen Natur der Umgebungen. Wir analysieren die Auswirkungen der Länge der Signaturekürzung, diskutieren die Überschät-

zung von Q-Werten und vergleichen die gelernte Strategie mit einer annähernd optimalen Basisstrategie.

Diese Arbeit leistet einen Beitrag zur Weiterentwicklung des Reinforcement Learning, indem sie historienbasierte Entscheidungsprozesse als Modellierungsrahmen einführt und das Lernen in historienabhängigen Umgebungen mit der Approximation von Pfadfunktionalen durch die Signatur von Pfaden verbindet. Der vorgeschlagene Algorithmus ist eine modellfreie, online und off-policy Methode und stellt damit ein vielseitiges Werkzeug zur Lösung nicht-markovscher Probleme in verschiedensten Anwendungsbereichen dar. Zukünftige Forschungsschwerpunkte umfassen die Konvergenzanalyse der Methode, eine Verbesserung der Stichprobeneffizienz sowie die Erweiterung des Ansatzes auf kontinuierliche Aktionsräume.

Abstract

Most reinforcement learning (RL) methods assume that the environment follows Markov Decision Process (MDP) dynamics, where the current state is sufficient for learning an optimal policy. However, many real-world problems exhibit history-dependent characteristics, in which transitions and rewards depend on past states and actions, rendering MDP-based learning methods inadequate. This thesis addresses this challenge, with the aim of developing a reinforcement learning framework for non-Markovian environments by introducing History-Based Decision Processes (HDPs) as a model and proposing Signature-Q-Learning, an algorithm that leverages path signatures to approximate optimal policies in history-dependent settings.

This thesis's main body is structured into three parts. First, we introduce History-based Decision Processes in Chapter 4, extending the MDP framework to account for history-dependent dynamics. HDPs provide a finite-horizon, non-stationary modeling framework in which an agent's decision-making is based on observation–action histories rather than individual states. We establish a dynamic programming principle for optimal value functions in HDPs, analogous to the Bellman equations for MDPs. We characterize optimal policies as those that maximize value functions satisfying this principle and prove their existence.

Second, in Chapter 5, we propose Signature-Q-Learning, a history-dependent reinforcement learning algorithm based on the universal approximation properties of path signatures, a notion that plays a prominent role in rough path theory. The algorithm formulates the optimal Q-function as a linear functional of the signature of an observation-action history, providing an efficient and expressive representation of temporal dependencies. Unlike traditional Q-learning in MDPs, which operates on state-action pairs, Signature-Q-learning encodes the entire history while maintaining computational efficiency through signature truncation and incremental updates based on Chen's identity.

Third, in Chapter 6, we implement and evaluate the algorithm through two distinct numerical experiments. The first experiment modifies the Mountain Car problem to a partially observable setting, demonstrating that Signature-Q-Learning effectively integrates past observations to solve the problem. The second experiment applies the algorithm to a financial execution problem using the ABIDES market simulator, where history-dependent price impacts play a crucial role. In both cases, Signature-Q-Learning successfully learns policies that maximize rewards despite the non-Markovian nature of the environments. We analyze the effects of signature truncation, discuss the overestimation of Q-values, and compare performance against a baseline policy.

This work contributes to reinforcement learning by providing History-based Decision Processes as a novel modeling framework and bridging learning in history-dependent domains with function approximation via path signatures. The proposed method is model-free, online, and off-policy, making it a versatile tool for solving non-Markovian RL problems across various domains. Future research directions include a convergence analysis, improving the method's sample efficiency, and extending it to continuous action spaces.

Acknowledgments

First of all, I would like to express my deepest gratitude to my supervisor, Christa Cuchiero, for her continuous support, guidance, and genuine curiosity in my project. Her invaluable advice whenever I encountered difficulties, together with the freedom she granted me in shaping the direction of my thesis, greatly enriched my research experience. I am especially thankful for Prof. Cuchiero's support in enabling my six-month stay at the University of California, Berkeley, which broadened both my research and perspective, as well as for the opportunities to present my work within and beyond her group, through which I learned a great deal about collaboration and to reflect on my own research.

I would like to extend my sincere gratitude to my host at Berkeley, Xin Guo. From the very beginning, she welcomed me as a full member of her research group, was open to my ideas, and offered her support through thoughtful discussions and valuable input. My time in her department exposed me to many fascinating and interconnected aspects of financial mathematical research and the many interesting people I met, making my stay in the Bay Area truly inspiring and unforgettable.

I am grateful to the Austrian Marshall Plan Foundation for providing financial support for my stay at Berkeley through their scholarship. Their ongoing efforts to foster transatlantic collaboration first inspired me to pursue a research stay in the US, and I hope many more researchers will have the opportunity to benefit from their support.

Furthermore, I would like to thank Sandra Trenovatz, who always offered a timely, knowledgeable, and helpful hand with all organizational matters during my studies.

I am incredibly thankful to my colleagues at TU Wien, especially Hozan, Dominik, Katrin and David, who were with me until the very end. At the start of my studies, I could not have imagined spending so much time together, but every hour shared, both in and outside of university, was truly worthwhile and made this journey a great experience. I hope this friendship continues indefinitely.

A particular thank you goes to my family, my parents, who trusted my decision to pursue further studies and provided their unconditional love and support, both emotionally and financially, throughout this time. Thank you for always encouraging me, even though you had little idea of what I had been working on over the years. Without you, I would not be where I am today.

Finally, my deepest and most heartfelt thank you to you, Katharina. You have been with me all these years. You shared the most enthusiastic and exuberant times with me. You were there when times got rough, when I doubted my decision to pursue this study. You helped me grow, live, and appreciate the good in every aspect of life. I cannot express the amount of gratitude and love I feel towards you. Your sincerity, kindness, and joyfulness continue to inspire me every day. Without you, I could not have done this. I am with you. Always.

Thank you!

Statement of originality

I hereby declare that I have authored the present master thesis independently and did not use any sources other than specified. I have not yet submitted the work to any other examining authority in the same or comparable form. It has not been published yet.

Vienna, December 17, 2025

Felix Stubenvoll-Pretschuh

Contents

1 Introduction **1**

2 The Signature of a Path **11**
2.1 The Signature of Paths of Bounded Variation 11
2.2 The Signature of a Stream of Data and its Computation 17

3 Reinforcement Learning and Markov Decision Processes **19**
3.1 Reinforcement Learning 19
3.2 Markov Decision Processes 21
3.3 Q-Learning in Markov Decision Processes 26
3.4 The Issue with MDPs as a Modeling Framework 30

4 A History-Dependent RL Environment **33**
4.1 History-Based Decision Processes 33
4.2 Value Functions and Optimal Policies in HDPs 39

5 Signature-Q-Learning **47**
5.1 Q-Functions in History-Based Decision Processes 47
5.2 The Signature-Q-Learning Algorithm 48
5.3 Implementation of Approximate Q-Functions 55

6 Numerical Experiments and Results **59**
6.1 Github Repositories 59
6.2 Mountain Car Experiment 59
6.2.1 The Environment 59
6.2.2 Training Details 62
6.2.3 Testing Details 64
6.2.4 Results 66
6.3 Simple Execution Experiment 74
6.3.1 The Market Simulator ABIDES 74
6.3.2 The Environment 79
6.3.3 Baseline Policy 82
6.3.4 Training Details 85
6.3.5 Testing Details 86
6.3.6 Results 88

A Appendix **101**
A.1 Stochastic Kernels 101
A.2 Measurable Selection 105

List of Figures 107

List of Tables 109

Bibliography 111

1 Introduction

Reinforcement learning (RL) is a machine learning paradigm designed to address sequential decision-making problems through goal-oriented learning. More precisely, a RL problem involves an agent interacting with an environment to learn optimal sequential decisions that lead to the achievement of a specific goal. This learning is typically based on feedback from repeated interactions, and occurs without explicit supervision. Many RL approaches do not even require prior knowledge of the environment's dynamics, however, they may be learned and used to improve decision-making. RL algorithms have achieved notable success across various domains, including problems in finance such as optimal execution, portfolio management, and option pricing (see [25] for a recent survey), surpassing human champions in complex board games such as Chess [68] and Go [69], as well as achieving human-level performance in video games [54, 78]. Furthermore, the RL variant *reinforcement learning from human feedback* was used to fine-tune OpenAI's language model *InstructGPT* [57], and its widely successful sibling model *ChatGPT* [56].

Reinforcement learning employs an abstract framework to model the interaction between a learning agent and its environment, characterized by states (or observations), actions, and rewards. This framework is typically formalized as a Markov Decision Process (MDP) [60], defined over a set of states $\mathcal{S}$, where transitions between states are influenced by actions chosen from a set $\mathcal{A}$. Interactions occur at discrete time steps and we consider a finite decision-making horizon $T \in \mathbb{N}$. At each step $t \in \{0, 1, \ldots, T\}$, the process is in some state $s_t \in \mathcal{S}$. Upon taking an action $a_t \in \mathcal{A}$, the process transitions to a new state $s_{t+1} \in \mathcal{S}$ and emits a reward $R_t(s_t, a_t) \in \mathbb{R}$, representing the immediate payoff associated with the state-action pair (s_t, a_t). The transition dynamics are governed by transition functions or kernels P_t, mapping pairs $(s, a) \in \mathcal{S} \times \mathcal{A}$ to probability distributions over $\mathcal{S}$ for each step $t \in \{0, 1, \ldots, T-1\}$. More specifically, for finite state and action spaces, the state transition probability at step t is given by

$$\begin{aligned} P_t(s_t, a_t, s') &= \mathbb{P}[S_{t+1} = s' \mid S_t = s_t, A_t = a_t] \\ &= \mathbb{P}[S_{t+1} = s' \mid S_0 = s_0, A_0 = a_0, \ldots, S_t = s_t, A_t = a_t] \end{aligned} \tag{1.1}$$

for any $s', s_0, s_1, \ldots, s_t \in \mathcal{S}$ and $a_0, a_1, \ldots, a_t \in \mathcal{A}$, where $\{S_u, A_u\}_{u=0,\ldots,t}$ and S_{t+1} denote the process' random states and random actions, respectively. Equation (1.1) is known as the eponymous *Markov property*, which states that the next state's distribution at $t+1$ depends solely on the current state and action values, and one may discard any information prior to time t. A policy is a tuple $\pi = \{\pi_t\}_{t=0}^{T-1}$ which governs action selection, where π_t specifies a rule, either deterministic or stochastic, for selecting actions at step t. The goal is to identify a policy that maximizes the expected sum of discounted rewards over the decision-making

horizon. Mathematically, this is equivalent to finding the optimal value functions defined as

$$V_t^*(s) = \sup_{\pi} \mathbb{E}\Big[\sum_{u=t}^{T-1} \gamma^{u-t} R_u(S_u, A_u) + \gamma^{T-t} R_T(S_T) \;\Big|\; S_t = s\Big], \quad \forall s \in \mathcal{S}, \tag{1.2}$$

for each $t \in \{0, 1 \ldots, T-1\}$, and as $V_T(s) := R_T(s)$ for $t = T$, with the terminal reward function $R_T : \mathcal{S} \to \mathbb{R}$, where the discount factor $\gamma \in [0, 1]$ weights future rewards. A policy that achieves the supremum in (1.2) is called *optimal*. Due to property (1.1), optimal polices in MDPs are functions of the current state s_t at each $t \in \{0, 1, \ldots, T-1\}$, ignoring prior states visited and actions taken. This Markovian nature of optimal policies is a key advantage of MDPs, contributing to their popularity as modeling framework in reinforcement learning.

If $\mathcal{S}$ and $\mathcal{A}$ are finite, MDPs can be solved effectively using dynamic programming methods, such as backward induction for finite-horizon problems [60, Section 4.5], and value or policy iteration for the infinite-horizon cases [60, Sections 6.3 and 6.4]. These methods can be generalized to continuous state and action spaces under certain regularity conditions on the transition kernels P_t and reward functions R_t, ensuring the measurability of policies and the existence of expectations, compare [29]. However, their computational cost grows exponentially with the dimension of state and action spaces, a phenomenon known as the *curse of dimensionality*. Additionally, these methods require complete knowledge of the environment, including transition kernels and reward functions, which is usually unavailable in practice.

Reinforcement learning algorithms circumvent the need for explicit knowledge of the environment by learning policies through sampling transitions from it. A prominent example is the Q-learning algorithm, introduced by Watkins [80, 81]. The algorithm keeps estimates of the optimal action-values or Q-values $Q_t^*(s, a) \in \mathbb{R}$ for all pairs (s, a), which represent the expected cumulative discounted reward over the remaining decision horizon from taking action $a \in \mathcal{A}$ in state $s \in \mathcal{S}$ at time t, and acting optimally thereafter. It builds on the fact that an optimal policy can be derived from the optimal Q-functions $Q_t^* : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ by selecting an action that maximizes $Q_t^*(s, \cdot)$ at each step for all $s \in \mathcal{S}$. Given that the process is in state s_t at step t, action a_t is obtained by

$$a_t = \pi_t^*(s_t) \in \arg\max_{a \in \mathcal{A}} Q_t^*(s_t, a), \tag{1.3}$$

implying that the knowledge of the functions Q_t^* is equivalent to an optimal policy. A dynamic programming relation for the optimal Q-functions Q_t^* allows for iterative updating of the estimates towards the true optimal Q-values $Q_t^*(s, a)$ by sampling transition tuples (s_t, a_t, r_t, s_{t+1}) from the environment. This procedure is a form of *temporal-difference learning* [71, Chapter 6], as it updates estimates using the immediately obtained rewards together with estimates at the subsequent step as the target, a process called *bootstrapping*. When $\mathcal{S}$ and $\mathcal{A}$ are finite and small the estimates of the values $Q_t^*(s, a)$ can be stored in a $|\mathcal{S}| \times |\mathcal{A}|$ matrix and the method is referred to as *tabular* Q-learning. For large or continuous spaces function approximation techniques are required, where one considers a class of parametrized functions $\{Q_t^\theta\}_\theta$ with finite-dimensional parameter vector $\theta \in \mathbb{R}^d$ for some $d \in \mathbb{N}$, and learns the parameter values that best approximate Q_t^* through gradient-based

methods.

The ability of most reinforcement learning algorithms, including Q-learning, to learn an optimal policy depends critically on the assumption that the environment adheres to the Markov property. When this assumption is violated, learned policies that are functions of the current state may be suboptimal. This raises a fundamental question: How can reinforcement learning address non-Markovian environments where transitions and rewards depend on the history of states and actions? Such dynamics arise in various contexts. In finance, for instance, models based on fractional Brownian motion lead to non-Markovian price dynamics. In an optimal execution setting, trading impacts prices in a history-dependent manner. In memory tasks, for example considered in [28], explicit information has to be remembered over a number of steps, and temporal dependencies, such as achieving sub-goals in a specific order, also exemplify history-dependent problems.[1] Furthermore, in situations where observations provide only partial information about states, the agent is required to infer knowledge about the true state from observation histories. As an example consider a robot navigating a maze with limited visibility, relying on memory of past moves to determine its location. Such scenarios necessitate modeling frameworks accommodating history dependence and corresponding learning methods.

This thesis has two primary objectives: (1) to introduce *history-based decision processes* (HDPs) as a general modeling framework for history-dependent dynamics, and (2) to develop *Signature-Q-learning*, a novel reinforcement learning algorithm leveraging *path signatures* as a core feature for handling history-dependence. HDPs provide a finite-horizon, non-stationary modeling framework for reinforcement learning. The agent again interacts with the environment at steps $t \in \{0, 1, \dots, T\}$, now receiving observations $o_t \in \mathcal{O}$ without assumptions about whether the observation space $\mathcal{O}$ represents true states or partial observations thereof. Transition dynamics may depend arbitrarily on the full history of observations and actions up to the current step,

$$h_t := (o_0, a_0, o_1, a_1, \dots, a_{t-1}, o_t).$$

Assuming countable $\mathcal{O}$, the transition probabilities at step t are governed by a transition kernel P_t, now a function of h_t, and given as

$$\mathbb{P}[O_{t+1} = o \mid H_t = h_t, A_t = a] = P_t(h_t, a, o), \tag{1.4}$$

where H_t denotes the random history, A_t the random action, and O_{t+1} the random observation at steps t and $t+1$ respectively. Similarly, the rewards $R_t(h_t, a_t)$ depend on the history-action pair (h_t, a_t). Equation (1.4) implies that optimal value functions in this framework that are too functions of histories, and we establish a dynamic programming procedure for them, analogous to the Markovian case. This provides the basis for characterizing optimal policies as maximizing policies in Theorem 4.16, which select actions maximizing the sum of the immediate reward and the expected value over the remaining episode at each single decision step. We prove the existence of such maximizing and thereby

[1] [28] considers a memory task of controlling a reacher where the target's position is only available at $t = 0$. Temporal dependencies are, for example, present in learning to follow a recipe or assembly manual, where the correct order of steps is important and transition dynamics depend on the past.

optimal policies in HDPs in Theorem 4.19.

The Signature-Q-learning algorithm builds upon the optimal Q-functions in HDPs. With some abuse of notation, they are again denoted as Q_t^* for $t \in \{0, 1, \dots, T\}$. However, in the HDP setting, optimal Q-functions are now functions of history-action pairs, such that $Q_t^*(h_t, a_t)$ provides the expected cumulative reward from taking action $a_t \in \mathcal{A}$ in history h_t, and acting optimally thereafter. We show that, analogously to their Markovian counterparts, knowledge of the functions Q_t^* yields an optimal policy by maximizing over actions at each step in the spirit of (1.3), now with histories h_t instead of states s_t as input. From the respective procedure for optimal value-functions, we show that the optimal Q-functions also satisfy a dynamic programming relation, *the optimal Q-iteration for HDPs*, which allows us to learn estimates through sampling histories by interaction with the environment.

A core component of Signature-Q-learning is the *signature of a path*, a mathematical concept originating in the work of Chen [14, 15] and foundational to rough path theory introduced by Lyons [48, 49]. The signature exhibits some important properties, specific references are provided in Section 2.1. For a path including time as one of its components, its signature at time $T \in (0, \infty)$ uniquely determines the path's trajectories over the interval $[0, T]$, providing a means of encoding the path's information. Chen's identity states an algebraic structure, which facilitates incremental updates of the signature via tensor operations, whenever new path information arrives. Linear functionals of the signature admit a universal approximation property, which establishes the time-augmented paths' signatures as a linear regression basis for continuous (with respect to some variation metric on path spaces) path functionals. These properties have driven the successful development of signature-based methods in mathematical finance [2, 4, 16, 35, 47, 53] and machine learning [83, 48, 43, 53].

In Signature-Q-learning, path signatures enable compact representation of histories. Since enumerating histories and keeping an estimate for each pair (h_t, a) is computationally unfeasible, the method represents the optimal Q-functions as linear functionals of the signature of observation-action histories and learns the functional's parameters. For a history h_t, we denote its time-augmented history, which includes the time-steps t as an additional component, by $\hat{h}_t$. Any pair $(\hat{h}_t, a_t)$ is then given as a stream

$$\hat{h}_t^{a_t} := (u, o_u, a_u)_{u=0}^t.$$

The linear interpolation of $\hat{h}_t^{a_t}$ constitutes a continuous path in $\mathbb{R}^{d+2}$, with $d \in \mathbb{N}$ the dimension of observation space $\mathcal{O}$. Let its signature be denoted by $S_t(\hat{h}_t^{a_t})$. It provides a feature vector for approximating the optimal Q-functions by parametrized maps

$$Q(h_t, a_t \mid \ell) := \ell(S_t(\hat{h}_t^{a_t})), \tag{1.5}$$

where ℓ is a linear functional acting on the signature $S_t(\hat{h}_t^{a_t})$. We show that, under some regularity conditions, the universal approximation property guarantees that the optimal Q-functions can be approximated arbitrarily well by such functionals. Signature-Q-Learning learns the parameters of a linear functional ℓ, which provides a sufficiently good estimation. The algorithm's learning mechanism is based on the optimal Q-iteration in HDPs. We highlight that the approximation argument together with the learning procedure yields a

single functional of the form (1.5) that approximates the optimal Q-functions at *all* steps $t \in \{0, 1, \ldots, T\}$, instead of a separate functional at each step, ensuring efficient use of samples. Furthermore, Chen's identity allows for computationally inexpensive updating of signatures with sequentially arriving data, aligning naturally with reinforcement learning's iterative nature. The proposed method constitutes a model-free, online, and off-policy RL algorithm, and offers a versatile tool for learning in history-dependent environments.

Related Work Using path signatures to approximate Q-functions in an RL setting is partly motivated by the work of Bonnier et al. [9], who propose a model for processing sequential data, with the signature acting as a layer within deep neural networks. Experiments include a partially observable RL problem similar to that considered in Section 6.2. The method differs from the one presented in this thesis as it relies on universal approximation properties for (deep) neural networks, resulting in a non-linear functional of the signature as Q-function approximation. To the best of our knowledge, apart from the experiment in [9], the signature has not been explicitly incorporated into an approximation procedure in an RL setting.

A substantial body of literature exists on Markov Decision Processes. We highlight the two primary references for the present work. The monograph of Puterman [60] provides a unified and comprehensive treatment of MDPs and their theoretical and computational solvability, whereas Hinderer [29] concentrates on finite-horizon MDPs and develops several frameworks ranging from models with finite state and action spaces to those on arbitrary measurable spaces and even random environments. In its mathematical treatment, the latter refrains from advanced measure theory, emphasizing the structure of solutions and the construction of specific models in various applications.

Approaches to address non-Markovian RL include expanding the state representation and introducing partial observability. In the former, one defines a new state variable $\tilde{s}_t := (s_{t-k+1}, s_{t-k+2}, \ldots, s_{t-1}, s_t)$, that includes the states over the last k steps at each time point as input to the learning algorithm. This is equivalent to implicitly assuming that the environment follows a *k-order* MDP, where the distribution of the next state, given a chosen action, depends on the last k states. If this assumption is true, the problem becomes Markovian with respect to $\tilde{s}_t$. This is, for example, the case in the work of Mnih et al. [54], who introduce Deep-Q-Networks, in which $\tilde{s}_t$ contains four frames of a computer game. In other cases, however, it remains unclear whether this approach indeed induces an MDP; see, for instance, Sadighian [64, 65]. The latter approach models the problem as a *partially observable Markov Decision Process* (POMDP), following the classical formulation by Kaelbling et al. [34], which is a latent MDP for which the decision maker only receives partial observations of the true Markovian states, and must infer more information about them from the history of observations. While POMDPs are of theoretical interest on their own, learning in such an environment proved to be notoriously hard; see, for example, Liu et al. [44].

Note that the history-based environment model proposed in the present thesis incorporates both mentioned approaches. If the environment follows a POMDP, it naturally uses the history of available observations and actions to create learning policies. If the environment follows a k-order MDP, the observations in the HDP setting represent the true state, and in incorporating the entire history prior to the current step, we include the necessary k past

steps on which the transition dynamics depend.

Another line of work on general non-Markovian RL is the Feature-RL framework proposed by Hutter [30, 31, 32]. It aims at learning a map ϕ that aggregates histories to a finite set of states, $s_t = \phi(h_t)$, such that the aggregated process is (approximately) a small MDP, which can be solved with classical methods such as policy and value iteration. Learning ϕ crucially depends on the class of functions to choose from, which dictates the possible MDP representation, the search method applied within this class, and the cost functional, which defines what a good MDP representation looks like. Typical function classes in this setting are tree-based, for example, suffix trees. Although Feature-RL is inherently model-based, Daswani and Hutter [18] propose a model-free variant, in which the learning objective for ϕ is the direct optimization of rewards. The authors propose a value-based cost function[2], which intertwines the selection of the feature map ϕ and Q-learning, such that the optimization for ϕ compares different maps based on the resulting Q-learning performance in the corresponding aggregated process. The process induced by ϕ no longer has to be a strict MDP, and the approach extends to linear function approximation. There are some similarities between [18] and the method in this thesis. Both consider general RL starting from histories; however, they differ in that we do not assume a finite observation space, and consider an a priori fixed feature map ϕ, specifically the signature transform. Furthermore, the resulting range of our feature map is still uncountable and does not aim at inducing any Markovian structure. Both works rely on linear function approximation, with the difference that we approximate optimal Q-functions defined directly in the history-based setting, as opposed to Q-functions for some finite-state aggregated process.

Learning in partially observable domains related to the present thesis is addressed in [27, 28]. Both works utilize recurrent neural networks as function approximators to memorize information through time. Heess et al. [28] extend two existing actor–critic algorithms, where the critic is a parametrized estimate of the Q-function whose gradient is used to update a parametrized policy. Policies and Q-functions are considered as functions of histories h_t, and are parametrized as recurrent and long-short-term memory networks. Note that if the policy in Algorithm 1 in [28] were to select actions according to the maximal action value provided by the current Q-function estimate, the algorithm would amount to a Q-learning variant, closely related to Signature-Q-Learning in Chapter 5. Hausknecht and Stone [27] build on the Deep-Q-Networks introduced by Mnih et al. [54], extending them through the addition of a recurrent layer to encode past information, whereby the network serves as a parametrized Q-function.

Finally, we provide some references to Q-learning in MDPs, as it is the starting point for Signature-Q-Learning. Under suitable conditions on the sampling policy and learning rates, tabular Q-learning was shown to converge in the stationary case by Tsitsiklis [75] and by Jaakkola et al. [33]. Convergence with function approximation is less established and may fail due to the combination of off-policy sampling, bootstrapping, and approximation; see [71, Section 11.3] and [76]. Convergence with linear function approximation was established by Szepesvári [73] under restrictions on the features, and by Melo [52] under restrictive assumptions on the sampling policy. Maei [50] relaxes some of these assumptions and proposes an algorithm that is guaranteed to converge to a local minimum. Furthermore,

[2] More precisely, a regularized, pathwise squared Bellman error for Q-functions.

Bhatnagar et al. [7] and Carvalho et al. [13] establish convergence for two time-scale variants of Q-learning with linear function approximation. Convergence results for Q-learning with non-linear function approximation remain scarce, although numerical performance is often strong; examples include Neural-fitted Q-learning by Riedmiller [62] and, most notably, the success of Deep-Q-Networks by Mnih et al. [54], where the Q-function is represented by a deep neural network.

We note that most work on Q-learning, including the references in the preceding paragraph, addresses infinite-horizon, stationary MDPs, where $T = \infty$ and the transition dynamics are time-homogeneous. In such cases, the optimal Q-function is time-independent, so that a single function Q^* describes the expected cumulative reward given a pair (s, a) at any time $t \in \mathbb{N}$, assuming optimal behaviour thereafter. Tabular Q-learning for finite-horizon, non-stationary MDPs is treated by Garcia and Ndiaye [21], and recently, Veness and Papadimitriou [79] established its convergence via stochastic approximation techniques. Moreover, stationary, infinite-horizon MDPs are often used as modeling assumptions in RL problems, either explicitly or implicitly through the application of RL methods built on such assumptions, even when the true problem is finite-horizon and potentially non-stationary. The work of Majeed and Hutter [51] explains why Q-learning often still performs well beyond the MDP setting by proving that convergence extends to certain classes of non-Markovian environments. Drawing on ideas from Feature-RL, the authors show that state-uniformity of the optimal Q-function is both necessary and sufficient for convergence. This means that whenever two histories are mapped to the same aggregated state via a feature map ϕ, their optimal Q-values must agree. This substantially strengthens the theoretical foundations of Q-learning, showing that it may remain a sound learning mechanism even in non-stationary and non-Markovian domains, which are common in practical applications.

Thesis Outline In Chapter 2 the signature of paths of bounded variation is defined. We state its properties which make it a natural choice for the application as function approximator in our proposed reinforcement learning method, most importantly its universal approximation property given in Theorem 2.15, and provide details on the signature's computation for discrete data streams.

Chapter 3 provides a thorough introduction to reinforcement learning, its components and MDPs as the usual modeling framework, which will serve as the basis for the generalization to the history-dependent set-up. We define in mathematical detail a finite-horizon, non-stationary MDP with Borel state space and finite action space and make precise the notion of optimality, with the help of optimal value and Q-functions. Equation (3.15) characterizes optimal policies as maximizing policies and the existence of such in the considered set-up is proved in Theorem 3.5. Furthermore, we present the Q-learning method for MDPs based on the Bellman equations (3.18), including a detailed procedure for the tabular case in Algorithm 1, and a description on how to use function approximation if the state space is large or infinite. The chapter concludes with remarks on the shortcomings of using MDPs as the model for RL problems, serving as motivation for the introduction of a more general framework.

Chapter 4 constitutes the main theoretical part of the thesis. We introduce History-dependent decision processes (HDPs) as a generic model for RL problems without any

Markovian assumptions on transition dynamics. Generalizing the exposition in the previous chapter a detailed description of its components is given, most notably transition dynamics depending arbitrarily on observation-action histories. This then leads to the definition of optimal value functions (Definition 4.15), now in terms of histories instead of single states. Analogously to MDPs, Theorem 4.16 characterizes optimal policies as maximizing policies, and further provides us with the *optimal value iteration* in HDPs, a dynamic programming procedure from which an optimal policy could be obtained, if the HDPs components are known and computable. The chapter concludes by proofing the existence of maximizing policies in Theorem 4.19.

Chapter 5 introduces Signature-Q-Learning as learning method for history-dependent decision processes, and as a generalization of Q-learning in MDPs. To this end, optimal Q-functions in HDPs are defined as functions of observation-action histories, and we detail how their knowledge is sufficient to obtain an optimal policy π^*. Equation (5.4) states the optimal Q-iteration in HDPs, which serves as the algorithm's basis. Building on the path signature's properties collected in Chapter 2, most notably the universal approximation property, we parametrize optimal Q-functions as a linear functional of the histories' signature and detail the process of learning the functional's parameters by sampling histories from the environment. A detailed algorithm for Signature-Q-Learning is provided in Algorithm 2, and aspects regarding its implementation are discussed.

Finally, in Chapter 6 we show Signature-Q-Learning's success in solving reinforcement learning problems in two experiments. In each experiment, we construct the environment with a description of its elements, give details on the implementation and the training procedure. The first experiment is a partially observable modification of the classic Mountain Car problem from the RL library `Gymnasium` [74]. The algorithm's performance is discussed by comparing different configurations for parameters distinct to the Signature-Q-learning, namely the signature's truncation order and the basepoint used for its computation. The environment offers a clear terminal state that must be reached to count an episode as a success, and while the method learns a strategy that reaches this states, it overestimates Q-values. We discuss this problem, including a hypothesis that identifies the different episode lengths as the reason. The second experiment constitutes a simple execution problem. It is implemented as custom environment based on the market simulator ABIDES [12, 3]. We compare the policies obtained from Signature-Q-Learning to a baseline policy that serves as proxy for optimality. Albeit the algorithm does not achieve full optimality, it converges towards the baseline policy, and to the true optimal Q-values, with the issues observed in the first experiment not being present in the second. The code for both experiments is available on the author's github account.[3]

Additionally, Appendix A.1, presents key results on stochastic transition kernels and provides a theoretical foundation for the decision processes discussed in earlier chapters. In particular, we revisit the construction of discrete-time stochastic processes governed by a specified collection of transition kernels. This framework underpins the modeling of reinforcement learning problems, where the evolution of the environment is determined step-by-step by these kernels.

[3] https://github.com/phaelicks

Notation The set of all probability measures on a measurable space $(\Omega, \mathcal{F})$ will be denoted $\mathrm{Prob}(\Omega, \mathcal{F})$ or simply $\mathrm{Prob}(\Omega)$ if the corresponding σ-algebra is clear from the context. For a real-valued random variable X on $(\Omega, \mathcal{F})$ with distribution $P \in \mathrm{Prob}(\Omega)$ we write $X \sim P$, while a realization of X will usually be written as lower case letter $x = X(\omega) \in \mathbb{R}$ for some $\omega \in \Omega$. Given a second measurable space $(\Omega', \mathcal{F}')$ the product-σ-algebra on $\Omega \times \Omega'$ is denoted $\mathcal{F} \otimes \mathcal{F}'$. With some notational overload, the symbol $\otimes$ will also be used to define product kernels, and as the tensor product, however, the respective meaning will always be clear from the objects involved. Given a topological space $(\mathcal{X}, \mathcal{T})$, we will denote the Borel σ-algebra on $\mathcal{X}$ induced by the topology $\mathcal{T}$ as $\mathcal{B}_{\mathcal{X}}$. Furthermore, for any $T \in \mathbb{N}$ define

$$[T] := \{0, 1, \ldots, T\}$$

and the extended real numbers as $\overline{\mathbb{R}} := \mathbb{R} \cup \{-\infty, +\infty\}$. Finally, for a function $f : \Omega \to \overline{\mathbb{R}}$ we denote its positive and negative absolute part by

$$f^+ := \max\{0, f\} \quad \text{and} \quad f^- := \max\{0, -f\}.$$

2 The Signature of a Path

This sections describes the main ingredient of the proposed reinforcement learning method, namely the *signature of a path*. We restrict ourselves to continuous paths of bounded 1-variation, however, we highlight that the definition of the signature and most of its properties can be extended to a much broader class of less regular paths, namely (weakly) geometric rough paths, see [49] for a thorough introduction to the theory.[1] We start with some basic notion before giving the precise definition of the signature, followed by an exposition of some of its algebraic and analytic properties that make it suitable for the application in a reinforcement learning setting. We especially emphasise a version of the universal approximation theorem in Theorem 2.15, which enables the signature's use to approximate linear functions on path spaces. Furthermore, we define the signature of a discrete stream of data and demonstrate its computation in practice. The exposition's structure loosely follows [9, 46] while its contents and notation draw from [16, 4].

2.1 The Signature of Paths of Bounded Variation

We start by defining the space in which the signature takes its values. For $d \in \mathbb{N}$ and $n \in \mathbb{N}_0$ the n-th tensor product of $\mathbb{R}^d$ is given by

$$(\mathbb{R}^d)^{\otimes 0} := \mathbb{R}, \quad (\mathbb{R}^d)^{\otimes n} := \underbrace{\mathbb{R}^d \otimes \cdots \otimes \mathbb{R}^d}_{n \text{ times}},$$

with $\otimes$ the tensor product.

Definition 2.1 (Extended tensor algebra)**.** Let $d \in \mathbb{N}$. The extended tensor algebra over $\mathbb{R}^d$ is defined as

$$T((\mathbb{R}^d)) := \prod_{n \geq 0} (\mathbb{R}^d)^{\otimes n} = \{\boldsymbol{a} := (a_0, a_1, \dots, a_n, \dots) \mid a_n \in (\mathbb{R}^d)^{\otimes n}, n \in \mathbb{N}_0\}.$$

We equip $T((\mathbb{R}^d))$ with an addition, a multiplication and scalar multiplication, defined for $\boldsymbol{a}, \boldsymbol{b} \in T((\mathbb{R}^d))$ and $\lambda \in \mathbb{R}$ by

$$\boldsymbol{a} + \boldsymbol{b} := (a_0 + b_0, a_1 + b_1, \dots, a_n + b_n, \dots) \in T((\mathbb{R}^d)),$$
$$\boldsymbol{a} \otimes \boldsymbol{b} := (c_0, c_1, \dots, c_n, \dots) \in T((\mathbb{R}^d)),$$

where

$$c_n := \sum_{k=0}^{n} (a_k \otimes b_{n-k}) \in (\mathbb{R}^d)^{\otimes n},$$

[1] It is possible to define the signature also for non-geometric rough paths, however some properties change, such as Chen's identity and integration by parts as provided by the shuffle product, compare [23].

and $\lambda \boldsymbol{a} := (\lambda a_0, \lambda a_1, \dots, \lambda a_n, \dots)$. Then, $(T((\mathbb{R}^d)), +, \cdot, \otimes)$ forms a non-commutative algebra over $\mathbb{R}$ with unit $\mathbf{1} = (1, 0, 0, \dots)$.

Intuitively, $T((\mathbb{R}^d))$ contains all sequences of tensors with of increasing order with components in $\mathbb{R}$. We further define the space of all finite sequences of tensors as well as the sub-set of all sequences of fixed length.

Definition 2.2 ((Truncated) tensor algebra)**.** Let $d \in \mathbb{N}$. The *truncated tensor algebra* of order $N \in \mathbb{N}$ over $\mathbb{R}^d$ is defined as

$$T^{(N)}(\mathbb{R}^d) := \{\boldsymbol{a} \in T((\mathbb{R}^d)) \mid \forall n \geq N : a_n = 0\},$$

with dimension[2] $d_N := \sum_{k=0}^{n} d^k = (d^{N+1} - 1)/(d-1)$. The *tensor algebra* over $\mathbb{R}^d$ is defined as

$$T(\mathbb{R}^d) := \bigcup_{N \in \mathbb{N}} T^{(N)}(\mathbb{R}^d).$$

Both, the truncated and the tensor algebra, equipped with addition, multiplication and scalar multiplication as in Definition 2.1 form a a non-commutative, unitary algebra over $\mathbb{R}$.

For $n \in \mathbb{N}$, and a multiindex $I = (i_1, i_2 \dots, i_n)$ with entries $i_j \in \{1, 2, \dots, d\}$ for $1 \leq j \leq n$, we set $|I| := n$. Further, let $E = \{e_1, \dots, e_d\}$ be the canonical basis of $\mathbb{R}^d$. For each multiindex I with $|I| = n \geq 1$ we set

$$e_I := e_{i_1} \otimes e_{i_2} \otimes \cdots \otimes e_{i_n}.$$

Observe that the set $\{e_I \mid |I| = n\}$ is the canonical orthonormal basis of $(\mathbb{R}^d)^{\otimes n}$ for each $n \in \mathbb{N}$. If we denote the basis element of $(\mathbb{R}^d)^{\otimes 0} = \mathbb{R}$ by $e_\emptyset$, and set $|\emptyset| := 0$ for the empty index $I = \emptyset$, each element $\boldsymbol{a} \in T((\mathbb{R}^d))$ can be written as the formal sum

$$\boldsymbol{a} = \sum_{n \geq 0} \sum_{|I|=n} a_I e_I,$$

with $a_I \in \mathbb{R}$ for each multiindex I.

An element $\boldsymbol{a} \in T((\mathbb{R}^d))$ is invertible if and only if $a_0 \neq 0$ [49]. In particular, we consider the subset $T_1((\mathbb{R}^d)) := \{\boldsymbol{a} \in T((\mathbb{R}^d) \mid a_0 = 1\}$. For $\boldsymbol{b} \in T_1((\mathbb{R}^d))$, its inverse is given as

$$\boldsymbol{b}^{-1} := \sum_{n \geq 0} (\mathbf{1} - \boldsymbol{b})^{\otimes n},$$

which is well defined since for each $n \geq 0$, due to $(\mathbf{1} - \boldsymbol{b})_0 = 0$, only finitely many terms of the sum contain non-zero tensors of degree n. Note that $(T_1((\mathbb{R}^d)), \otimes)$ forms a group, and the signature of a path defined below takes its values in this group.

Next, we describe linear functionals acting on elements $\boldsymbol{a} \in T((\mathbb{R}^d))$. Given a multiindex I and basis element e_I we define the corresponding linear functional acting on $\boldsymbol{a}$ by $\langle e_I, \boldsymbol{a} \rangle := a_I$. These basic functionals are extended to linear functionals $\langle \ell, \cdot \rangle : T((\mathbb{R}^d)) \to \mathbb{R}$

[2] Strictly speaking, $T^{(N)}(\mathbb{R}^d) \subset T((\mathbb{R}^d))$ is infinite-dimensional, however it can readily be identified with the d^N-dimensional space $\bigoplus_{n=0}^{N} (\mathbb{R}^d)^{\otimes n}$.

for any element $\ell = \sum_{|I|\geq 0} \ell_I e_I \in T(\mathbb{R}^d)$ with $\ell_I \in \mathbb{R}$, that act on $\boldsymbol{a}$ by

$$\langle \ell, \boldsymbol{a} \rangle := \sum_{|I|\geq 0} \ell_I \langle e_I, \boldsymbol{a} \rangle = \sum_{|I|\geq 0} \ell_I a_I.$$

Observe that the sum is well defined since $\ell_I = 0$ for all but finitely many I. With a slight abuse of notation we will refer to $\ell \in T(\mathbb{R}^d)$ as the corresponding linear functional $\langle \ell, \cdot \rangle$ as well as the family of coefficients $\{\ell_I\}_{|I|\geq n}$. For practical purposes, we will consider linear functionals of fixed length $N \in \mathbb{N}$ in Chapter 5, such that $\ell_I = 0$ for all multiindices with $|I| \geq N$. In this case $\ell \in T^{(N)}(\mathbb{R}^d)$, and we will implicitly identify ℓ with a finite-dimensional vector in $\mathbb{R}^{d_N}$, where d_N is the dimension of $T^{(N)}(\mathbb{R}^d)$, when considering practical implementations.

A path $X : [0,T] \to \mathbb{R}^d$ is said to be of finite 1-variation on the interval $[s,t] \subset [0,T]$, if its total variation is finite,

$$\|X\|_{1,[s,t]} := \sup_{\mathcal{D}\subset[s,t]} \sum_{t_i \in \mathcal{D}} \|X_{t_i} - X_{t_{i-1}}\|_1 < \infty, \tag{2.1}$$

where the supremum is taken over all finite partitions $\mathcal{D} = \{t_i\}_i$ of $[s,t]$, and $\|\cdot\|_1$ denotes the 1-norm on $\mathbb{R}^d$. We denote the space of all continuous paths of finite 1-variation on $[0,T]$ as $\mathcal{V}_1([0,T],\mathbb{R}^d)$ and the subset of all such paths starting at the origin by $\mathcal{V}_1^0([0,T],\mathbb{R}^d) := \{X \in \mathcal{V}_1([0,T],\mathbb{R}^d) \mid X_0 = 0\}$. For two paths $X, Y \in \mathcal{V}_1([0,T],\mathbb{R}^d)$ we define their 1-variation distance over an interval $[s,t] \subset [0,T]$ as

$$d_{1,[s,t]}(X,Y) := \|X - Y\|_{1,[s,t]}. \tag{2.2}$$

Note, that the space $\mathcal{V}_1^0([0,T],\mathbb{R}^d)$ equipped with $d_{1,[0,T]}$, becomes a metric space.

We are now ready to define the signature of a path of finite variation.

Definition 2.3. Let $X \in \mathcal{V}_1([0,T],\mathbb{R}^d)$. We define the *signature* of X over the interval $[s,t] \subset [0,T]$ as

$$S_{s,t}(X) := (1, S^1_{s,t}(X), \dots, S^n_{s,t}(X), \dots) \in T_1((\mathbb{R}^d)), \tag{2.3}$$

with

$$S^n_{s,t}(X) := \int_{s<u_1<\cdots<u_n<t} \mathrm{d}X_{u_1} \otimes \cdots \otimes \mathrm{d}X_{u_n} \in (\mathbb{R}^d)^{\otimes n}.$$

Further, we define the *truncated signature up to order* $N \in \mathbb{N}$ of X as

$$S^{(N)}_{s,t}(X) := (1, S^1_{s,t}(X), \dots, S^N_{s,t}(X)) \in T^{(N)}(\mathbb{R}^d). \tag{2.4}$$

For the case $s = 0$ we write $S_t(X) := S_{s,t}(X)$ and if we only refer to the signature of X, we implicitly mean the signature over the whole interval, i.e. $S_T(X)$.

The n-th component of the signature of X is an element in the n-th tensor product of $\mathbb{R}^d$. The individual coordinate values of those tensors are given by the following.

Definition 2.4. For a multiindex $I = (i_1, \dots, i_n) \in \{1, \dots, d\}^n$, $n \in \mathbb{N}$, and $X \in \mathcal{V}_1([0,T], \mathbb{R}^d)$, we define the *coordinate iterated integral* of X over $[s,t] \subset [0,T]$ as

$$S^I_{s,t}(X) := \langle e_I, S_{s,t}(X) \rangle = \int_{s<u_1<\dots<u_n<t} \mathrm{d}X^{(i_1)}_{u_1} \dots \mathrm{d}X^{(i_n)}_{u_n} . \tag{2.5}$$

Remark 2.5. (i) Since we consider paths of finite 1-variation, the integrals in Definition 2.3 are understood in the sense of the Stieltjes integral. We have noted that the signature can be defined for less regular paths [49]. For example, if X is a $\mathbb{R}^d$-valued semimartingale, then the integrals can be understood as either the Itô integral or the Stratonovich integral. We refer to [16] for the definition of the signature for (paths of) continuous semimartingales in the context of finance and to [17, 20] for the more general setting of càdlàg paths.

(ii) For the truncated signature up to order N seen as vector in $\mathbb{R}^{d_N}$, its coordinate iterated integrals are ordered according to ordering of multiindices,

$$((1), (2), \dots, (d), (1,1), (1,2), \dots, (1,d), (2,1), \dots, (d,d), (1,1,1), \dots, (\overbrace{d, \dots, d}^{N-\text{times}})).$$

(iii) Note the difference between $S^n_{s,t}(X), S^{(N)}_{s,t}(X)$ and $S^I_{s,t}(X)$ for $n, N \in \mathbb{N}$ and a mutliindex I. The first denotes the n-th component of the signature of X as element in $(\mathbb{R}^d)^{\otimes n}$, the second one is the truncated signature up to order N in $T^{(N)}(\mathbb{R}^d)$, while the third is the real-valued coordinate iterated integral for multiindex I.

Definition 2.6. For $X \in \mathcal{V}_1([0,T], \mathbb{R}^d)$ we define its corresponding *time augmented* path as $\widehat{X}_t := (t, X_t)$, which is a $\mathbb{R}^{d+1}$-valued path of finite variation. The 0-th component corresponds to the time component, that is $\widehat{X}^{(0)}_t = t$.

Remark 2.7. One may alternatively define a time-augmented path $\widehat{X}_t$ with the first component given as the normalized time t/T, the remaining time $T - t$, as well as its normalized version $(T-t)/T$. The following Lemma 2.13 regarding the uniqueness of the signature still holds true for the time-augmentation given in any of those ways.

Example 2.8. (i) Consider $\widehat{X}_t = (t, X_t)$ for some $X \in \mathcal{V}_1([0,T], \mathbb{R})$. Its signature over $[s,t] \subset [0,T]$ is given as

$$S_{s,t}(\widehat{X}) = \Big(1, t-s, X_t - X_s, \frac{1}{2}(t-s)^2, \int_s^t u \, \mathrm{d}X_u \, , \int_s^t X_u \, \mathrm{d}u \, , \frac{1}{2}(X_t - X_s)^2, \\ \frac{1}{6}(t-s)^3, \frac{1}{2}\int_s^t (u_3 - s)^2 \, \mathrm{d}X_{u_3} \, , \dots \Big).$$

In general, for any path $X \in \mathcal{V}_1([0,T], \mathbb{R})$ and $I = (1, 1, \dots, 1)$ with $|I| = n$ we have

$$S^I_{s,t}(X) = \frac{(X_t - X_s)^n}{n!}.$$

(ii) Let $X : [0,T] \to \mathbb{R}^d$ be a linear path, that is $X_t = X_0 + t\frac{X_T - X_0}{T}$ for $X_0, X_T \in \mathbb{R}^d$. Since we have

$$\mathrm{d}X_u^{(i_j)} = \frac{X_T^{(i_j)} - X_0^{(i_j)}}{T}\,\mathrm{d}u \quad \text{and} \quad \int_{0<u_1<\cdots<u_n<t} \mathrm{d}u_1 \dots \mathrm{d}u_n = \frac{T^n}{n!},$$

for $i_j \in \{1, \dots, d\}$, the coordinate iterated integral of X for multiindex $I = (i_1, \dots, i_n)$ is

$$S_T^I(X) = \frac{1}{n!}\prod_{j=1}^{n} X_T^{(i_j)} - X_0^{(i_j)}. \tag{2.6}$$

With the map defined as $\mathbb{R}^d \ni a \mapsto \exp(a) := \sum_{n\geq 0} \frac{1}{n!} a^{\otimes n} \in T((\mathbb{R}^d))$, with $a^{\times 0} := 1$, the signature of X takes the simple form

$$S_T(X) = \exp(X_T - X_0). \tag{2.7}$$

The signature exhibits various key properties that make it useful in the reinforcement learning setting considered in this thesis. The first property, which is due to [14, 15], states that concatenation of paths corresponds to tensor multiplication of the paths' signatures.

Definition 2.9. For $0 \leq u \leq T$ let $X : [0,u] \to \mathbb{R}^d$ and $Y : [u,T] \to \mathbb{R}^d$ be two continuous paths. Their concatenation is the continuous path $X \star Y : [0,T] \to \mathbb{R}^d$ defined as

$$(X \star Y)_t := \begin{cases} X_t, & \text{if } t \in [0,u), \\ X_u + Y_t - Y_u, & \text{if } t \in [u,T]. \end{cases} \tag{2.8}$$

Lemma 2.10 (Chen's identity)**.** *For $u \in [0,T]$, and two paths $X \in \mathcal{V}_1([0,u],\mathbb{R}^d)$ and $Y \in \mathcal{V}_1([u,T],\mathbb{R}^d)$, it holds that*

$$S_T(X \star Y) = S_{0,u}(X) \otimes S_{u,T}(Y). \tag{2.9}$$

For the proof we refer to [49]. Lemma 2.10 has some implications. First, it allows for easy updating of the signature over an extended interval whenever new information about a path arrives, a property that is crucial in a reinforcement learning setting. Second, a path's signature over the whole interval can be pieced together by considering the path on subintervals, which indeed is the basis for explicitly computing the signature of a stream of data, see Section 2.2. Furthermore, Chen's identity states that the range of the map $X \mapsto S_T(X)$ is closed under multiplication, and since $S_T(X)_0 = 1$, it turns out that it is even a group.

Proposition 2.11 (Proposition 2.14 in [49])**.** *For $X \in \mathcal{V}_1([0,T],\mathbb{R}^d)$, let $\overleftarrow{X}$ be the path run backwards, that is $\overleftarrow{X}_t := X_{T-t}$ for $t \in [0,T]$. Then, for any $[s,t] \subset [0,T]$ it holds that $S_{s,t}(\overleftarrow{X}) = S_{s,t}(X)^{-1}$ and the range of the map $X \mapsto S_{s,t}(X)$ with the multiplication $\otimes$ on $T((\mathbb{R}^d))$ is a subgroup in $T((\mathbb{R}^d))$.*

Proposition 2.11 together with Chen's identity provides a way to obtain the signature of a shortened path by tensor multiplication with the inverse of the signature of the path over

a subinterval. More precisely for $0 \leq t \leq T$ we have

$$S_{t,T}(X) = S_{0,t}(X)^{-1} \otimes S_{0,T}(X).$$

The next property essentially states that the signature captures the order of events along a path, while ignoring the "speed" at which those events take place.

Proposition 2.12 (Invariance under time reparametrization)**.** *Let $X \in \mathcal{V}_1([0,T],\mathbb{R}^d)$ and $\varphi : [0,T] \to [0,T]$ be non-decreasing and surjective. Then, $S_T(X) = S_T(X \circ \varphi)$.*

This property can be an advantage in handling paths as input data in applications since identifying paths with different parametrization constitutes a substantial dimensionality reduction. If, however, the time parametrization is in fact important, as it is often the case with financial data, one can simply consider the time augmented path $\widehat{X}$ from Definition 2.6. Adding a time component is also a sufficient condition for the uniqueness of the signature.

Lemma 2.13 (Uniqueness)**.** *Let $X, Y \in \mathcal{V}_1^0([0,T],\mathbb{R}^d)$ and consider their time augmented paths $\widehat{X}_t$ and $\widehat{Y}_t$. Then, $S_T(\widehat{X}) = S_T(\widehat{Y})$ if, and only if $X_t = Y_t$ for each $t \in [0,T]$.*

In general, the signature of $X \in \mathcal{V}_1([0,T],\mathbb{R}^d)$ and of the translated path $X + c_1$ for some $c_1 \in \mathbb{R}^d$, agree, and the same holds true for $\widehat{X} + c_2$ and $\widehat{X}$ for any $c_2 \in \mathbb{R}^{d+1}$, however one can always consider $X_t - X_0$. In [24], the authors showed that the signature $S_T(X)$ of X without time-component, determines X up to so called tree-like equivalences.[3] Lemma 2.13 is a direct implication of this result, since adding time as strictly monotone component eliminates any tree-like sections of a path. The result was later extended to the more general rough path setting in [8].

The next property is the main ingredient for the signature's application in the approximation procedure in our reinforcement learning setting. It states that continuous functionals on paths can be approximated arbitrarily well by linear functionals on the paths' signature, uniformly in time. For a path $X \in \mathcal{V}_1([0,T],\mathbb{R}^d)$ we set $X_{[0,t]} := (X_s)_{s\in[0,t]}$ for its trajectory over the interval $[0,t]$, and analogue for its time-augmentation $\widehat{X}_{[0,t]} := (s, X_s)_{s\in[0,t]}$. Define the space of all time-augmented paths starting at the origin until $t \in [0,T]$ as

$$\Omega_t^0 := \{\widehat{X}_{[0,t]} \mid X \in \mathcal{V}_1^0([0,T],\mathbb{R}^d)\}.$$

which is a metric space equipped with the 1-variation distance $d_{1,[0,t]}$. Given $\widehat{X} \in \Omega_t^0$, we define the extended path $\widehat{X}^t$ on $[0,T]$ as

$$\widehat{X}_s^t := \begin{cases} \widehat{X}_s, & \text{if } s \in [0,t] \\ (s, X_t), & \text{if } s \in (t,T]. \end{cases} \tag{2.10}$$

Definition 2.14. We call $\Lambda_T := \bigcup_{t=0}^T \Omega_t^0$ the *space of stopped paths* and equip it with the metric

$$d_{\Lambda_T}(\widehat{X}_{[0,t]}, \widehat{Y}_{[0,s]}) := d_{1,[0,t]}(\widehat{X}_{[0,t]}, \widehat{Y}^s_{[0,t]}) + |t-s|, \tag{2.11}$$

[3] One can think of a tree-like path as one that retraces itself. For the exact definition in the bounded variation case see [24, Definition 1.2].

for any $\widehat{X}, \widehat{Y} \in \Lambda_T$, where we assume w.l.o.g. $s \leq t$, and $d_{1,[0,t]}$ is the 1-variation distance (2.2) $[0,t]$, and $\widehat{Y}^s$ is the extended path defined in (2.10).

Theorem 2.15 (Universal approximation)**.** *Let $K \subset \Omega_T^0$ be compact and $F : \Lambda_T \to \mathbb{R}$ be a continuous path functional. Then, for any $\epsilon > 0$ there exists a linear functional $\ell \in T(\mathbb{R}^{d+1})$ such that*

$$\sup_{\widehat{X} \in K} \sup_{t \in [0,T]} \left| F(\widehat{X}_{[0,t]}) - \langle \ell, S_t(\widehat{X}) \rangle \right| < \epsilon. \tag{2.12}$$

The theorem is proved in a more general form in [35] and [4]. The case of continuous paths of bounded 1-variation follows as special case.

2.2 The Signature of a Stream of Data and its Computation

So far we have been considering the signature of continuous paths on some interval $[0,T]$ for $T \in \mathbb{R}$. In practice, however, one usually is given a set of discrete data points. By embedding such a stream into a linear path, we can define the signature of a stream of discrete data, and we illustrate how it can be computed explicitly.

Following [9], we define the set of streams of data in $\mathbb{R}^d$ as

$$\mathcal{S}(\mathbb{R}^d) := \{\mathrm{x} = (x_0, x_1, \ldots, x_n) \mid x_i \in \mathbb{R}^d, n \in \mathbb{N}\}.$$

For $\mathrm{x} = (x_0, \ldots, x_n) \in \mathcal{S}(\mathbb{R}^d)$, its length is given by $|\mathrm{x}| = n+1$. We assume that there is a natural time series structure and the elements of $\mathcal{S}(\mathbb{R}^d)$ consist of sequential data points. If the data does not represent a time series, the methods in this section still apply, but one first has to decide on the order in which the points are visited. We embed a stream x into a continuous path with values in $\mathbb{R}^d$ by linear interpolation. For a stream $\mathrm{x} \in \mathcal{S}(\mathbb{R}^d)$ with $|\mathrm{x}| = n+1$, fix a time $T > 0$, and choose $t_0, t_1, \ldots, t_n \in [0,1]$ such that

$$0 = t_0 < t_1 < \cdots < t_n = 1.$$

If the stream's indices already correspond to time points the canonical choice would be $T = n$. We define the path $X : [0,T] \to \mathbb{R}^d$ as the linear interpolation of x,

$$X_t = \begin{cases} x_i, & \text{if } t \in \{t_0, \ldots, t_n\}, \\ x_i + (t - t_i)\dfrac{x_{i+1} - x_i}{t_{i+1} - t_i}, & \text{if } t \in (t_i, t_{i+1}) \text{ for } i \in \{0, \ldots, n-1\}. \end{cases}$$

Definition 2.16. Let $\mathrm{x} \in \mathcal{S}(\mathbb{R}^d)$ be a d-dimensional stream of data with length $|\mathrm{x}| = n+1$, and let $X : [0,T] \to \mathbb{R}$ be its linear interpolation. For $i, j \in \{0, 1, \ldots, n\}$ with $i \leq j$, we define the signature of x from i to j as

$$S_{i,j}(\mathrm{x}) := S_{t_i,t_j}(X). \tag{2.13}$$

The truncated signature up to order $N \in \mathbb{N}$ of x is defined analogously via X. If $i = 0$ we write $S_j(\mathrm{x}) = S_{0,j}(\mathrm{x})$ for any $j \in \{0, \ldots, n\}$.

Proposition 2.12 ensures that $S_{i,j}(\mathrm{x})$ is well defined, as it does not depend on the specific choice of the partition points t_i. Example 2.8(i) illustrates that the signature of a linear path takes a simple form as the exponential in $T((\mathbb{R}^d))$ of the total increment of the path. The continuous interpolation X of some $\mathrm{x} \in \mathcal{S}(\mathbb{R}^d)$ is just a concatenation of linear paths. Together with Chen's identity (2.9) we are provided with a way of explicitly computing $S_{t_i,t_j}(X)$.

Proposition 2.17. *Let* $\mathrm{x} \in \mathcal{S}(\mathbb{R}^d)$ *with length* $|\mathrm{x}| = n+1$. *Then, for any* $i, j \in \{0, 1, \ldots, n\}$ *with* $i < j$,

$$S_{i,j}(\mathrm{x}) = S_{t_i,t_j}(X) = \exp(x_{i+1} - x_i) \otimes \exp(x_{i+2} - x_{i+1}) \otimes \cdots \otimes \exp(x_j - x_{j-1}),$$

where for $a \in \mathbb{R}$,

$$\exp(a) := \sum_{n \geq 0} \frac{1}{n!} a^{\otimes n} \in T((\mathbb{R}^d)).$$

Proof. W.l.o.g. fix $i = 0$ and $j = n$. For any two consecutive points x_k, x_{k+1}, the path segment $X_{[t_k,t_{k+1}]}$ is linear, thus $S_{t_k,t_{k+1}}(X) = \exp(x_{k+1} - x_k)$. Since $X_{[0,T]}$ is the concatenation of all linear segments $\mathbf{x}_{[t_k,t_{k+1}]}$ for $k = 0, \ldots, n-1$, iterative application of Chen's identity yields the result. □

Remark 2.18. Proposition 2.17 illustrates that computing the signature for an incoming stream of data can be done in an efficient way. Assume that for $\mathrm{x} \in \mathcal{S}(\mathbb{R}^d)$ with length $|\mathrm{x}| = n+1$ we have computed $S_n(\mathrm{x})$, and additional data $\mathrm{x}' \in \mathbb{R}^d$ with $|\mathrm{x}'| = m+1$ arrives. Further, assume we have appended x_n to the beginning of x' such that $x_n = x'_0$. Instead of concatenating x and x', and computing the signature of the concatenation containing $n+m+1$ elements, one only needs to compute $S_m(\mathrm{x}')$ and tensor multiply it with $S_n(\mathrm{x})$ to obtain the signature of the full stream. This property proves to be of particular advantage in a reinforcement learning setting since there, new information arrives step-wise at each transition. Its applicability will be seen in Algorithm 2.

There are multiple Python libraries that provide functionalities for computing signatures such as the `iisignature` [61], `esig` [45], which was originally developed by Terry Lyons et al., as well as the `Signatory` [37], with the latter used in this work. Further computational details are provided for the experiments in Sections 6.2.2 and 6.3.4.

3 Reinforcement Learning and Markov Decision Processes

3.1 Reinforcement Learning

Reinforcement learning [71] is a machine learning paradigm where an algorithm, commonly referred to as an 'agent', learns to make a series of decisions by interacting with an evolving environment and adjusting its behaviour based on numeric feedback signals in the form of rewards or penalties. The goal is to maximize its cumulative performance over time by learning which actions yield the highest subsequent rewards, given a particular state or history of states the underlying environment was in. Environments can take various forms, ranging from the physical world when training robots to simulated environments like game simulators or replayed sequences of real-life events such as financial time series. A typical reinforcement learning setting is depicted in Figure 3.1. At step t, the agent receives the state from the environment and a reward associated with the transition from the previous step's state. At step t, the agent observes that the environment is in some state S_t. Based on this information and, potentially, information from previous steps, the agent calculates an action A_t, which it sends to the environment. Subsequently, the environment transitions to a new state S_{t+1} and emits a reward R_{t+1} associated with this transition. Both are sent back to the agent, who internally adapts its action selection mechanism based on the reward received, and the cycle repeats. This concept derives from how humans learn by trial

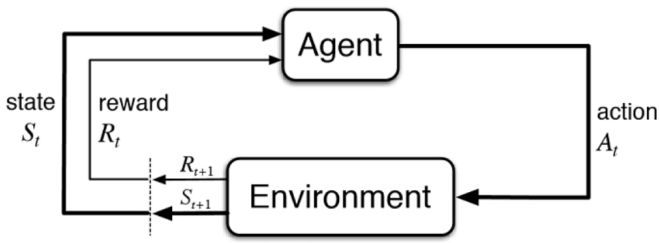


Figure 3.1: Reinforcement learning in a nutshell. [71, Figure 3.1]

and error. As we try out new behaviour and, in this way, interact with our environment, we receive feedback in various forms, such as gaining points in a game or experiencing physical sensations like pain or pleasure. Based on this feedback, we adjust our behaviour accordingly to achieve the desired outcome in the future.

Reinforcement learning differs from supervised and unsupervised learning, constituting a third machine learning paradigm [71, Section 1.1]. In supervised learning, a model is trained on a labelled data set of correct input-output examples, with the aim of generalizing the model's responses to input not included in the training set. While we can retrain a model

on new data, it is usually impossible to obtain correct examples in interactive, evolving settings, and thus, in reinforcement learning, an agent must learn without relying on some ground truth provided by an external supervisor. Although reinforcement learning does not rely on labelled data, it differs from the unsupervised learning paradigm in terms of its objective. While the latter aims to find structure in unlabelled data, reinforcement learning agents try to maximize a reward signal in a dynamic environment.

The usual formal model of the agent-environment framework depicted in Figure 3.1 is that of a *Markov Decision Process*, presented in the following Section 3.2. However, this work introduces a *history-dependent decision process* as an alternative model in Section 4.1. Aside from the agent-environment model, further essential aspects of reinforcement learning are the representation of states, the value function, the policy, the reward design, and the trade-off between exploration and exploitation.

State representation is crucial in reinforcement learning. States encapsulate any information available to the agent about the environment at certain times and serve as input to the agent's decision-making rule or policy. From a mathematical point of view, the state is often assumed to be the fully observable true state of the environment, as is the case in the Markov Decision Process framework in Section 3.2. Since, in practice, this is mostly not achievable, one can drop this assumption. Letting the agent only have access to a partial observation of the true environment state leads to a partially observable Markov decision process; see also Section 3.4. In the history-dependent framework proposed in Section 4.1, we work with observations, not distinguishing whether these represent fully or partially observable states.

The *reward signal* defines the objective in a reinforcement learning problem. At each step, the environment provides the agent with a single numeric value as a reward. By defining which state-action-state combinations are favourable or unfavourable, the reward signal serves as the primary basis for altering the agent's behaviour and guiding it towards maximizing the cumulative reward received. The function that produces the signal is generally stochastic and unknown to the agent and is usually part of the problem formulation. Thus, the agent's ability to successfully solve a problem hinges on the reward signal it provides.

While the reward signal indicates what is good in an immediate sense, the *value function* determines what is good in the long run. The value of a state represents its long-term desirability and is given as the expected total reward achievable in the future, taking into account states and actions that are likely to follow in the future. Decisions are made based on value estimates from observed states and actions chosen in those states. The agent chooses the actions it expects to lead to states with the highest value. Thus, efficiently estimating values is one of the most critical tasks in reinforcement learning. Value functions originally stem from the problem of optimal control of a dynamically evolving (stochastic) system. Its solution, using value functions and dynamic programming principles such as Bellman equations, serves as the foundation for many methods in reinforcement learning.

A *policy* defines the agent's decision-making rules and is a mapping from perceived states or sequences of states to actions. It is at the core of each reinforcement learning method and fully determines an agent's behaviour. A policy can be stochastic in that it outputs a probability distribution over actions. Policies can take on various forms. In value-based methods, the agent learns a value function from which a policy is derived by selecting actions that lead to states with the highest value. Such methods include the famous Q-learning

[81] or SARSA [71, Section 6.4]. In policy-based methods, the agent directly learns a parametrized mapping from states to actions instead of a value function. Examples include REINFORCE [82], Trust Region Policy Optimization [66], and state-of-the-art Proximal Policy Optimization [67]. Actor-critic methods are a mixture of learning a value function and a policy. The value function serves as a critic and provides a gradient direction for the policy update, while the policy as actor decides which action to choose next [41]. In this work, we propose a value-based method, which is an adaptation of the Q-learning algorithm for history-dependent environments.

Another key aspect and inherent challenge in reinforcement learning, distinct from other learning approaches, is navigating the *trade-off between exploration and exploitation*. To maximize rewards, an agent must prioritize actions previously found to be effective. However, discovering such actions requires exploring alternatives that have not yet been selected. Thus, the agent must balance exploiting known strategies for immediate reward and exploring new possibilities to improve future decision-making. It may achieve this by following a strategy of gradually favouring actions known to be good ones while maintaining a level of exploration. Especially in stochastic environments, repeated trials are essential to estimate the expected rewards of different actions accurately. The exploration-exploitation trade-off remains a complex issue, not present in other learning paradigms.

Reinforcement learning has diverse applications across various domains, ranging from robotics and gaming to finance and healthcare. Classic reinforcement learning methods have been instrumental in solving problems such as game playing, automated control systems, and scheduling tasks. In recent years, integrating reinforcement learning with deep learning has led to significant advancements in domains like computer vision, natural language processing, and autonomous vehicles. Deep reinforcement learning techniques, leveraging the power of neural networks, have demonstrated remarkable success in complex tasks such as image recognition, language understanding, and sequential decision-making. This connection of reinforcement learning with deep learning has opened up new possibilities for solving intricate problems that were previously challenging to address using traditional methods alone.

3.2 Markov Decision Processes

The usual mathematical framework for modeling a reinforcement learning problem is that of a Markov Decision Process (MDP). We give an introduction to this framework, characterize optimal policies, and illustrate the Q-learning method in MDPs. In Chapter 4, we will build on this exposition, generalize it to a history-dependent decision process, and adapt the Q-learning method to this history-dependent framework. Various modeling assumptions can be taken when defining an MDP. In view of the Q-learning method illustrated in the following section, we introduce an MDP with finite-time horizon, finite action space, and Borel state space. For a thorough introduction to MDPs we refer to the monograph [60] for finite and countable state and action spaces, and to [29] for the case where the involved spaces are allowed to be more general measurable spaces. If not mentioned otherwise, this exposition builds on Sections 2 and 4 in [60], as well as [72], and Chapter 16 in [29], where the latter reference guarantees the existence of an optimal deterministic Markovian policy

in the considered setup with general Borel state space.

A discrete-time, finite time-horizon, Markov Decision Process is given as the tuple

$$(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, T), \tag{3.1}$$

where its components are defined as follows. The *state space* $\mathcal{S}$ is the set of possible environment states. It is assumed to be a separable complete metric space $(\mathcal{S}, d_{\mathcal{S}})$, endowed with its Borel σ-algebra $\mathcal{B}_{\mathcal{S}}$. The *action space* $\mathcal{A}$ is a finite, non-empty set of actions with cardinality $|\mathcal{A}| \in \mathbb{N}$.[1] We may assume without loss of generality $\mathcal{A} = \{1, 2, \dots, |\mathcal{A}|\}$, endow $\mathcal{A}$ with the discrete metric, and denote the σ-algebra on $\mathcal{A}$ as $\mathcal{B}_{\mathcal{A}} = 2^{\mathcal{A}}$. If both $\mathcal{S}$ and $\mathcal{A}$ were finite, the tuple (3.1) is referred to as *finite* Markov Decision Process. Moreover, $T \in \mathbb{N}$ is the finite time-horizon at which the process terminates, if no other termination criterion was met before. The collection of *state transition kernels* is denoted by $\mathcal{P} = (P_t)_{t \in [T-1]}$. For each $t \in [T-1]$, the state transition kernel P_t is a stochastic kernel from $\mathcal{S} \times \mathcal{A}$ to $\mathcal{S}$, such that for any state-action pair $(s, a) \in \mathcal{S} \times \mathcal{A}$ and measurable set $S \in \mathcal{B}_{\mathcal{S}}$, $P_t(S \,|\, s, a)$ returns the probability that the next state is in S, provided the current state is s and action a is selected (more on this in Equation (3.8) below). The probability measure on $\mathcal{S}$ corresponding to the pair (s, a) is denoted $P_t(\,\cdot\,|s, a)$. Further, $\mathcal{R} = (R_t)_{t \in [T]}$ is a collection of real valued *reward functions*. For each $t \in [T-1]$ the reward function $R_t : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is measurable with respect to the product σ-algebra $\mathcal{B}_{\mathcal{S}} \otimes \mathcal{B}_{\mathcal{A}}$, and for $t = T$, the *terminal reward function* $R_T : \mathcal{S} \to \mathbb{R}$ is $\mathcal{B}_{\mathcal{S}}$-measurable. We assume that rewards are bounded by some $0 < C < \infty$, such that $|R_t(s, a)| \leq C$ for each $t \in [T-1]$ and any pair $(s, a) \in \mathcal{S} \times \mathcal{A}$, and $|R_T(s)| \leq C$ for any $s \in \mathcal{S}$. Finally, $\gamma \in [0, 1]$ is a discount factor, which weights future rewards.

An agent interacts with a given MDP in consecutive episodes at discrete steps. At the beginning of episode $k \in \mathbb{N}$, the process is in some state $s_0 \in \mathcal{S}$ sampled from a fixed initial distribution $\mu \in \mathrm{Prob}(\mathcal{S})$. Then, at each step $t \in [T-1]$ the agent observes the process' state s_t, and chooses an action $a_t \in \mathcal{A}$ based on some decision rule at t. As a result, the it receives a reward $R_t(s_t, a_t)$ and the process transitions to a new state s_{t+1}, which is sampled from the probability distribution $P_t(\cdot \,|\, s_t, a_t)$ on $\mathcal{S}$. The last action is taken at step $T-1$ after which the process reaches a terminal state s_T, and the agent receives the additional terminal reward $R_T(s_T)$ before the cycle starts anew in episode $k+1$. This behaviour produces a state-action history[2]

$$h_T = (s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T) \in (\mathcal{S} \times \mathcal{A})^T \times \mathcal{S}. \tag{3.2}$$

Definition 3.1. For a given MDP (3.1) and any $t \in [T]$, the space of histories up to time t is defined as

$$\mathcal{H}_t := (\mathcal{S} \times \mathcal{A})^t \times \mathcal{S},$$

with the convention $\mathcal{H}_0 = \mathcal{S}$ and elements $h_t = (s_0, a_0, s_1, a_1, \dots, s_{t-1}, a_{t-1}, s_t) \in \mathcal{H}_t$. Each

[1] More generally, the action space too can be assumed to be a separable, complete metric space without any restriction on cardinality, cf. [29]. However, dealing with general, continuous action spaces is more involved and in view of the computational method proposed in this work, not necessary.

[2] More specifically it produces an state-action-reward history, however we exclude the reward in (3.2) since in practice the reward part r_{t+1} can usually be derived from the transition tuple (s_t, a_t, s_{t+1}).

$\mathcal{H}_t$ is equipped with the Borel σ-algebra[3] $\mathcal{B}_{\mathcal{H}_t} = \bigotimes_{i=0}^{t}(\mathcal{B}_\mathcal{S} \otimes \mathcal{B}_\mathcal{A}) \otimes \mathcal{B}_\mathcal{S}$.

Remark 3.2. The rewards may also depend on the environment's state at the next step, such that they are determined by measurable reward functions $\tilde{R}_t : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ for $t \in [T-1]$, where $\tilde{R}_t(s, a, s')$ returns the reward received when the state of environment at time t is s, action a is chosen and the environment transitions to state s' at time $t+1$. In this case, we define the *expected immediate reward function* at $t \in [T-1]$ by

$$R_t(s, a) := \int_\mathcal{S} \tilde{R}_t(s, a, s') P_t(\mathrm{d}s' \,|s, a), \quad \text{for all } (s, a) \in \mathcal{S} \times \mathcal{A}. \tag{3.3}$$

Since $\tilde{R}_t$ is measurable, so is R_t for any $t \in [T-1]$ due to Lemma A.6, and the subsequent results in this section remain true if R_t is defined by (3.3). From a reinforcement learning perspective, it is irrelevant if the reward sample $r_t \in \mathbb{R}$ that the agent receives at time t, depends only on (s_t, a_t) or, additionally on the next state s_{t+1}, because we only require, the value r_t to be fixed before the agent selects the next action a_{t+1}, and that it does not depend on its choice.

We now formalize the concept of a policy which the agent uses to select actions.

Definition 3.3 (Policy in MDPs)**.** For an MDP, a policy is defined as the family $\pi = (\pi_t)_{t\in[T-1]}$, where we call π a

- *randomized policy*, if π_t is a stochastic kernel from $(\mathcal{S}, \mathcal{B}_\mathcal{S})$ to $(\mathcal{A}, \mathcal{B}_\mathcal{A})$ for each $t \in [T-1]$,
- *deterministic policy*, if π_t is a measurable map $\pi_t : \mathcal{S} \to \mathcal{A}$ for each $t \in [T-1]$.

The sets of all randomized and deterministic policies are denoted as Π^R and Π^D, respectively. Any $\pi \in \Pi^\mathrm{D}$ can be considered as a special case of a randomized policy by setting $\tilde{\pi}_t(\cdot|s) := \delta_{\pi_t(s)}(\cdot)$ for each $s \in \mathcal{S}$, thus $\Pi^\mathrm{D} \subset \Pi^\mathrm{R}$, and $\pi \in \Pi^\mathrm{R}$ always includes the case $\pi \in \Pi^\mathrm{D}$.

Policies in Definition 3.3 are also called *Markovian* policies (either deterministic or randomized), because π_t is a function of the current state s_t only, and we will see that those are sufficient for optimality in an MDP. In general, policies may be history-dependent (compare Definition 4.9 below).

A fixed policy $\pi \subset \Pi^\mathrm{R}$ together with an initial distribution $\mu \in \mathrm{Prob}(\mathcal{S})$ induces the canonical probability space

$$(\mathcal{H}_T, \mathcal{B}_{\mathcal{H}_T}, \mathbb{P}^\pi), \tag{3.4}$$

where the probability measure $\mathbb{P}^\pi$ is defined as in Appendix A.1, by the product of the initial distribution μ, and the stochastic kernels $\pi = (\pi_t)_{t\in[T-1]}$ and $\mathcal{P} = (P_t)_{t\in[T-1]}$, such that it satisfies

$$\begin{aligned}\mathbb{P}^\pi[\mathrm{d}h_T] = {} & P_{T-1}(\mathrm{d}s_T|s_{T-1}, a_{T-1})\, \pi_{T-1}(\mathrm{d}a_{T-1}|s_{T-1})\, P_{T-2}(\mathrm{d}s_{T-1}|s_{T-2}, a_{T-2}) \\ & \ldots P_0(\mathrm{d}s_1|s_0, a_0)\, \pi_0(\mathrm{d}a_0|s_0)\, \mu(\mathrm{d}s_0),\end{aligned} \tag{3.5}$$

where $h_T = (s_0, a_0, s_1, \ldots, s_T) \in \mathcal{H}_T$. Note, that the notation $\mathbb{P}^\pi$ omits the dependence on μ and $\mathcal{P}$, since both are fixed, to highlight, that the measure $\mathbb{P}^\pi$ is determined by the

[3] Since $\mathcal{S}$ is a separable metric space, the Borel σ-algebra on $\mathcal{H}_t$ induced by the product topology on $\mathcal{H}_t$ coincides with the product σ-algebra of the σ-algebras $\mathcal{B}_\mathcal{S}$ and $\mathcal{B}_\mathcal{A}$.

choice of π. For any history $h_T \in \mathcal{H}_T$, define the random states $S_t, t \in [T]$, and the random actions $A_t, t \in [T-1]$ as the coordinate maps on (3.4),

$$S_t(h_T) := s_t, \quad \text{and} \quad A_t(h_T) := a_t.$$

Note, that S_t and A_t are $\mathcal{S}$ and $\mathcal{A}$-valued random variables with their conditional probability distributions determined by the kernels P_t and π_t, respectively. The process $S_0, A_0, S_1, \ldots, A_{T-1}, S_T$ is just the canonical process on the probability space (3.4). Further, we define the history process by

$$H_0 := S_0, \quad \text{and} \quad H_t := (S_0, A_0, S_1, \ldots, S_t), \text{ for } t \in [T] \setminus \{0\},$$

where again, H_t is a random variable on (3.4) with values in $\mathcal{H}_t$. For any $t \in [T]$, history $h_t = (s_0, a_0, s_1, \ldots, s_t) \in \mathcal{H}_t$, Borel set $B \in \mathcal{B}_{\mathcal{S}}$, and action $a_t \in \mathcal{A}$, the measure $\mathbb{P}^\pi$ satisfies

$$\mathbb{P}^\pi[S_0 \in B] = \mu(B), \tag{3.6}$$

$$\mathbb{P}^\pi[A_t = a_t \,|\, H_t = h_t] = \mathbb{P}^\pi[A_t = a_t \,|\, S_t = s_t] = \pi_t(a \,|\, s_t), \tag{3.7}$$

$$\mathbb{P}^\pi[S_{t+1} \in B \,|\, H_t = h_t, A_t = a_t] = \mathbb{P}^\pi[S_{t+1} \in B \,|\, S_t = s_t, A_t = a_t] = P_t(B \,|\, s_t, a_t). \tag{3.8}$$

Equation (3.8) is the Markov property, which states that the distribution of the next state S_{t+1} under $\mathbb{P}^\pi$ depends on the history past states and actions only through the current values s_t and a_t. If $\pi \in \Pi^{\mathrm{D}}$, then for any $t \in [T-1]$, $A_t := \pi_t(S_t)$, the right-hand side in Equation (3.7) is $\delta_{\pi_t(s_t)}(a_t)$, and Equation (3.5) changes to

$$\begin{aligned}\mathbb{P}^\pi[\mathrm{d}h_T] = {} & P_{T-1}(\mathrm{d}s_T|s_{T-1}, a_{T-1})\delta_{\pi_{T-1}(s_{T-1})}(\mathrm{d}a_{T-1})P_{T-2}(\mathrm{d}s_{T-1}|s_{T-2}, a_{T-2}) \\ & \ldots P_0(\mathrm{d}s_1|s_0, a_0)\delta_{\pi_0(s_0)}(\mathrm{d}a_0)\mu(\mathrm{d}s_0).\end{aligned}$$

Additionally, the state process $(S_t)_{t\in[T]}$ is an $\mathcal{S}$-valued Markov chain with initial distribution μ and transition kernels $P_{\pi_t}(s, ds') := P_t(\mathrm{d}s'|s, \pi_t(s))$.

Remark 3.4. In the rest of Section 3.2 and the following Section 3.3 we only consider policies $\pi \in \Pi^{\mathrm{D}}$, because Theorem 3.5 below shows that the deterministic policies are sufficient to obtain optimality.

Following a policy π, the agent receives a sequence of rewards, which, from it's point of view, is regarded as random. The agent's objective is to find a policy that maximizes the expected value of the resulting random reward sequence. Value functions allow to make this statement precise. Fix a policy π and $t \in [T]$. The *value function* at time t, corresponding to π is defined as

$$V_t^\pi(s) := \mathbb{E}^\pi\Big[\sum_{u=t}^{T-1} \gamma^{u-t} R_u(S_u, A_u) + \gamma^{T-t} R_T(S_T) \,\Big|\, S_t = s\Big], \quad \text{for } s \in \mathcal{S}, \tag{3.9}$$

and as $V_T^\pi(s) := R_T(s)$ for any $s \in \mathcal{S}$ if $t = T$, where it is understood that π is followed from t onwards. The notation $\mathbb{E}^\pi$ indicates, that the conditional expectation is taken under $\mathbb{P}^\pi$. Note, that for $t > 0$, V_t^π is a function of s only, because of the Markovian structure of both P_t and π_t. $V_0^\pi(s)$ gives the *expected total discounted reward* over the whole decision

making horizon if the agent follows policy π and the environment starts in state s. For any $t \in [T]$, we define the *optimal value function* of the MDP (3.1) at time t as

$$V_t^*(s) := \sup_{\pi \in \Pi^R} V_t^\pi(s), \qquad \text{for } s \in \mathcal{S}, \tag{3.10}$$

and as $V_T^*(s) := R_T(s)$ if $t = T$. The value $V_t^*(s)$ corresponds to the maximal attainable expected reward over the remaining decision horizon and is also referred to as the optimal value given state s at time t. Because of the boundedness assumption for reward functions, $V_t^\pi(s)$ exists and is bounded by a constant independent of π, for all $s \in \mathcal{S}$, and all $t \in [T]$, and, thus, the same holds true for holds true for $V_t^*(s)$ for any $s \in \mathcal{S}$ and all $t \in [T]$. Furthermore, since the functions R_T, R_t, and π_t are measurable for all $t \in [T]$, and the composition of measurable functions is again measurable, V_t^π is a measurable function from $\mathcal{S}$ to $\mathbb{R}$ for each $t \in [T]$ according to Lemma A.6. A priori V_t^* may not be measurable, since the supremum in (3.10) is taken over an uncountable family of measurable functions, which need not be measurable. However, with our assumption $|\mathcal{A}| < \infty$, the optimal value functions V_t^* are indeed measurable, which follows from the fact, that the supremum in (3.10) equals the maximum over the finite set of all possible action sequences $\{a_i\}_{i=0}^{T-1}$. Additionally, $\mathbb{E}^\pi[|V_t^*(S_t)|] < +\infty$ for all $t \in [T]$ and the functions V_t^* satisfy the so called *Bellman optimality equations*,

$$V_t^*(s) = \max_{a \in \mathcal{A}} \Big\{ R_t(s,a) + \gamma\, \mathbb{E}_{P_{t+1}(\cdot|s,a)} \big[V_{t+1}^*(S_{t+1}) \big] \Big\}, \tag{3.11}$$

for all $s \in \mathcal{S}$ and $t \in [T-1]$, with terminal condition

$$V_T^*(s) = R_T(s), \tag{3.12}$$

where $\mathbb{E}^\pi_{P_{t+1}(\cdot|s,a)}$ is the conditional expectation of $V_t^*(S_{t+1})$ given $S_t = s, A_t = a$, which determined by the transition kernel P_t, as in Equation (A.7). Any solution to the non-linear system of equations (3.11) and (3.12) is a collection of optimal value functions for the underlying MDP.[4] With the notion of value functions, we define optimality of a policy. A policy $\pi^* \in \Pi^{\mathrm{R}}$ is called an *optimal policy* if,

$$V_0^{\pi^*}(s) \geq V_0^\pi(s), \quad \text{for all } s \in \mathcal{S}, \text{ and } \pi \in \Pi^{\mathrm{R}}. \tag{3.13}$$

The corresponding value functions for an optimal policy π^* satisfy

$$V_t^{\pi^*}(s) = V_t^*(s), \qquad \text{for all } s \in \mathcal{S}, \text{ and } t \in [T]. \tag{3.14}$$

Specifically, $V_0^{\pi*}(s) = V_0^*(s)$ for any $s \in \mathcal{S}$, and π^* achieves the maximal attainable expected discounted reward over the whole decision period. In general, an optimal policy may not exist under all modeling assumptions. However, when $\mathcal{A}$ is finite, an optimal policy does indeed exist in the class of deterministic policies, in other word, the supremum in (3.10) is attained for some $\pi \in \Pi^{\mathrm{D}}$.

[4] Suppose $(v_t)_{t\in[T]}$ is a solution of Equations (3.11) and (3.12), then $(v_t + c)_{t\in[T]}$ for any $c \in \mathbb{R}$ is also a solution. Thus, the optimal value function of a finite-horizon MDP given by Equation (3.1) is not unique.

Theorem 3.5. *For any finite-horizon Markov Decision Process given by* (3.1)*, there exits a deterministic Markovian policy that is optimal.*

Proof. Proposition 4.4.3 in [60] proves the case with finite or countable $\mathcal{S}$. For general, Borel state space, [29, Theorem 16.1.6] shows the existence of an optimal policy for stationary Markov Decision Processes, where $P_t \equiv P$ and $R_t \equiv R$ for all $t \in [T-1]$ for some fixed state transition kernel P and reward function R, with the help of the so called Structure Theorem, Theorem 16.1.12 therein. The non-stationary case, follows from the non-stationary version of the Structure Theorem, Theorem 16.1.28 therein. □

From the Bellman equations (3.11) and (3.12), and the fact that the value functions of an optimal policy satisfy (3.14), it is evident that for a policy to be optimal it must select actions that attain the maximum on the right hand side of (3.11) at each $t \in [T-1]$, thus,

$$\pi_t^*(s) \in \arg\max_{a \in \mathcal{A}} \Big\{ R_t(s,a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}s_{t+1}|s,a)} \big[V_{t+1}^*(S_{t+1}) \big] \Big\}. \tag{3.15}$$

A policy which satisfies (3.15) for all $t \in [T-1]$ and $s \in \mathcal{S}$ is called a *maximizing* policy. The measurable selection theorem in Appendix A.2, guarantees the measurability of a maximizing policy π_t^* at each t, thus it is indeed a deterministic policy according to Definition 3.3.

The problem of optimal control of a Markov Decision Process is concerned with computing an optimal policy when the transition kernels and rewards functions are known. This can be done through dynamic programming by recursively solving the Bellman equations (3.11), (3.12) to get optimal value functions, and subsequently, choosing for each state a policy that maximizes the right hand side of (3.11). There exists vast literature on solving MDPs, and on the existence of optimal policies with varying modeling assumptions, we refer to [60, 29, 19] for some monographs on this matter. In contrast to the optimal control of an MDP, reinforcement learning is concerned with learning an optimal or approximately policy without the knowledge of $\mathcal{P}$ and $\mathcal{R}$, solely from sampled transitions (s_t, a_t, r_t, s_{t+1}). In this case, the rewards and expectations in (3.11) cannot be computed explicitly. Nevertheless, the Bellman optimality equations serve as basis for many RL algorithms, including Q-Learning which is presented in the following Section 3.3.

3.3 Q-Learning in Markov Decision Processes

When reward functions R_t and state transition kernels P_t are unknown, and the expectation in (3.11) can not be computed explicitly, exact dynamic programming methods are not applicable. In Q-Learning, introduced in [81], the agent interacts with the environment modeled as MDP, to collect transition samples (s_t, a_t, r_t, s_{t+1}). Those are subsequently used in a sample based version of the Bellman equations (3.11), (3.12) to learn so called Q-functions from which an optimal policy is derived. Note, hat most existing work on Q-Learning focuses on the infinite-horizon, stationary case, where transition kernels and reward functions do not depend on time. As a result, Q-functions are time independent, and optimal policies are stationary. In contrast, this section describes Q-Learning for a finite-horizon, non-stationary MDP given by the tuple Equation (3.1), where both, policies and Q-functions retain the time-dependence.

For an MDP given by (3.1), the *optimal Q-function* at time $t \in [T-1]$ is defined as

$$Q_t^*(s,a) := R_t(s,a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}s_{t+1}|s,a)}\big[V_{t+1}^*(S_{t+1})\big], \quad \text{for } (s,a) \in \mathcal{S} \times \mathcal{A}, \tag{3.16}$$

and for $t = T$ as $Q_T^*(s,a) := R_T(s)$ for $s \in \mathcal{S}$, and we refer to the collection $(Q_t^*)_{t\in[T]}$ as the optimal Q-functions. The function Q_t^* is also known as the optimal *action-value function* at time t, because it provides the optimal value over the remaining episode if the process is in state s at time t, and action a is selected. Note, that since R_t is measurable, and the map $(s,a) \mapsto \mathbb{E}_{P_t(\cdot|s,a)}[V_{t+1}^*(S_{t+1})]$ is measurable according to Lemma A.6, $(s,a) \mapsto Q_t^*(s,a)$ is a measurable map from $\mathcal{S} \times \mathcal{A}$ to $\mathbb{R}$. Additionally, it is bounded.

The optimal value functions and optimal Q-functions are related in the following way:

$$V_t^*(s) = \max_{a\in\mathcal{A}} Q_t^*(s,a), \quad s \in \mathcal{S}. \tag{3.17}$$

If we plug (3.17) into (3.11), we get the *Bellman equations for optimal Q-functions*,

$$Q_t^*(s,a) = R_t(s,a) + \gamma\, \mathbb{E}_{P_{t+1}(\cdot|s,a)}\Big[\max_{\alpha\in\mathcal{A}} Q_{t+1}^*(S_{t+1},\alpha)\Big], \quad t \in [T-1], \tag{3.18}$$

with the terminal condition

$$Q_T^*(s,a) = R_T(s), \tag{3.19}$$

for any $(s,a) \in \mathcal{S} \times \mathcal{A}$. Since $\mathcal{A}$ is finite, for any $s \in \mathcal{S}$, the maximum in (3.17) is attained. Thus, according to the measurable selection theorem in Appendix A.2, for any $t \in [T]$, the map $s \mapsto \max_{a\in\mathcal{A}} Q_t^*$ is measurable, and the conditional expectation $\mathbb{E}_{P_{t+1}(\cdot|s,a)}$ in (3.18) is well-defined. Analogous to value functions, any solution of the non-linear system of equations (3.18) and (3.19), is a collection of optimal action-value functions for the underlying MDP. Since an optimal deterministic policy π^* satisfies (3.15), it follows directly from the definition of the Q-functions that

$$\pi_t^*(s) \in \arg\max_{a\in\mathcal{A}} Q_t^*(s,a), \quad \text{for all } s \in \mathcal{S}, t \in [T-1]. \tag{3.20}$$

We call a policy that maximizes $Q_t^*(s,\cdot)$ for each $s \in \mathcal{S}$ greedy with respect to Q_t^*. A policy that is greedy with respect to Q_t^* for all $t \in [T-1]$ is called *greedy with respect to the collection* $(Q_t^*)_{t\in[T]}$. Equation (3.20) implies that knowledge of the optimal Q-functions is sufficient for acting optimally in the underlying MDP, by following a policy that is greedy with respect to $(Q_t^*)_{t\in[T]}$.

The question of how to find the functions Q_t^* in a computational manner without the knowledge P_t and R_t was considered by Watkins in [80]. The author proposed the Q-learning algorithm, a sample based version of the Bellman equations (3.18), where observed transition tuples (s_t, a_t, r_t, s_{t+1}) are used to update estimates of the functions Q_t^*. We present the method for a finite-horizon, non-stationary MDP (3.1), where additionally to $|\mathcal{A}| < +\infty$, we assume $|\mathcal{S}| < +\infty$, a setting that is considered in [79]. Then, at each step $t \in [T]$, the optimal Q-function Q_t^* can be represented as a $|\mathcal{S} \times \mathcal{A}|$-matrix, with its entries given as the optimal Q-values $Q_t^*(s,a) \in \mathbb{R}$ for pairs $(s,a) \in \mathcal{S} \times \mathcal{A}$. Q-learning starts with some arbitrary initial value $Q_t^0(s,a) \in \mathbb{R}$ as an estimate for $Q_t^*(s,a)$ for each pair $(s,a) \in \mathcal{S} \times \mathcal{A}$,

and each $t \in [T]$. Let $Q_t^n(s,a)$ be the estimate of $Q_t^*(s,a)$ after the n-th update iteration. Assume that, by interaction with the environment, the agent has observed the transition sample (s_t, a_t, r_t, s_{t+1}) at $t \in [T-1]$, where s_t is the state at t, a_t the action selected in this state, $r_t = R_t(s_t, a_t)$ is the reward the agent received, and s_{t+1} is the new state to which the environment transitioned to, sampled from $P_t(\cdot|s_t, a_t)$. Then, for each $(s,a) \in \mathcal{S} \times \mathcal{A}$, we update the current estimate $Q_t^n(s,a)$ by

$$Q_t^{n+1}(s,a) \leftarrow Q_t^n(s,a) + \alpha_n \left[r_t + \gamma \max_{a' \in \mathcal{A}} Q_{t+1}^n(s_{t+1}, a') - Q_t^n(s,a) \right] \times \mathbb{1}_{\{s,a\}}(s_t, a_t), \quad (3.21)$$

where $\alpha_n \in (0,1]$ is the learning rate for update iteration $n+1$. For the observed pair (s_t, a_t), it balances the value of the new estimate $Q_t^{n+1}(s_t, a_t)$, between the current estimate $Q_t^n(s_t, a_t)$ and the target value $r_t + \gamma \max_{a' \in \mathcal{A}} Q_{t+1}^n(s_{t+1}, a')$, which resembles a sample of the Bellman equation (3.18), and is a supposedly better estimate of the true value $Q_t^*(s_t, a_t)$ due to r_T being the actual reward received from the environment. This process of learning estimates from other estimates is called bootstrapping. Note that, if we have observed the specific pair (s_t, a_t), then for any other pair $(s,a) \in \mathcal{S} \times \mathcal{A} \setminus \{(s_t, a_t)\}$, we have $Q_t^{n+1}(s,a) = Q_t^n(s,a)$. If $t = T-1$, then the agent additionally receives terminal reward $r_T = R_T(s_T)$, which serves as target to update terminal estimates by

$$Q_T^{n+1}(s,a) \leftarrow Q_T^n(s,a) + \alpha_n \left[r_T - Q_T^n(s,a) \right] \times \mathbb{1}_{\{s\}}(s_T), \quad (3.22)$$

for all $s \in \mathcal{S}$, and $a \in \mathcal{A}$.

Transition samples are obtained by following some fixed exploration policy $\pi \in \Pi^{\mathrm{R}}$. The full Q-learning algorithm for an MDP (3.1), with finite state and action spaces is provided in Algorithm 1. Using stochastic approximation arguments, the authors of [79] proved, that $Q_t^n(s,a)$ converges to $Q_t^*(s,a)$ for $n \to \infty$ almost surely for each pair $(s,a) \in \mathcal{S} \times \mathcal{A}$, and all $t \in [T]$, provided the learning rates satisfy the so-called Robbins-Monro conditions [63],

$$\sum_{n=0}^{\infty} a_n = \infty, \qquad \sum_{n=0}^{\infty} a_n^2 < \infty,$$

and each pair (s,a) is sampled infinitely often at each time $t \in [T-1]$.[5] [81] proved convergence for the case of an infinite-horizon, stationary MDP.

Remark 3.6. Q-learning is an *off-policy* reinforcement learning method, where the policy that is learned is different from the exploration policy π that generates transition samples. In general, transition samples could be collected before learning estimates takes place. However, the exploration policy must guarantee sufficient *exploration* of the state space to ensure learning of correct estimates and, theoretically, visit each pair (s,a) infinitely often. A popular choice as exploration policy is ε-greedy, with $\varepsilon \in [0,1]$. At each step $t \in [T-1]$, this policy if greedy w.r.t to the current estimate Q_t^n with probability $1-\varepsilon$, and samples an action uniformly from $\mathcal{A}$ otherwise. To ensure sufficient *exploitation* of estimates already learned, the value of exploration parameter ε is usually gradually reduced over the training period.

[5] [79] does not explicitly mention the second condition, however, the used sampling policy implicitly guarantees it.

Algorithm 1 Q-learning for finite-horizon, non-stationary, finite MDPs

Require: Total number of iterations $N \in \mathbb{N}$, exploration policy π, learning rates $\{\alpha_n\}_{n=0}^{N-1}$

```
Initialize Q_t^0(s, a) for all (s, a) ∈ S × A, and t ∈ [T].
Sample s_0 ∈ S from initial distribution μ.
for iteration n = 0, 1, 2, ..., N − 1 do
    Set t = n mod T.
    Sample action a_t from π_t(·|s_t).
    Receive reward r_t and new state s_{t+1}, after taking a_t.
    Update estimates Q_t^n(s, a) for all (s, a) according to (3.21).
    if t = T − 1 then
        Receive terminal reward r_T.
        Update Q_T^n(s, a) for all (s, a) according to (3.22).
    end if
end for
```

Algorithm 1 hinges on the assumption that $\mathcal{S}$ is finite. When the cardinality of $\mathcal{S}$ is infinite or large, such that storing an array of size $|\mathcal{S}| \times |\mathcal{A}| \times T$ becomes infeasible, one has to resort to function approximation to estimate optimal Q-values. Consider a family of parametrized functions $\mathcal{Q} = \{Q_\theta \mid \theta \in \mathbb{R}^m\}$ for fixed $m \in \mathbb{N}$, with $Q_\theta : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ for any parameter vector θ. For $Q_\theta \in \mathcal{Q}$ and a pair $(s, a) \in \mathcal{S} \times \mathcal{A}$ we write $Q(s, a; \theta) := Q_\theta(s, a)$. For each time $t \in [T]$, we approximate the optimal Q-function Q_t^* by

$$Q_t^*(s, a) \approx Q(s, a; \theta_t), \quad \text{for all } (s, a) \in \mathcal{S} \times \mathcal{A}, \tag{3.23}$$

with parameter vector $\theta_t \in \mathbb{R}^m$, and we replace iteration (3.21) by a suitable iteration to find a parameter vector $\theta_t^*, t \in [T]$, that yields the best approximation of Q_t^* in $\mathcal{Q}$. Analogue to the finite case, Q-learning with function approximation starts with some arbitrary initial parameter vector θ_t^0 for each $t \in [T]$, which provides initial estimates $Q(s, a; \theta_t^0)$. Let θ_t^n be parameter vector at step t, after update iteration $n \in \mathbb{N}$. Upon observing the transition tuple (s_t, a_t, r_t, s_{t+1}) at time $t \in [T-1]$, where again $r_t = R_t(s_t, a_t)$, and s_{t+1} is sampled from $P_t(\cdot|s_t, a_t)$, we define the target value

$$y_t^n := r_t + \gamma \max_{a \in \mathcal{A}} Q(s_{t+1}, a; \theta_t^n), \tag{3.24}$$

and update the current approximation given by θ_t^n towards the target value y_t^n via single-step stochastic gradient descent, minimizing the squared loss[6]

$$\mathcal{L}(y_t^n, \theta_t^n) = \tfrac{1}{2}\big(y_t^n - Q(s_t, a_t; \theta_t^n)\big)^2. \tag{3.25}$$

This corresponds to updating the parameter vector θ_t^n to

$$\theta_t^{n+1} \leftarrow \theta_t^n + \alpha_n \Big[r_t + \gamma \max_{a \in \mathcal{A}} Q(s_{t+1}, a; \theta_{t+1}^n) - Q(s_t, a_t; \theta_t^n)\Big] \nabla_\theta Q(s_t, a_t; \theta)\Big|_{\theta = \theta_t^n} \tag{3.26}$$

[6] Other loss functions may be applied, as for example the smooth-L_1 loss function used in the experiments in Chapter 6. The loss function's gradient in (3.25) changes accordingly.

with learning rate $\alpha_n \in (0, 1]$ in update iteration $n + 1$. The algorithm for Q-learning with functions approximation is analogue to Algorithm 1, with the finite-case update (3.21) replaced by the parameter vector update (3.26).

A typical choice for the family $\mathcal{Q}$ is the class of neural networks, such that $Q(s, a; \theta) = F(s, a; \theta)$ where F is a multilayer-perceptron parametrized by $\theta = (\boldsymbol{W}, \boldsymbol{b})$ with a collection of weight matrices $\boldsymbol{W}$ and bias vectors $\boldsymbol{b}$. Neural Fitted Q-learning proposed in [62] applies a shallow neural network approximation in an off-line fashion, with pre-collected sets of transition samples. In [54], the authors introduced Deep Q-Network, where a more complex deep neural network is used to estimate the optimal Q-function (stationary case), together with the use of a separate target neural network for action selection, and experience reply to enhance learning performance. Their approach demonstrated impressive learning and generalization results across a wide range of Atari 2600 computer games [5], using the same learning network configuration and hyperparameter values across games.

Another common approach is to set $\mathcal{Q}$ to be the linear span of some set of feature maps $\phi_k(s, a) : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ for $k = 1, \dots, m$, resulting in linear approximations,

$$Q(s, a; \theta) = \sum_{k=0}^{m} \phi_k(s, a)\theta_k = \phi(s, a)^\top \theta$$

with the feature vector $\phi(s, a) = (\phi_1(s, a), \dots, \phi_m(s, a))^\top$, and the parameter vector $\theta = (\theta_1, \theta_2, \dots \theta_m)^\top$. The paper [52] established conditions under which Q-Learning with linear function approximation converges, these including that the exploration policy induces the state process $(S_t)_{t \geq 0}$ to be an ergodic Markov chain, and that the feature maps ϕ_k are linearly independent. With the goal of weakening those conditions, [13] provides a convergent two-time scale variant of Q-learning with linear function approximation with less stringent assumptions on the exploration policy, as well as error bound for the method. Note that the mentioned references on Q-learning with linear function approximation [52, 13], and on Q-learning with neural network approximation [54, 62], assumed an infinite-horizon, stationary MDP as environment model. Nevertheless, the approaches can be readily applied to the finite-horizon, non-stationary case by considering separate estimates for each time t, however, convergence results and sampling efficiency would have to be reassessed. In this work we take a similar approach to Q-learning with linear function approximation in MDPs, in that we also use linear functions of some feature maps to approximate optimal Q-function, albeit in a history-dependent setting. Here, the feature maps have to be functions of the observed history $h_t = (s_0, a_0, s_1, \dots, a_{t-1}, s_t)$, and are given by the signature of a path, introduced in Chapter 2.

3.4 The Issue with Markov Decision Processes as a Modeling Framework

Reinforcement learning problems are typically formalized as Markov decision processes, where states satisfy the Markov property (3.8), and the information contained in the current state is sufficient for the agent to learn an optimal policy. This formalization however, implicitly assumes that the true underlying environment indeed permits to be modeled as

such, and that the model captures all the relevant information. In many scenarios this assumption may not be true, and thus, modeling the environment as MDP is problematic. We highlight three cases.

First, the environment might be described better as an n-step Markov decision process for some $n \in \mathbb{N}$. The difference to MDPs defined in Section 3.2 is that the distribution of the state S_{t+1} depends on states and action at the prior steps $t, t-1, \dots, t-n$, such that

$$\mathbb{P}[S_{t+1} \in B \,|\, H_t = h_t] = \mathbb{P}[S_{t+1} \in B \,|\, S_{t-k} = s_{t-k}, A_{t-k} = a_{t-k}, \dots S_t = s_t, A_t = a_t],$$

for any Borel set $B \in \mathcal{B}_{\mathcal{S}}$, with history $h_t = (s_0, a_0, s_1, \dots, a_{t-1}, s_t)$. When the true environment follows an n-step MDP, one can construct an 1-step MDP by defining new states as

$$\tilde{s}_t := (s_{t-k}, a_{t-k}, \dots, a_{t-1}, s_t).$$

and corresponding transition kernels $\tilde{P}_t(\mathrm{d}\tilde{s}_{t+1} \,|\tilde{s}_t, a_t)$.[7] The random states $\tilde{S}_{t+1}$ exhibit the Markov property (3.8) w.r.t. to $\tilde{s}_t$ and a_t. For example, in [54], the Deep Q-Network trained to play Atari 2600 games, receives the last $n = 4$ frames as input with selected actions repeated during those frames, and the model for the games considered becomes Markovian. Other games, however, would require more than four frames, and are still non-Markovian w.r.t to the newly constructed states. This shows that, even with the knowledge that an environment is an n-step MDP for *some* n, selecting the correct n still poses an issue.

Second, in real-world environments it is seldom possible to determine the full true state, let alone provide it to the agent, leading to partial observability. Such environments are modeled as *partially observable Markov Decision Processes* (POMDP) [34], which are defined as an MDP given by (3.1), together with an additional measurable space, *observation space* $\mathcal{O}$, and a collection of *observation kernels* $(O_t)_{t\in[T]}$. While the true states $s \in \mathcal{S}$ of an POMDP satisfy the Markov property (3.8), this structure is hidden. Assume that at time t the process is in state $s_t \in \mathcal{S}$. Instead of state s_t, the agent receives only a partial observation $o_t \in \mathcal{O}$, which is generated according to the distribution $O_t(\mathrm{d}o|s_t)$. Reinforcement learning in POMDPs is known to be a computationally as well as statistically challenging task. In the worst case, learning a near-optimal policy may require a sample size exponential in episode length [42]. Although the true underlying environment is an MDP, incomplete state information prohibits modeling it as such. In terms of Q-learning this means that in general $Q(o, a; \theta_t) \neq Q(s, a; \theta_t)$ for any Q-function approximation parametrized by θ_t, with s being the true state and o the observation received. Estimating Q-values as in Section 3.3 simply with states replaced by observations can be arbitrarily bad. Coming back to the Atari 2600 games as an example, with the number of frames less than four, the games considered in [54] are partially observable environments. Specifically, in the game "Pong", the agent requires both, the position and velocity of a ball to infer an optimal paddle move. However, with only one frame as state, the agent receives just the ball's position, with no means to infer its velocity, and can thus not determine an optimal paddle move as action. In other cases, computational constraints could prevent to store the true state, if it was fully observable,

[7] The new state transition kernels $\tilde{P}_t$ are determined by the true transition kernels P_t which are functions of the last n states and action. Since we consider the constructed 1-step MDP in a reinforcement learning set-up we do not need $\tilde{P}_t$ exactly.

and some compact state representation has to be employed which might not include all necessary information for the model to be an MDP. This again shows the importance of state representation in reinforcement learning as discussed at the beginning of this chapter.

Finally, an environment may simply exhibit dependence on the past, in that the distribution of the state at time $t+1$, and possibly also the reward, depend on the whole state-action history h_t, or parts of it at specific steps prior to $t+1$. In such environments it is required to maintain information over many steps and they are also referred to as long-term memory tasks. They can be either fully or partially observable and subsume n-step MDPs and POMDPs mentioned above. In [28], the authors considered reinforcement learning in such environments using recurrent neural networks to tackle the memory requirements. One of the problems considered is to control a 3-joint reacher which must hit a randomly placed target, but the targets position is only provided in the very first observation. Thus, the probability of hitting a target state at $t+1$ depends on the observation or state at time $t=0$ and the agent agent has to memorize the position throughout the whole episode.

In all three cases, the problem's nature does not admit an MDP as model. By only considering the current state, important contextual information and dependencies between previous states and actions are overlooked and valuable information that can be derived from considering the agent's history is ignored. This can lead to sub-optimal decision-making as the agent may not be able to effectively capture and utilize all relevant information for maximizing long-term rewards. As a result, history dependence has to be taken into account in the modeling and learning process. To this end, we introduce a history dependent environment in the following Chapter 4, which assumes no Markovian structure in the first place, and propose a method for learning in such environments in Chapter 5.

4 A History-Dependent Reinforcement Learning Environment

In this chapter we generalize the exposition outlined in Sections 3.3 and 3.4 for MDPs to a history-dependent reinforcement learning environment. We introduce a history-based decision process (HDP) as an alternative modeling framework, describe its components, and further define (optimal) value functions and optimality of a policy. No Markov assumptions on the dynamics are made, and the distribution of future observations and rewards may depend on the full history prior to the current time. This leads to the necessity to adapt policies and optimal value functions, and define them in terms of histories.

Unless otherwise stated, the definitions and results in this chapter are, to the best of our knowledge, new, but of course, they build on existing work from the literature. The definition of HDPs as a general RL framework is inspired by [51], which extends convergence of Q-learning from MDPs to a subset of non-MDPs, so called Q-value Uniform Decision Processes, and further by [32], which is related to Feature-RL [30], and concerned with aggregating histories to states such that the aggregated process solves the original problem. Both works are discussed in the literature review in Chapter 1. The approach of expressing the conditional expectations that define (optimal) value functions as integrals w.r.t. product kernels, and to decompose this product to establish recursion relations for said functions, draws upon [6, Chapters 9. to 11.] and [29, Chapter 16], where cost respectively value functionals are a priori introduced as integrals w.r.t. to (products of) stochastic kernels. The latter reference further provides the measurable selection theorem detailed in Appendix A.2, foundational for proving the existence of a measurable maximizing policy in an HDP, which, as a result, yields measurability of the optimal value function and existence of an optimal policy.

Remark 4.1. To avoid notational overload, many objects in the history-dependent setting are denoted in the same way as in the Markovian setting in Chapter 3. In the following, the objects in question refer to the history-dependent case. In the case where an object relates to the Markovian setting, this will be mentioned explicitly.

4.1 History-Based Decision Processes

A discrete-time, finite-horizon, *History-based Decision Process* (HDP) is defined as the tuple

$$(\mathcal{O}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, T), \tag{4.1}$$

with *observation space* $\mathcal{O} \subset \mathbb{R}^d$ for $d \in \mathbb{N}$, finite *action space* $\mathcal{A}$, finite time horizon $T \in \mathbb{N}$, and discount factor $\gamma \in [0, 1]$. Additionally, $\mathcal{P} = (P_t)_{t \in [T-1]}$ is a collection of *observation transition kernels*, and $\mathcal{R} = (R_t)_{t \in [T]}$ a collection of *reward functions*. The components are

described in detail in the paragraphs below. Analogous to an MDP, the agent interacts with an HDP in consecutive episodes. At step $t = 0$ of episode $k \in \mathbb{N}$, the environment emits some initial observation $o_0 \in \mathcal{O}$ according to some fixed initial distribution $\mu \in \text{Prob}(\mathcal{O})$. Then, at each step $t \in [T-1]$ the agent receives some observation of the environment's state $o_t \in \mathcal{O}$ and chooses an action $a_t \in \mathcal{A}$. Upon taking action a, the environment emits a reward $r_t \in \mathbb{R}$ and subsequently transitions to a new state with observation $o_{t+1} \in \mathcal{O}$. This cycle repeats until at time $T-1$ the last action is taken, and upon transitioning to a terminal observation $o_{T+1} \in \mathcal{O}$, the agent receives a terminal reward $r_T \in \mathbb{R}$, and the next episode $k+1$ begins. This behaviour produces an observation-action-history[1], which at time $t \in [T]$ is given as,

$$h_t := (o_0, a_0, o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t) \in (\mathcal{O} \times \mathcal{A})^t \times \mathcal{O}. \tag{4.2}$$

Observation, action and history spaces The observation space $\mathcal{O}$ is given as a compact subset of $\mathbb{R}^d$ for some $d \in \mathbb{N}$, and considered a measurable space with its Borel σ-algebra $\mathcal{B}_{\mathcal{O}}$. We use the notion of observations to describe the environment's condition, as opposed to states, to include potential partial observability. The action space $\mathcal{A}$ is again a finite, non-empty set of actions with cardinality $|\mathcal{A}| \in \mathbb{N}$. Without loss of generality we assume $\mathcal{A} = \{1, 2, \dots, |\mathcal{A}|\}$, and, where needed, we denote its Borel σ-algebra as $\mathcal{B}_{\mathcal{A}} = 2^{\mathcal{A}}$.

Remark 4.2. In some problems the set of actions to choose from depends on the current observation $o \in \mathcal{O}$ the environment is in, and is given as subset $\mathcal{A}_o \subset \mathcal{A}$, such that $\mathcal{A} = \bigcup_{o \in \mathcal{O}} \mathcal{A}_o$, compare [55, 29, 60]. For simplicity, and because the experiments considered in Chapter 6 are modeled such that all actions are selectable at each step, we do not consider observation dependent actions sets. This simplifies notation without a great loss in generality. At the same time, one could allow $\mathcal{O}$ and $\mathcal{A}$ to depend on time. This case can be included in the formulation (4.1) by setting $\mathcal{O} = \bigcup_{t \in [T]} \mathcal{O}_t$, and analogously for $\mathcal{A} = \bigcup_{t \in [T]} \mathcal{A}_t$, and by modifying the transition probabilities accordingly.

Definition 4.3. For an HDP (4.1), the *space of observation-action histories* up to time t is defined as

$$\mathcal{H}_t := (\mathcal{O} \times \mathcal{A})^t \times \mathcal{O}, \quad \text{for each } t \in [T],$$

with the convention $\mathcal{H}_0 := \mathcal{O}$, and elements $h_t \in \mathcal{H}_t$ given by (4.2). Note, that $\mathcal{H}_t \subset \mathbb{R}^{d+t(d+1)}$ for any $t \in [T]$, and we consider it as measurable space with its Borel-σ-algebra $\mathcal{B}_{\mathcal{O}_t} = \bigotimes_{i=1}^{t} (\mathcal{B}_{\mathcal{O}} \otimes \mathcal{B}_{\mathcal{A}}) \otimes \mathcal{B}_{\mathcal{O}}$.[2]

Additionally to the spaces $\mathcal{H}_t$, their time-augmented versions will be important for the proposed reinforcement learning method.

Definition 4.4. For $t \in [T]$ and $h_t \in \mathcal{H}_t$, we define the *time-augmented history* $\hat{h}_t$ by

$$\hat{h}_t := (0, o_0, a_0, 1, o_1, a_1, \dots, t-1, o_{t-1}, a_{t-1}, t, o_t) \in \hat{\mathcal{H}}_t, \tag{4.3}$$

[1] More specifically it produces an observation-action-reward-history, however we exclude the reward in (4.2) since the reward r_{t+1} can usually be derived from the tuple (o_t, a_t, o_{t+1}).

[2] The Borel σ-algebra on $\mathcal{H}_t$ again coincides with the product σ-algebra generated by the σ-algebras $\mathcal{B}_{\mathcal{O}}$ and $\mathcal{B}_{\mathcal{A}}$.

with the time-augmented history space $\hat{\mathcal{H}}_t := \bigtimes_{u=0}^{t-1}(\{u\} \times \mathcal{O} \times \mathcal{A}) \times \{t\} \times \mathcal{O}$.

Remark 4.5. Similar to time-augmented paths of bounded variation in Definition 2.6, the time-augmented history $\hat{h}_t$ may be defined with the time-component given by normalized steps u/T, remaining time steps $T - u$, or normalized remaining steps $1 - u/T$, for $u \in [t]$. From a reinforcement learning perspective, normalizing the inputs to the learning algorithm improves stability, which is why in the experiments in Chapter 6 the time-component will always be normalized.

Let $t \in [T]$ and consider a history $h_t \in \mathcal{H}_t$ and its time-augmentation $\hat{h}_t \in \hat{\mathcal{H}}_t$. We introduce the following notation and definitions regarding histories and their spaces:

(i) For h_t (and $\hat{h}_t$), the last observation's time index t will be called the (time-augmented) history's length. Note, that t corresponds to the number of actions contained in the (time-augmented) history, while the number of observation is $t + 1$.

(ii) Given time points $s, u \in [t]$ with $s < u$, we define the sub-history from s until u of h_t and $\hat{h}_t$, respectively, as

$$h_{s:u} := (o_s, a_s, \dots, o_{u-1}, a_{u-1}, o_u) \in \mathcal{H}_{s:u} \tag{4.4}$$

$$\hat{h}_{s:u} := (s, o_s, a_s, \dots, u-1, o_{u-1}, a_{u-1}, u, o_u) \in \hat{\mathcal{H}}_{s:u} \tag{4.5}$$

with the spaces $\mathcal{H}_{s:u} := \mathcal{H}_{u-s}$ and $\hat{\mathcal{H}}_{s:u} := \bigtimes_{u=s}^{t}(\{u\} \times \mathcal{O} \times \mathcal{A}) \times \{t\} \times \mathcal{O}$. The length of the sub-histories $h_{s:u}$ and $\hat{h}_{s:u}$ is given by $u - s$.

(iii) Often, we consider the concatenation of h_t with the next selected action $a_t \in \mathcal{A}$ and the subsequent observation $o_{t+1} \in \mathcal{O}$, which we denote as (h_t, a_t, o_t), with the understanding that h_t is "unpacked", in the sense that its components are inserted into tuple (h_t, a_t, o_t) without parenthesis, yielding a concatenated history of length $t + 1$. If $a \in \mathcal{A}$ and $o \in \mathcal{O}$ are given without time-index, the concatenation (h_t, a, o) is defined in the same way. Analogously, we denote the concatenation of $\hat{h}_t$ with $a_t, t + 1, o_{t+1}$ as $(\hat{h}_t, a_t, t + 1, o_{t+1})$.

(iv) Given a function of histories $f : \mathcal{H}_t \to \mathbb{R}$ for some $t \in [T]$, we sometimes want to emphasise certain input components to f, by splitting the input history. For example, for any $h_t \in \mathcal{A}$, we may write $f(h_t) = f(h_{t-1}, a_t, o_{t+1})$ to emphasize the last two history components a_t and o_{t+1}, or $f(h_t) = f(h_u, a_u, h_{u+1:t})$ to highlight the history up to, or from some time $0 \leq u < t$ as well as action a_u. In the same way we may write $f(h_{t-1}, a, o)$ where the input to f is the concatenated history (h_{t-1}, a, o) of length t for some $a \in \mathcal{A}$ and $o \in \mathcal{O}$.

(v) The concatenations of h_t and $\hat{h}_t$ with a single action $a \in \mathcal{A}$, are denoted h_t^a and $\hat{h}_t^a$, respectively. They are given in the form of the discrete streams of data

$$h_t^a := (o_u, a_u)_{u \in [t]} \quad \text{and} \quad \hat{h}_t^a = (u, o_u, a_u)_{u \in [t]},$$

where we set $a_t := a$, and with components $(o_u, a_u) \in \mathbb{R}^{d+1}$ and $(u, o_u, a_u) \in \mathbb{R}^{d+1}$, respectively, for $u \in [t]$.

(vi) In Chapter 5 we compute the signature of histories h_t^a, and their time-augmentations $\hat{h}_t^a$. Since the history as argument already includes the time index, we omit it in the signature, and set

$$S(h_t^a) := S_{0,t}(h_t^a), \qquad S(\hat{h}_t^a) := S_{0,t}(\hat{h}_t^a),$$

where the signature of the discrete streams of data h_t^a and $\hat{h}_t^a$ is defined via the signature of their linear interpolations $H_{[0,t]}^a$ and $\hat{H}_{[0,t]}^a$, as in Definition 2.16. In the same way, for the signature of a sub-history, we set $S(h_{s:u}^{a_u}) := S_{s:u}(h_t^a)$. Specifically, we will encounter $S(h_{t:t+1}^a)$ when updating the signature of a concatenated history after new data has arrived, which is simply the signature of the linear interpolation between the data point (t, o_t, a_t) at time t and $(t+1, o_{t+1}, a)$ at $t+1$.

The reason why we consider time augmented histories is twofold. First, including time-steps in observations creates time-aware agents which, have shown superior performance compared to their time-unaware counterparts in finite-horizon reinforcement learning tasks in a variety of methods [58]. And second, having a time component in observation-action histories comes in handy when approximating Q-functions by linear functionals of the histories' signatures over time in Chapter 5 by means of Lemma 2.13 and Theorem 2.15.

Observation transition kernels At the beginning of each episode, the initial observation o_0 is sampled according to some initial distribution $\mu \in \mathrm{Prob}(\mathcal{O})$. Observations at subsequent steps are emitted according to the observation transition kernels $\mathcal{P} = (P_t)_{t\in[T-1]}$, defined as follows.

Definition 4.6. For each $t \in [T-1]$, the *observation transition kernel* $\mathcal{P}_t$ is a stochastic kernel from $(\mathcal{H}_t \times \mathcal{A}, \mathcal{B}_{\mathcal{H}_t} \otimes \mathcal{B}_{\mathcal{A}})$ to $(\mathcal{O}, \mathcal{B}_{\mathcal{O}})$.

The transition kernel $\mathcal{P}$ have the same role as for MDPs in the previous chapter, with the difference that here they are functions of action-observation histories prior the current time step. For $t \in [T-1]$, the workings of $\mathcal{P}_t$ are as follows: For any set $B \in \mathcal{B}_{\mathcal{O}}$, the probability of the next observation o_{t+1} belonging to B, provided that up to t the environment's history is $h_t \in \mathcal{H}_t$ and the current action chosen by the agent is $a_t \in \mathcal{A}$, is given by $P_t(B \,|\, h_t, a_t)$. Put differently, o_{t+1} is sampled according to the probability distribution $P_t(\,\cdot\,|\, h_t, a_t)$ on the measurable space $(\mathcal{O}, \mathcal{B}_{\mathcal{O}})$.

Reward functions Analogue to the observation transitions kernels, the rewards are functions of histories. For $t \in [T-1]$, the *reward function* $R_t : \mathcal{H}_t \times \mathcal{A} \to \mathbb{R}$ is measurable w.r.t. $\mathcal{B}_{\mathcal{H}_t} \otimes \mathcal{B}_{\mathcal{A}}$. It provides the reward $R_t(h_t, a)$ from choosing action $a \in \mathcal{A}$ in history $h_t \in \mathcal{H}_t$ at time t.[3] Additionally, at terminal time T, the terminal reward is determined by the $\mathcal{B}_{\mathcal{H}_T}$-measurable *terminal reward function* $R_T : \mathcal{H}_T \to \mathbb{R}$. We assume throughout that there exists a constant $C \in \mathbb{R}_+$, such that $|R_t(h_t, a)| < C$ for all $t \in [T-1]$ and $(h_t, a) \in \mathcal{H}_t \times \mathcal{A}$

[3] This definition includes the case where the reward depends on history h_t only through the last observation o_t.

as well as $|R_T(h_T)| < C$ for all $h_T \in \mathcal{H}_T$. From this assumption it immediately follows that

$$\|R_t\|_\infty = \sup_{(h_t,a)\in\mathcal{H}_t\times\mathcal{A}} |R_t(h_t,a)| \leq C,$$

as well as $\|R_T\|_\infty \leq C$.

Remark 4.7. As in the Markovian case, compare Remark 3.2, rewards may additionally depend on the environment's observation at the next step, such that the reward at time $t \in [T-1]$ is determined by the measurable reward function $\tilde{R}_t : \mathcal{H}_t \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$. where $\tilde{R}_t(h_t, a, o)$, gives the reward at time t when the history is h_t, action a is selected and the environment transitions to observation o at time $t+1$. In this case we can again define an *expected immediate reward function* at $t \in [T-1]$ by

$$R_t(h_t, a) := \int_{\mathcal{O}} \tilde{R}_t(h_t, a, o)\, P_t(\mathrm{d}o \,|\, h_t, a) \quad \text{for all } (h_t, a) \in \mathcal{H}_t \times \mathcal{A}. \tag{4.6}$$

Since $\tilde{R}_t$ is measurable, so is R_t for any $t \in [T-1]$ due to Lemma A.6, and the subsequent results in this section remain true if R_t is defined by (4.6). From a reinforcement learning perspective, it is irrelevant if the reward sample $r_t \in \mathbb{R}$ that the agent receives at time t, depends only on (h_t, a_t) or, additionally on the next observation o_{t+1}, since we only require, the value r_t to be fixed before the agent selects the next action a_{t+1}, and that it does not depend on its choice.

Remark 4.8. No stationarity or Markov assumptions are made about the observation transition kernels $\mathcal{P}$, nor about whether $\mathcal{O}$ resembles partial observations, or the true states itself. This provides that the methods developed in this thesis are readily applicable to partially observable MDPs as environment model or even fully non-Markovian models.

Policies and the agent's objective The agent's objective is to find a sequence of actions $(a_0, a_1, \ldots, a_T)$, where at each time t the choice of a_t is dependent on the information contained in h_t, that maximizes the expected cumulative sum of discounted rewards over each episode. Actions are selected according to a policy the agent follows.

Definition 4.9. For an HDP (4.1), a policy is defined as a family $\pi = (\pi_t)_{t\in[T-1]}$, where π is a

- *history-dependent, randomized policy*, if π_t is a stochastic kernel from $(\mathcal{H}_t, \mathcal{B}_{\mathcal{H}_t})$ to $(\mathcal{A}, \mathcal{B}_{\mathcal{A}})$
- *history-dependent, deterministic policy*, if π_t is a measurable map $\pi_t : \mathcal{H}_t \to \mathcal{A}$,

for each $t \in [T-1]$. The sets of all randomized and deterministic history-dependent policies are denoted as Π^{HR} and Π^{HD}, respectively. Any $\pi \in \Pi^{\mathrm{HD}}$ can be considered as a special case of a randomized policy by setting $\tilde{\pi}_t(\cdot|h_t) := \delta_{\pi_t(h_t)}(\cdot)$ for each $h_t \in \mathcal{H}_t$, thus $\Pi^{\mathrm{HD}} \subset \Pi^{\mathrm{HR}}$, and $\pi \in \Pi^{\mathrm{HR}}$ always includes the case $\pi \in \Pi^{\mathrm{HD}}$.

When observation transitions depend on histories, we can not hope for Markovian optimal policies, and all policies are considered functions of histories. The existence of at least one

such policy is guaranteed by setting $\pi_t(h_t) = a$ for fixed $a \in \mathcal{A}$, and all $t \in [T-1]$. Note, that this is possible because we assumed that the set of possible actions to choose from at each step does not depend on the observation the process is in, otherwise greater care has to be taken to guarantee the existence of measurable policies, compare [29, Chapter 16].

A policy $\pi \in \Pi^{\mathrm{HR}}$ together with a fixed initial probability distribution $\mu \in \mathrm{Prob}(\mathcal{O})$ induces the canonical probability space

$$(\mathcal{H}_T, \mathcal{B}_{\mathcal{H}_T}, \mathbb{P}^\pi), \tag{4.7}$$

where the measure $\mathbb{P}^\pi$ is determined as in Appendix A.1 by the product of the measure μ and the stochastic kernels $(P_t)_{t\in[T-1]}$ and $(\pi_t)_{t\in[T-1]}$. We suppress the dependence on the initial distribution μ since it is considered fixed. The measure $\mathbb{P}^\pi$ satisfies

$$\begin{aligned}\mathbb{P}^\pi[\mathrm{d}h_T] = {} & P_{T-1}(\mathrm{d}o_T|h_{T-1}, a_{T-1})\, \pi_{T-1}(\mathrm{d}a_{T-1}|h_{T-1})\, P_{T-2}(\mathrm{d}o_{T-1}|h_{T-2}, a_{T-2}) \\ & \dots P_0(\mathrm{d}o_1|o_0, a_0)\, \pi_0(\mathrm{d}a_0|o_0)\, \mu(\mathrm{d}o_0),\end{aligned} \tag{4.8}$$

with $h_T = (o_0, a_0, a_1, \dots, a_{T-1}, o_T)$. We again omit the dependence on μ and $\mathcal{P}$ in $\mathbb{P}^\pi$, since both are fixed, and to highlight that the measure $\mathbb{P}^\pi$ is determined by the choice of π. For any history $h_t \in \mathcal{H}_T$, we define the random observations $(O_t)_{t\in[T]}$ and random actions $(A_t)_{t\in[T-1]}$ as the coordinate maps on (4.7),

$$O_t(h_T) := o_t, \qquad A_t(h_T) := a_t.$$

The conditional distributions of the random variables O_t and A_t are determined by P_t and π_t, respectively. The process $O_0, A_0, O_1, \dots, A_{T-1}, O_T$ is just the canonical process on the probability space (4.7). Further, we define the history process by

$$H_0 := O_0, \quad \text{and} \quad H_t := (O_0, A_0, O_1, \dots, A_{t-1}, O_t), \text{ for } t \in [T] \setminus \{0\},$$

where H_t is a $\mathcal{H}_t$-valued random variable on (4.7) for each $t \in [T]$. Similar to the Markovian case, for any $t \in [T]$, history $h_t \in \mathcal{H}_t$, Borel set $B \in \mathcal{B}_{\mathcal{O}}$, and action $a \in \mathcal{A}$, the measure $\mathbb{P}^\pi$ satisfies

$$\begin{aligned}\mathbb{P}^\pi[O_0 \in B] &= \mu(B), \\ \mathbb{P}^\pi[A_t = a|H_t = h_t] &= \pi(a|h_t), \\ \mathbb{P}^\pi[O_{t+1} \in B|H_t = h_t, A_t = a] &= P_t(B|h_t, a).\end{aligned} \tag{4.9}$$

The difference to a MDP is apparent in that the conditional probabilities given $H_t = h_t$, and potentially $A_t = a$, are functions of the full history h_t, given by the observation transition and policy kernels. For a policy $\pi \in \Pi^{\mathrm{HD}}$, the random actions are given as $A_t := \pi_t(O_t)$, and Equations (4.8) and (4.9) change to

$$\mathbb{P}^\pi[A_t = a|H_t = h_t] = \delta_{\pi(h_t)}(a).$$

and

$$\mathbb{P}^\pi[\mathrm{d}h_T] = P_{T-1}(\mathrm{d}o_T|h_{T-1}, a_{T-1})\, \delta_{\pi_{T-1}(h_{T-1})}(\mathrm{d}a_{T-1})\, P_{T-2}(\mathrm{d}o_{T-1}|h_{T-2}, a_{T-2}) \\ \dots P_0(\mathrm{d}s_1|o_0, a_0)\, \delta_{\pi_0(o_0)}(\mathrm{d}a_0)\, \mu(\mathrm{d}o_0).$$

When π_t is deterministic, observation transition kernels only depend on h_t and we define

$$P_{\pi_t}(\mathrm{d}o \,|\, h_t) := P_t(\mathrm{d}o \,|\, h_t, \pi_t(h_t)) \quad \text{for } t \in [T-1],$$

which is a stochastic kernel from $\mathcal{H}_t$ to $\mathcal{O}$.

Remark 4.10. For $s, t \in [T]$ with $s < t$, denote by $H_{s:u} := (O_s, A_s, O_{s+1}, \dots, A_{u-1}, O_u)$ the random sub-history from time s until t. To ease notation in the following section, we define the stochastic kernels

$$(\pi \otimes P)^{(t,T)} := \bigotimes_{u=t}^{T-1} (\pi_u \otimes P_u), \quad \text{for } t \in [T-1], \tag{4.10}$$

$$(P \otimes \pi)^{(t,T)} := P_t \otimes (\pi \otimes P)^{(t+1,T)}, \quad \text{for } t \in [T-2], \tag{4.11}$$

where it follows from Corollary A.5 that $(\pi \otimes P)^{(t,T)}$ is a stochastic kernel from $\mathcal{H}_t$ to $\mathcal{A} \times \mathcal{H}_{t+1:T}$, and a regular conditional distribution of the random sub-history $(A_t, H_{t+1:T})$ for each $t \in [T-1]$. Moreover, $(P \otimes \pi)^{(t,T)}$ is a stochastic kernel from $\mathcal{H}_t \times \mathcal{A}$ to $\mathcal{H}_{t+1:T}$, and a regular conditional distribution of the random sub-history $H_{t+1:T}$.

4.2 Value Functions and Optimal Policies in History-Based Decision Processes

The agent's goal is to find a policy π which maximizes the expected total discounted reward over an episode. A policy that achieves this goal is called an *optimal policy*. As in the MDP case we want to characterize optimality of a policy through the notion of value functions. For an HDP, those need to be defined in terms of histories.

Definition 4.11 (Value functions in HDPs)**.** Let $\pi \in \Pi^{\mathrm{HR}}$. The value function $V_t^\pi : \mathcal{H}_t \to \mathbb{R}$ at time $t \in [T-1]$ corresponding to π in the HDP (4.1), is defined for every $h_t \in \mathcal{H}_t$ by

$$V_t^\pi(h_t) := \mathbb{E}^\pi\left[\sum_{u=t}^{T-1} \gamma^{u-t} R_u(H_u, A_u) + \gamma^{T-t} R_T(H_T) \,\middle|\, H_t = h_t\right] \tag{4.12}$$

and for $t = T$ as $V_T^\pi(h_T) := R_T$ for every $h_T \in \mathcal{H}_T$. It is understood that policy π is followed from t onwards, and the conditional expectation $\mathbb{E}^\pi$ is given on the canonical probability space with probability measure $\mathbb{P}^\pi$.

For $t = 0$, the quantity $V_0^\pi(o_0)$ provides the expected total discounted reward over the whole episode, if the agent follows policy π, and the environment starts in observation $o_0 \in \mathcal{O}$. Note, that in this setup, with finite time horizon, the discount factor could also be encoded in the rewards. Since all reward functions $(R_t)_{t\in[T]}$ are bounded, so is V_t^π for each

policy $\pi \in \Pi^{\mathrm{HR}}$ and $t \in [T]$. For a fixed policy, the corresponding value functions may be computed via the following value iteration.

Theorem 4.12 (Value iteration in HDPs)**.** *Fix a $\pi \in \Pi^{HR}$. Then, the corresponding value functions V_t^π are $\mathcal{B}_{\mathcal{H}_t}$-measurable for each $t \in [T]$, and satisfy the recursion*

$$V_t^\pi(h_t) = \sum_{a \in \mathcal{A}} \pi_t(a|h_t)\Big(R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}\big[V_{t+1}^\pi(h_t, a, O_{t+1})\big]\Big), \quad t \in [T-1], \tag{4.13}$$

with the terminal condition $V_T^\pi(h_T) = R_T(h_T)$, and where $\mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}$ is defined as in Equation (A.8) as the conditional expectation of $V_{t+1}^\pi(H_{t+1}) = V_{t+1}^\pi(H_t, A_t, O_{t+1})$ given $\{H_t = h_t, A_t = a\}$. If $\pi \in \Pi^{\mathrm{HD}}$ the recursion is given as

$$V_t^\pi(h_t) = R_t(h_t, \pi_t(h_t)) + \gamma\, \mathbb{E}_{P_{\pi_t}(\mathrm{d}o|h_t)}\big[V_{t+1}^\pi(h_t, \pi_t(h_t), O_{t+1})\big], \quad t \in [T-1], \tag{4.14}$$

with the stochastic kernel $P_{\pi_t}(\mathrm{d}o \,|\, h_t) = P_t(\mathrm{d}o \,|\, h_t, \pi_t(h_t))$.

Remark 4.13. We did not state the value iteration for a given policy in the Markovian case in Section 3.2, however, we highlight that for an MDP, the value iteration can be stated in the same form as in Equations (4.13) and (4.14), in this case for a fixed Markovian randomized or deterministic policy, respectively. Replace all time-t histories h_t by the state s_t, the random observation O_{t+1} by the random state S_{t+1}, and let P_t be a Markovian transition kernel, only depending on the pair (s_t, a). Note that in this case, the value function at $t+1$ in the expectation is a function of the current state only and given as $V_{t+1}^\pi(S_{t+1})$.

Proof of Theorem 4.12. For each $t \in [T-1]$, we define the auxiliary function $f_t : \mathcal{H}_T \to \mathbb{R}$ by

$$f_t(h_T) = f_t(h_t, a_t, h_{t+1:T}) := R_t(h_t, a_t) + \sum_{u=t+1}^{T-1} \gamma^{u-t} R_u(h_u, a_u) + R_T(h_T) \tag{4.15}$$

and for $t = T$ as $f_T(h_T) := R_T(h_T)$ for any history $h_T \in \mathcal{H}_T$, and where $h_u \in \mathcal{H}_u$ is the sub-history from 0 to u, and $a_u \in \mathcal{A}$ is the action at time u of the input history h_T, for each $u \in \{t, t+1, \dots, T\}$. The function f_t provides the cumulative discounted reward obtained from t onwards, if the action-observation history over the whole episode is h_T. Note, that f_t is $\mathcal{B}_{\mathcal{H}_T}$-measurable for each $t \in [T]$, due to the measurability of the reward functions R_t, and that the functions $(f_t)_{t \in [T]}$ satisfy

$$f_t(h_T) = R_t(h_t, a_t) + \gamma\, f_{t+1}(h_T). \tag{4.16}$$

Recall, that the stochastic kernel $(\pi \otimes P)^{(t,T)}$ defined in Equation (4.10) is a regular conditional distribution of $(A_t, H_{t+1:T})$ given H_t. With the help of f_t, the kernel $(\pi \otimes P)^{(t,T)}$ and Equation (A.10) we may express the value function at time t as

$$\begin{aligned} V_t^\pi(h_t) &= \mathbb{E}^\pi[f_t(H_t, A_t, H_{t+1:T}) \,|\, H_t = h_t] \\ &= \int_{\mathcal{A} \times \mathcal{H}_{t+1:T}} f_t(h_t, a, h_{t+1:T})\, (\pi \otimes P)^{(t,T)}(\mathrm{d}(a, h_{t+1:T}) \,|\, h_t). \end{aligned} \tag{4.17}$$

It follows from Lemma A.6, that V_t^π is $\mathcal{B}_{\mathcal{H}_t}$-measurable for each $t \in [T-1]$, and V_T^π is $\mathcal{B}_{H_T}$-measurable by definition as R_T. This means that for $(h_t, a) \in \mathcal{H}_t \times \mathcal{A}$, the conditional expectations

$$\mathbb{E}^\pi_{P_t(\mathrm{d}o|h_t,a)}[V^\pi_{t+1}(h_t, a, O_{t+1})] = \mathbb{E}^\pi[V^\pi_{t+1}(h_t, A_t, O_{t+1}) \,|\, H_t = h_t, A_t = a]$$

as in Equation (A.8) is well-defined, and moreover, can be expressed as integral with respect to the kernel $(P \otimes \pi)^{t+1,T}$ defined in (4.11), which is a regular conditional distribution of $H_{t+1:t}$ given (H_t, A_t).

For the recursion, at $t = T$, we get by definition the terminal condition. Now let $t \in [T-1]$. We want to apply Theorem A.4 to the measure $(\pi \otimes P)^{(t,T)}(\cdot|h_t)$ on $\mathcal{A} \times \mathcal{H}_t$. Fix input $h_t \in \mathcal{H}_t$, and define the probability measure μ_1 on $\mathcal{A}$ as $\mu_1 := \pi(\cdot|h_t)$ and the stochastic kernel κ_1 as

$$\begin{aligned}\kappa_1 : \mathcal{A} \times \mathcal{B}_{\mathcal{H}_{t+1:T}} &\to [0,1],\\ (a, F) &\mapsto \kappa_1(a, F) := (P \otimes \pi)^{(t,T)}(F \,|\, h_t, a),\end{aligned}$$

where again $\kappa_1(\mathrm{d}h_{t+1:T} \,|\, a) := \kappa_1(a, \mathrm{d}h_{t+1:T})$ for the probability measure on $\mathcal{H}_{t+1:T}$ for action $a \in \mathcal{A}$. Then, by definition

$$(\pi \otimes P)^{(t,T)}(\cdot \,|\, h_t) = \mu_1 \otimes \kappa_1,$$

and from Equation (4.17), and application of Theorem A.4 to the product $\mu_1 \otimes \kappa_1$, we obtain

$$\begin{aligned}V_t^\pi(h_t) = \mathbb{E}^\pi[f_t(H_T)|H_t = h_t] &= \int_{\mathcal{A}\times\mathcal{H}_{t+1:T}} f_t(h_t, \cdot, \cdot)\, \mathrm{d}(\mu_1 \otimes \kappa_1)\\ &= \sum_{a\in\mathcal{A}} \int_{\mathcal{H}_{t+1:T}} f_t(h_t, a, h_{t+1:T})\, \kappa_1(\mathrm{d}h_{t+1:T} \,|\, a)\, \mu_1(a)\\ &= \sum_{a\in\mathcal{A}} \pi_t(a|h_t) \Big(\int_{\mathcal{H}_{t+1:T}} f_t(h_t, a, h_{t+1:T})\, \kappa_1(\mathrm{d}h_{t+1:T} \,|\, a) \Big)\\ &= \sum_{a\in\mathcal{A}} \pi_t(a|h_t) \Big(R_t(h_t, a) + \gamma \int_{\mathcal{H}_{t+1:T}} f_{t+1}(h_t, a, h_{t+1:T})\, \kappa_1(\mathrm{d}h_{t+1:T} \,|\, a) \Big) \qquad (4.18)\end{aligned}$$

where the second equality follows from inserting the definition of μ_1, and the last equality from the recursive property (4.16) of the function f_t and the linearity of the integral. By definition of κ_1 and $(P \otimes \pi)^{(t,T)}$ we get

$$\kappa_1(\cdot \,|\, a) = \big(P_t \otimes (\pi \otimes P)^{(t+1,T)}\big)(\cdot \,|\, h_t, a),$$

for each $a \in \mathcal{A}$ and fixed $h_t \in \mathcal{H}_t$. We want to apply Theorem A.4 again to the probability measure $\kappa(\cdot|a)$ on $\mathcal{H}_{t+1:T} = \mathcal{O} \times \mathcal{A} \times \mathcal{H}_{t+2:T}$. For fixed $(h_t, a) \in \mathcal{H}_t \times \mathcal{A}$, define the probability

measure μ_2 on $\mathcal{O}$ as $\mu_2 := P_t(\cdot \,|\, h_t, a_t)$, and the stochastic kernel κ_2 as

$$\begin{aligned}\kappa_2 : \mathcal{O} \times \mathcal{B}_{\mathcal{A}\times\mathcal{H}_{t+2:T}} &\to [0,1],\\ (o, F) &\mapsto \kappa_2(o, F) := (\pi \otimes P)^{(t+1,T)}(F \,|\, (h_t, a, o))\end{aligned}$$

Then, $\kappa_1(\cdot \,|\, a_t) = \mu_2 \otimes \kappa_2$, and application of Theorem A.4 to the product $\mu_2 \otimes \kappa_2$ yields for the integrals in (4.18),

$$\begin{aligned}\int_{\mathcal{H}_{t+1:T}} f_{t+1}(h_t, a, h_{t+1:T})\, \kappa_1(\mathrm{d}h_{t+1:T} \,|\, a) &= \int_{\mathcal{H}_{t+1:T}} f_{t+1}(h_t, a, \cdot)\, \mathrm{d}(\mu_2 \otimes \kappa_2)\\ &= \int_{\mathcal{O}} \int_{\mathcal{A}\times\mathcal{H}_{t+2:T}} f_{t+1}(h_t, a, o, a', h_{t+2:T})\, \kappa_2(\mathrm{d}(a', h_{t+2:T}) \,|\, o)\, \mu_2(\mathrm{d}o)\\ &= \int_{\mathcal{O}} V^{\pi}_{t+1}(h_t, a_t, o)\, \mu_2(\mathrm{d}o) = \mathbb{E}_{P_t(\mathrm{d}o|h_t,a_t)}\big[V^{\pi}_{t+1}(h_t, a_t, O)\big],\end{aligned} \tag{4.19}$$

where the second to last equality follows from expression (4.17) for the value function V^{π}_{t+1} at $t+1$ with input history (h_t, a, o), as integral w.r.t to $\kappa_2(\cdot \,|\, o) = (\pi \otimes P)^{(t+1,T)}(\cdot \,|\, (h_t, a, o))$ over $\mathcal{A} \times \mathcal{H}_{t+2:T}$ with the auxiliary function f_{t+1}, and the last equality follows by definition of $\mu_2 = P_t(\cdot|h_t, a)$ and that of the conditional expectation $\mathbb{E}_{P_t(\mathrm{d}o|h_t,a_t)}$ as in Equation (A.8). Since $h_t \in \mathcal{H}_t$ was arbitrary, the claim for $\pi \in \Pi^{\mathrm{HR}}$ follows by plugging (4.19) into (4.18).

If $\pi' \in \Pi^{\mathrm{HD}}$, we interpret it as randomized policy by setting $\tilde{\pi}_t(a|h_t) := \delta_{\pi'_t(h_t)}(a)$ for each $t \in [T-1]$ and any $(h_t, a) \in \mathcal{H}_t \times \mathcal{A}$. Substituting $\pi(a|h_t)$ in expression (4.13) by $\tilde{\pi}_t(a|h_t)$ yields recursion (4.14) for deterministic π'. □

Analogously to the Markovian case, we define a policy to be optimal if it obtains the maximal attainable expected total discounted reward over the episode.

Definition 4.14 (Optimal policies in HDPs)**.** A policy $\pi^* \in \Pi^{\mathrm{HR}}$ is called optimal for the HDP (4.1), if it satisfies

$$V_0^{\pi^*}(o) \geq V_0^{\pi}(o), \quad \text{for all } \pi \in \Pi^{\mathrm{HR}}, o \in \mathcal{O}.$$

The agent's goal is to find such an optimal policy. We will characterize an optimal policy with the help of optimal value functions.

Definition 4.15 (Optimal value functions in HDPs)**.** The optimal value function $V_t^* : \mathcal{H}_t \to \mathbb{R}$ at time $t \in [T-1]$ for the HDP (4.1) is defined as

$$V_t^*(h_t) := \sup_{\pi \in \Pi^{\mathrm{HR}}} V_t^{\pi}(h_t), \quad \text{for } h_t \in \mathcal{H}_t, \tag{4.20}$$

and at time $t = T$ as $V_T^*(h_T) := R_T(h_T)$ for $h_T \in \mathcal{H}_T$.

The optimal value function at time t gives the maximal attainable value over the remaining episode, and thus, at $t = 0$ the maximum attainable value over the whole decision period. Due to the boundedness of reward functions, for any $t \in [T]$, the value V_t^{π} is bounded uniformly in π, which implies the boundedness of V_t^*. In Theorem 4.16 we will show, that the value functions corresponding to an optimal policy π^* coincide with the optimal

value functions, and that optimal policies are deterministic and can be characterized as maximizing policies. For $\pi \in \Pi^{\mathrm{HD}}$, we say that π_t is a *maximizer* at time $t \in [T-1]$ if

$$\begin{aligned} R_t(h_t, \pi_t^*(h_t)) + \gamma\, \mathbb{E}_{P_{\pi_t}(\mathrm{d}o|h_t)}[V_{t+1}^*(h_t, \pi_t^*(h_t), O_{t+1})] \\ = \max_{a \in \mathcal{A}} \Big\{ R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V_{t+1}^*(h_t, a, O_{t+1})] \Big\}, \end{aligned} \tag{4.21}$$

whenever the map V_{t+1}^* is $\mathcal{B}_{\mathcal{H}_{t+1}}$-measurable such that the conditional expectation $\mathbb{E}_{P_t(\cdot|h_t,a)}$ is well-defined according to Equation (A.8). A policy which is a maximizer at each $t \in [T-1]$ is called a *maximizing policy*. The value functions corresponding to a maximizing policy satisfy the following optimality equations.

Theorem 4.16 (Optimal value iteration in HDPs)**.** *Let $\pi^* \in \Pi^{\mathrm{HD}}$ be a maximizing policy. Then, $V_t^* = V_t^{\pi^*}$ for $t \in [T]$, specifically, $V_0^{\pi^*} = V_0^*$, which implies π^* is an optimal policy. Furthermore, V_t^* is $\mathcal{B}_{\mathcal{H}_t}$-measurable for each $t \in [T]$, and the optimal value iteration*

$$V_t^*(h_t) = \max_{a \in \mathcal{A}} \Big\{ R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V_{t+1}^*(h_t, a, O_{t+1})] \Big\}, \quad t \in [T-1], \tag{4.22}$$

holds for any with terminal condition $V_T^*(h_T) = R_T(h_T)$.

Remark 4.17. The optimality equations (4.22), are an analogue of the Bellman optimality equations (3.11) for an MDP, where the state s_t is defined as the full history up to time t. If we replace the time-t history h_t by a state s_t, the history-dependent transition kernel $P_t(\cdot|h_t, a)$ by a Markovian kernel $P_t(\cdot|s_t, a)$, and the triplet (h_t, a, O_{t+1}) in the expectation by the random state S_{t+1}, Equation (4.22) becomes the Bellman optimality equation (3.11) at time t, where the respective Markovian optimal value functions are functions of the current state only.

For the proof of Theorem 4.16 we need the following lemma.

Lemma 4.18. *Let $t \in [T-1]$, and assume that V_{t+1}^* is $\mathcal{B}_{\mathcal{H}_{t+1}}$-measurable. Then, for any $h_t \in \mathcal{H}_t$,*

$$V_t^*(h_t) \leq \max_{a \in \mathcal{A}} \Big\{ R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V_{t+1}^*(h_t, a, O_{t+1})] \Big\}. \tag{4.23}$$

Proof. Fix a policy $\pi \in \Pi^{\mathrm{HR}}$. By definition we have $V_t^\pi \leq V_t^*$, which, for $(h_t, a) \in \mathcal{H}_t \times \mathcal{A}$ implies

$$\mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V_t^\pi(h_t, a, O_{t+1})] \leq \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V_t^*(h_t, a, O_{t+1})].$$

Application of the value iteration in Theorem 4.12 with policy π in the first step, and above inequality in the second step yields

$$\begin{aligned} V_t^\pi(h_t) &= \sum_{a \in \mathcal{A}} \pi_t(a|h_t) \Big(R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V_{t+1}^\pi(h_t, a, O_{t+1})] \Big) \\ &\leq \sum_{a \in \mathcal{A}} \pi_t(a|h_t) \Big(R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V_{t+1}^*(h_t, a, O_{t+1})] \Big) \\ &\leq \max_{a \in \mathcal{A}} \Big\{ R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V_{t+1}^*(h_t, a, O_{t+1})] \Big\}, \end{aligned}$$

where the last inequality follows from the fact that $\pi_t(a|h_t)$ are probabilities and sum to one. If π is deterministic, we again interpret it as kernels δ_{π_t}, and in the above sums only the term with action $a = \pi_t(h_t)$ remains. Since π was arbitrary, the inequality also holds if we take the supremum on the left hand side. □

Proof of Theorem 4.16. The proof is carried out by backward induction.

For $t = T$, by definition $V_T^{\pi^*} = R_T = V_T^*$, which yields the terminal condition. By assumption on R_T, the optimal value function $V_T^* = R_T$ is $\mathcal{B}_{\mathcal{H}_T}$-measurable, and as such, the conditional expectation

$$\mathbb{E}^{\pi}_{P_{T-1}(\mathrm{d}o|h_{T-1},a)}[V_T^*(h_{T-1}, a, O_{t+1})] = \mathbb{E}^{\pi}[V_T^*(H_{T-1}, A_{T-1}, O_T \,|\, H_{t-1} = h_{t-1}, A_{T-1} = a]$$

is well defined for any $(h_{T-1}, a) \in \mathcal{H}_{T-1} \times \mathcal{A}$.

For $t \in [T-1]$, assume that $V_{t+1}^* = V_{t+1}^{\pi^*}$, and that V_{t+1}^* is $\mathcal{B}_{\mathcal{H}_t}$-measurable.[4] Then, the conditional expectation

$$\mathbb{E}^{\pi}_{P_{t+1}(\mathrm{d}o|h_{t+1},a)}[V_{t+1}^*(h_{t+1}, a, O_{t+2})]$$

is well-defined according to Equation (A.8), and application of Lemma 4.18, together with the definition of the maximizing policy π^* in Equation (4.21), the induction hypothesis for $t+1$, and the value iteration (4.14) applied to the value functions corresponding to π^*, yield

$$\begin{aligned}
V_t^*(h_t) &\leq \max_{a\in\mathcal{A}}\Big\{R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}\big[V_{t+1}^*(h_t, a, O_{t+1})\big]\Big\} \\
&= R_t(h_t, \pi_t^*(h_t)) + \gamma\, \mathbb{E}_{P_{\pi_t^*}(\mathrm{d}o|h_t)}\big[V_{t+1}^*(h_t, \pi_t^*(h_t), O_{t+1})\big] \\
&= R_t(h_t, \pi_t^*(h_t)) + \gamma\, \mathbb{E}_{P_{\pi_t^*}(\mathrm{d}o|h_t)}\big[V_{t+1}^{\pi^*}(h_t, \pi_t^*(h_t), O_{t+1})\big] \\
&= V_t^{\pi^*}(h_t) \leq \sup_{\pi\in\Pi} V_t^{\pi}(h_t) = V_t^*(h_t),
\end{aligned}$$

which shows the equality $V_t^* = V_{t+1}^{\pi^*}$, and that the optimal value iteration (4.22) holds for the considered step t. Moreover, since the value function $V_{t+1}^{\pi^*}$ for deterministic policy π^* is $\mathcal{B}_{\mathcal{H}_t}$-measurable, so is V_t^*, which concludes the induction step. For $t = 0$, equality $V_0^* = V_0^{\pi^*}$, shows that the maximizing policy π^* is optimal. □

For a maximizing policy π^* to be a policy in the sense of Definition 4.9 ,and for the application of Theorem 4.16, at step $t \in [T-1]$, π_t^* not only has to select an action that maximizes the right hand side of (4.21), but has to do so in a measurable way.

Theorem 4.19 (Existence of maximizing policy)**.** *For the HDP (4.1), there exists a maximizing policy π^* (4.21).*

Proof. For $t \in [T-1]$, the reward function R_t is by assumption $\mathcal{B}_{\mathcal{H}_t} \otimes \mathcal{B}_{\mathcal{A}}$-measurable. By Lemma A.6, and the expression of the conditional expectation as integral, the same holds true for the map

$$(h_t, a) \mapsto \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V_{t+1}^*(h_t, a, O_{t+1})],$$

[4] Measurability of V_{t+1}^* actually follows from the equality $V_{t+1}^* = V_{t+1}^{\pi^*}$ and the measurability of $V_{t+1}^{\pi^*}$.

and thus, also the for the function $g : \mathcal{H}_t \times \mathcal{A}$ defined as

$$g(h_t, a) := R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}[V^*_{t+1}(h_t, a, O)].$$

It follows from the Measurable Selection Theorem in Appendix A.2, by setting $(\Omega, \mathcal{F}) = (\mathcal{H}_t, \mathcal{B}_{\mathcal{H}_t})$ and with finite action space $\mathcal{A}$, that there exists a maximizer $\pi^*_t : \mathcal{H}_t \to \mathcal{A}$ of g, which satisfies $\pi^*(h_t) \in \arg\max_{a\in\mathcal{A}} g(h_t, a)$ for each $h_t \in \mathcal{H}_t$, and that the map

$$h_t \mapsto g^*(h_t) := \max_{a\in\mathcal{A}} g(h_t, a) = g(h_t, \pi^*(h_t))$$

is $\mathcal{B}_{\mathcal{H}_t}$-measurable. □

Under certain conditions, one could in principle find optimal value functions and optimal policies explicitly via dynamic programming. Assume that, in addition to $\mathcal{A}$, the observation space $\mathcal{O}$ is finite, and that the transition kernel P_t and the reward function R_t are known at each step t. This allows one to solve the optimal value iteration (4.22) through evaluating for each history h_t the term in the curly brackets, and subsequently selection an action that maximizes it, which gives the optimal value function at step t. By keeping a record of the set of maximizing actions at each step $t \in [T]$, and for all histories h_t, optimal policies can be derived by choosing from those sets. However, evaluating (4.22), would mean to compute the expectation

$$\sum_{o\in\mathcal{O}} P_t(o|h_t, a)V^*_{t+1}(h_t, a, o)$$

for each possible history at each step. With $|\mathcal{A}| = n \in \mathbb{N}$ and $|\mathcal{O}| = m \in \mathbb{N}$, the number of possible histories over the whole episode is $|\mathcal{H}_T| = (mn)^T m$. Thus, computing the expectation at time $t \in [T-1]$ requires $|\mathcal{H}_t||\mathcal{O}||\mathcal{A}| = (|\mathcal{O}||\mathcal{A}|)^t|\mathcal{O}||\mathcal{O}||\mathcal{A}| = m^{t+2}\, n^{t+1}$ multiplications, and the whole dynamic programming procedure amounts to

$$\sum_{t=0}^{T-1} |\mathcal{H}_t||\mathcal{O}||\mathcal{A}| = \sum_{t=0}^{T-1} m^{t+2}\, n^{t+1}$$

operations, rendering this approach infeasible even for small action and observation spaces, and short episodes length T.

5 Signature-Q-Learning

We now turn to the question of how to find a maximizing policy (4.21) in a reinforcement learning setting, in which transition probabilities and reward functions are unknown. This will again require Q-functions, which in the history-based setting, will be defined as functions of histories, and we provide an explicit method, *Signature-Q-Learning*, to learn estimates approximating them.

5.1 Q-Functions in History-Based Decision Processes

Consider a HDP given by Equation (4.1). Based on the optimal value functions (4.20) we define optimal Q-functions for history-action pairs $(h_t, a) \in \mathcal{H}_t \times \mathcal{A}$.

Definition 5.1 (Optimal Q-functions in HDPs)**.** For an HDP given by Equation (4.1), the *optimal Q-function* $Q_t^* : \mathcal{H}_T \times \mathcal{A} \to \mathbb{R}$ at time $t \in [T-1]$ is defined as

$$Q_t^*(h_t, a) := R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}\big[V_{t+1}^*(h_t, a, O_{t+1})\big], \tag{5.1}$$

for any $(h_t, a) \in \mathcal{H}_t \times \mathcal{A}$, and for $t = T$ as $Q_T(h_T, a) := R_T(h_T)$ for any $(h_T, a) \in \mathcal{H}_T \times \mathcal{A}$.

The optimal Q-value $Q_t^*(h_t, a)$, gives the maximal attainable value over the remaining episode, if action a is chosen in history h_t at time t. This value is decomposed into the reward generated from the pair (h_t, a) at time t, and the expected optimal value from following an optimal policy from $t+1$ onwards. The optimal value functions and optimal Q-functions are connected in the following way,

$$V_t^*(h_t) = \max_{a \in \mathcal{A}} Q_t^*(h_t, a), \tag{5.2}$$

and by the definition of Q_t^* and the characterization as maximizing policy in Theorem 4.16, an optimal policy π^* satisfies

$$\pi_t^*(h_t) \in \arg\max_{a \in \mathcal{A}} Q_t^*(h_t, a), \quad \text{for } h_t \in \mathcal{H}_t, \tag{5.3}$$

and each $t \in [T-1]$. A policy that satisfies (5.3) for all $h_t \in \mathcal{H}_t$ at each $t \in [T-1]$, is called *greedy* with respect to the collection $(Q_t^*)_{t \in [T]}$. Analogue to the MDP case, (5.3) implies that knowledge of Q_t^* is sufficient for acting optimally in the underlying HDP, by following a policy that is greedy w.r.t Q^*. Substituting (5.2) for V_{t+1}^* in Equation (5.1) yields the *optimal Q-iteration in HDPs*,

$$Q_t^*(h_t, a) = R_t(h_t, a) + \gamma\, \mathbb{E}_{P_t(\mathrm{d}o|h_t,a)}\Big[\max_{a' \in \mathcal{A}} Q_{t+1}^*((h_t, a, O_{t+1}), a')\Big], \tag{5.4}$$

for each $(h_t, a) \in \mathcal{H}_t \times \mathcal{A}$, with the terminal condition $Q_T^*(h_T, a) = R_T(h_T)$ for each $(h_T, a) \in \mathcal{H}_T \times \mathcal{A}$. In contrast to the optimal value iteration (4.22), the maximum in the optimal Q-iteration is taken inside the expectation. This enables us to sample Equation (5.4) through sampling histories h_T by interaction with the environment, and utilize those samples to learn estimates of $(Q_t^*)_{t\in[T]}$ exploiting their recursive structure. If the learned estimates are close to the true optimal Q-functions, a policy that is greedy with respect to the estimates is close to optimal. The optimal Q-iteration will thus be the basis for updating the estimates in the proposed method.

5.2 The Signature-Q-Learning Algorithm

This section introduces the reinforcement learning method *Signature-Q-Learning*, details the method's approximation argument and provides an algorithm for the procedure. Analogous to Q-Learning in MDPs, Signature-Q-Learning learns an estimate of the optimal Q-functions $(Q_t^*)_{t\in[T]}$, albeit in history-dependent domains modeled through HDPs. In this setting function approximation is crucial, since an enumeration of the high-dimensional inputs is not feasible, even if the observation space was finite. Given the signature's properties, detailed in Section 2.1, it is a natural choice to act as feature map on history spaces. More precisely, our aim is to approximate the optimal Q-functions at all time steps $t \in [T]$ by a single estimate, which is parametrized as a linear functional $\ell \in T^{(N)}(\mathbb{R}^d)$, acting on the time-augmented histories' signatures. We justify this approximation by the universal approximation property from Theorem 2.15, which holds uniformly in time. The functional's parameters[1] are updated utilizing the optimal Q-iteration (5.4) with histories sampled through interaction with the environment in an online fashion.

To apply Theorem 2.15 as the algorithm's approximation argument, we will interpret the optimal Q-functions $(Q_t^*)_{t\in[T]}$ as one functional acting on the space of linearly interpolated, time-augmented histories. Recall the definitions of the metric spaces $(\Omega_t^0, d_{1,[0,t]})$, for $t \in [0, T]$, and $(\Lambda_T, d_{\Lambda_T})$ from Section 2.1, where here we will consider paths $X \in \mathcal{V}^0([0, T], \mathbb{R}^{d+2})$. We make the following assumption, with the objective of removing the translation invariance of the 1-variation norm (2.1) of an interpolated path by fixing the histories' starting value. A discussion on this assumption in practice is provided below in Remarks 5.3.

Assumption A. *The sets $\mathcal{O}$ and $\mathcal{A}$ contain the origin in $\mathbb{R}^d$ and $\mathbb{R}$, respectively, and for the remaining section we consider histories starting at $o_0 = 0 \in \mathbb{R}^d$ and $a_0 = 0 \in \mathbb{R}$. To emphasise this, for $t \in [T]$, let $\hat{\mathcal{H}}_t^0 = \{\hat{h}_t \in \hat{\mathcal{H}}_t \mid \hat{h}_0 = (0, o_0, a_0) = 0 \in \mathbb{R}^{d+2}\}$ be the space of all time-augmented, time-t histories starting at the origin.*

Fix $t \in [T]$. For a time-augmented history $\hat{h}_t \in \hat{\mathcal{H}}_t^0$, recall that its concatenation with an action $a \in \mathcal{A}$ is given as the stream of data $\hat{h}_t^a = (u, o_u, a_u)_{u\in[t]} \in \hat{\mathcal{H}}_t^0 \times \mathcal{A}$, where we set $a_t := a$, and with components $(u, o_u, a_u) \in \mathbb{R}^{d+2}$ for $u \in [t]$. Its linear interpolation $\hat{H}^a : [0, t] \to \mathbb{R}^{d+2}$ is a continuous, piecewise linear path of finite variation with values in $\mathbb{R}^{d+2}$, starting at the origin. Define the space of all trajectories on $[0, t]$ of such linearly

[1] We identify $\mathbb{R}^{d_N}$ with the truncated tensor algebra $T^{(N)}(\mathbb{R}^d)$ up to level N of dimension d_N without specifying an explicit isomorphism.

interpolated, time-augmented histories of length $t \in [T]$ as

$$\mathcal{K}_t^0 := \{\hat{H}^a_{[0,t]} \mid \hat{H}^a \text{ is linear interpolation of } \hat{h}_t^a \in \hat{\mathcal{H}}_t^0 \times \mathcal{A}\}. \tag{5.5}$$

Further, the union of all linearly interpolated, continuous-time histories of arbitrary length is defined as

$$\Lambda_{[T]} := \bigcup_{t \in [T]} \mathcal{K}_t^0. \tag{5.6}$$

Due to Assumption A we have $\mathcal{K}_t^0 \subset \Omega_t^0$ for all $t \in [T]$, and thus $\Lambda_{[T]} \subset \Lambda_T$. Additionally, the following lemma holds.

Lemma 5.2. *For each $t \in [T]$, the set $\mathcal{K}_t^0 \subset \Omega_t^0$ is a compact with respect to the topology induced by $d_{1,[0,t]}$ on Ω_t^0. Moreover, the set $\Lambda_{[T]} \subset \Lambda_T$ is compact with respect to the topology induced by d_{Λ_T} on Λ_T.*

Proof. For any $m \in \mathbb{N}$ we consider $\mathbb{R}^m$ endowed with the 1-norm $\|\cdot\|_{1,\mathbb{R}^m}$.[2] We use the notation

$$h_t^a = (o_0, a_0, \dots, o_t, a_t), \quad \text{and} \quad \mathfrak{h}_t^{\mathfrak{a}} = (\mathfrak{o}_0, \mathfrak{a}_0, \dots, \mathfrak{o}_t, \mathfrak{a}_t),$$

to differentiate between two (not necessarily) distinct histories in $\mathcal{H}_t \times \mathcal{A}$ for any $t \in [T]$, and analogously their time-augmented versions $\hat{h}_t^a$ and $\hat{\mathfrak{h}}_t^{\mathfrak{a}}$.

For the first claim, recall that $\mathcal{O} \subset \mathbb{R}^d$ and $\mathcal{A} \subset \mathbb{R}$ are by assumption compact, which implies the finite product $\mathcal{H}_{t-1} \times \mathcal{A} \subset \mathbb{R}^{d+1}$ is compact. Since $\hat{\mathcal{H}}_t^0 \times \mathcal{A}$ is canonically homeomorphic to $\mathcal{H}_{t-1} \times \mathcal{A}$ it is also compact. If we show that the interpolation map $\Phi_t : \hat{\mathcal{H}}_t^0 \times \mathcal{A} \to \Omega_t^0$, which sends a history $\hat{h}_t^a$ to its linear interpolation $\Phi(h_t^a) = \hat{H}^a_{[0,t]}$, is continuous, then $\Phi_t(\hat{\mathcal{H}}_t^0 \times \mathcal{A}) = \mathcal{K}_t^0$ is compact in Ω_t^0 as the continuous image of a compact set. For a piecewise linear path with values in $\mathbb{R}^{d+2}$, its total variation is simply the sum of the norms of its successive linear increments, and the difference of two piecewise linear paths is again piecewise linear. It follows by a simple calculation that the 1-variation distance between the linear interpolations for any two histories $\hat{h}_t^a, \hat{\mathfrak{h}}_t^{\mathfrak{a}} \in \mathcal{H}_t^0 \times \mathcal{A}$ is

$$\begin{aligned} d_{1,[0,t]}(\Phi_t(\hat{h}_t^a), \Phi_t(\hat{\mathfrak{h}}_t^{\mathfrak{a}})) &= \|\Phi_t(\hat{h}_t^a) - \Phi_t(\hat{\mathfrak{h}}_t^{\mathfrak{a}})\|_{1,[0,t]} \\ &= \sum_{u=1}^{t} \Big(\|(o_u - o_{u-1}) - (\mathfrak{o}_u - \mathfrak{o}_{u-1})\|_{1,\mathbb{R}^d} + |(a_u - a_{u-1}) - (\mathfrak{a}_u - \mathfrak{a}_{u-1})| \Big). \end{aligned}$$

The right hand side is a finite sum of continuous functions of the coordinates of $\hat{h}_t^a$ and $\hat{\mathfrak{h}}_t^{\mathfrak{a}}$, where the time component vanishes since it is the same in both histories, hence Φ_t is continuous and the first claim follows.

For the second claim, note that for each $t \in [T]$ the canonical embedding $\Omega_t^0 \hookrightarrow \Lambda_T$ is continuous with respect to d_{Λ_T}, which implies $\mathcal{K}_t^0$ remains compact when viewed as subset of

[2] Note that for finite products $C := C_1 \times \cdots \times C_n$, with $C_i \subset \mathbb{R}^{m_i}$ and $n, m_i \in \mathbb{N}$, the product topology on C coincides with the subspace topology from $\mathbb{R}^{m_1 \cdots m_n}$.

Λ_T. Finally, $\Lambda_{[T]}$ is defined as a finite union of compact sets (5.6), and hence itself compact in the metric space $(\Lambda_T, d_{\Lambda_T})$. □

Remarks 5.3. (i) Without fixing the starting value of histories in Assumption A, the 1-variation distance $d_{1,[0,t]}$ would be translation invariant in the observation and action component and thus only defines a pseudometric on $\mathcal{K}_t^0$ rather than a true metric. In the case $(0,0) \notin \mathcal{O} \times \mathcal{A}$, the lemma above remains valid, if we assume the first point of each history is fixed at an arbitrary $(o,a) \in \mathcal{O} \times \mathcal{A}$ and translate all streams by $-(o,a)$. The resulting space of translated linear interpolations is compact, and the original space is obtained from this by a continuous translation. Moreover, the requirement $a_0 = \alpha$ for some fixed $\alpha \in \mathcal{A}$ can be omitted if Ω_t^0 is redefined to contain all paths whose d components, which correspond to the observations in a linearly interpolated path, start at the origin, while the component corresponding to the actions may start at an arbitrary value. Fixing the initial observation already removes the translation invariance of the 1-variation distance.

(ii) In applications one can always add a fixed artificial basepoint (o_{-1}, a_{-1}) prior to any actually observed history $\hat{h}_t$ to make sure that Assumption A is satisfied. Adding a basepoint furthermore helps with the calculation of the signature at the initial step $t = 0$ which is further discussed on Section 5.3. Moreover, we empirically evaluate the effect of using different basepoint values in the experiment in Section 6.2.

As next step, we define the path functional $Q^* : \Lambda_{[T]} \to \mathbb{R}$ such that for any linearly interpolated history $\hat{H}^a_{[0,t]} \in \Lambda_{[T]}$, it recovers the respective time-t optimal Q-function with input $(h_t, a) \in \mathcal{H}_t \times \mathcal{A}$, from which the interpolation was derived, i.e.

$$Q^*\Big(\hat{H}^a_{[0,t]}\Big) := Q^*(t, h_t, a) = Q^*_t(h_t, a). \tag{5.7}$$

Note that the dependence of Q^* on the time component in $\hat{H}^a_{[0,t]}$ is only through its last point (t, o_t, a). The functional Q^* is going to be learned in a RL setting, justified via the universal approximation property for signatures from Theorem 2.15. Assume Q^* is continuous on the subset of all linearly interpolated paths $\Lambda_{[T]} \subset \Lambda_T$. Since $\Lambda_{[T]}$ is compact it is also closed, and by Tietze's extension theorem [36, Satz 12.10.3], there exists a continuous extension of the functional Q^* to all of Λ_T which agrees with the original Q^* on $\Lambda_{[T]}$.[3] With a slight abuse of notation we denote this extension also by Q^*. Since $\mathcal{K}_T^0 \subset \Omega_T^0$ is compact it follows by Theorem 2.15 that for any $\epsilon > 0$, there exists a linear functional $\ell \in T(\mathbb{R}^{d+2})$, such that

$$\sup_{\hat{H}^a \in \mathcal{K}_T^0} \sup_{t \in [0,T]} \Big| Q^*(\hat{H}^a_{[0,t]}) - \langle \ell, S_t(\hat{H}^a) \rangle \Big| < \epsilon. \tag{5.8}$$

Taking the supremum over $\mathcal{K}_T^0$ is equivalent to taking the supremum over histories in $\mathcal{H}_T^0 \times \mathcal{A}$, and then passing their time-augmented linear interpolations as inputs in (5.8). Since $[T] \subset [0,T]$, and by the construction of Q^*, such that it agrees with the true optimal

[3] Specifically, there exists a continuous map $F : \Lambda_T \to \mathbb{R}$ with $F|_{\Lambda_{[T]}} = Q^*$ and $\sup\{|F(\hat{X}_{[0,t]})| : \hat{X}_{[0,t]} \in \Lambda_T\} = \sup\{|Q^*(\hat{H}^a_{[0,t]})| : \hat{H}^a_{[0,t]} \in \Lambda_{[T]}\}$.

Q-functions $(Q_t^*)_{t\in[T]}$ on $\Lambda_{[T]}$, we obtain

$$\sup_{h_T^a \in \mathcal{H}_T^0 \times \mathcal{A}} \sup_{t\in[T]} \left| Q_t^*(h_t, a_t) - \langle \ell, S_t(\hat{h}_T^a) \rangle \right| < \epsilon, \tag{5.9}$$

where the time-t signature of the discrete-time history $\hat{h}_T^a$ is defined as the signature of its linear interpolation $S_t(\hat{h}_T^a) = S_t(\hat{H}^a)$ as in Definition 2.16, and the input $(h_t, a_t) = h_t^{a_t}$ to the true time-t optimal Q-function Q_t^* is understood as the truncation of the time-T history h_T^a at step $t \in [T]$. Equation (5.9) constitutes the main approximation argument of the proposed Signature-Q-Learning method. We highlight that this approximation to the true optimal Q-functions is realized uniformly in $t \in [T]$ via a single linear functional $\ell \in T(\mathbb{R}^{d+2})$.

Remark 5.4. For the applicability of Theorem 2.15 the continuity of Q^* w.r.t to the metric d_{Λ_T} on $\Lambda_{[T]}$ is required, which is implicitly assumed for the experiments in Chapter 6. To ensure this assumption analytically, additional smoothness assumptions on the observation kernels $\mathcal{P}$ and reward functions $\mathcal{R}$ of the HDP model (4.1) are necessary. An alternative to constructing Q^* and estimating it via Theorem 2.15 would be to approximate Q_t^* at each $t \in [T]$ with a separate linear functional $\ell^t \in T^{(N)}(\mathbb{R}^{d+2})$. This approach relies on a similar approximation theorem for paths of fixed lengths [39, Theorem 1]. It does not require the continuity of Q^* in the time component, but rather continuity of Q_t^* in (h_t, a) for each $t \in [T]$, which is a weaker assumption. The elegance of Signature-Q-Learning, however, is its uniform approximation over time, which is furthermore advantageous from a computational perspective, as training separate linear functionals ℓ_k^t at each step requires substantially less sample efficient.

In the implementation we have to truncated signatures at some level, and the Signature-Q-Learning algorithm aims at learning a truncated version of a linear functional ℓ that satisfies (5.9) for some $\epsilon > 0$. The procedure is as follows. Fix a truncation order $N \in \mathbb{N}$. For any linear functional $\ell \in T^{(N)}(\mathbb{R}^{d+2})$, we define the *approximate Q-function corresponding to* ℓ, acting on the space of all time-augmented histories of arbitrary length, as

$$Q(\,\cdot\,|\,\ell) : \bigcup_{t\in[T]} (\hat{\mathcal{H}}_t^0 \times \mathcal{A}) \to \mathbb{R} \quad \text{by} \quad Q(\hat{h}_t^a | \,\ell) := \langle \ell, S_t^{(N)}(\hat{h}_t^a) \rangle, \tag{5.10}$$

where the truncated signature up to level N of the discrete time-history $\hat{h}_t^a$ is defined as the truncated signature of its linear interpolation $\hat{H}_{[0,t]}^a$. The algorithm starts with some initial linear functional $\ell_0 \in T^{(N)}(\mathbb{R}^{d+2})$, where the parameters of ℓ_0 are initialized randomly, and its corresponding approximate Q-function $Q(\,\cdot\,|\,\ell_0)$. Let $\ell_k \in T^{(N)}(\mathbb{R}^{d+2})$ be the linear functional after the k-th update step, such that $Q(\,\cdot\,|\,\ell_k)$ is the algorithms approximation to Q^* after update step $k = mt$ corresponding to step $t \in [T]$ in episode $m \in \mathbb{N}$. By following some exploration policy, for example an ε-greedy policy with respect to the current approximate Q-function $Q(\,\cdot\,|\,\ell_k)$, see Remarks 5.5, we sample histories and rewards. At time $t \in [T-1]$ in the current episode, the exploration policy selects action $a_t \in \mathcal{A}$, based on the history $h_t \in \mathcal{H}_t$ observed so far. Subsequently, we receive the reward $r_t = R_t(h_t, a_t)$, and the environment transitions to the next observation o_{t+1}. This results in a transition

sample (h_t, a_t, r_t, o_{t+1}), from which we create the *Q-target value* y_t at step t for update step $k+1$ as

$$y_t := r_t + \gamma \max_{a \in \mathcal{A}} Q(\hat{h}_{t+1}^{a} \mid \ell_k), \tag{5.11}$$

based on the current Q^* estimate provided by ℓ_k. The Q-target value y_t constitutes a sample of the right-hand side of the optimal Q-iteration (5.4). It is a supposedly better estimate of the true value $Q_t^*(h_t, a_t)$ than our current estimate $Q(\hat{h}_t^{a_t} \mid \ell_k)$, due to the actual reward r_t received from the environment, and the iterative nature of the optimal Q-functions in (5.4). Thus, the linear functional ℓ_{k+1}, which provides an updated approximate Q-function, is selected such that it minimizes the error between the Q-target value y_t and the estimate for the true optimal Q-value of the tuple (h_t, a_t), measured by some loss functional $\mathcal{L} : \mathbb{R}^2 \to \mathbb{R}$, concretely,

$$\ell_{k+1} \in \arg\min_{\ell} \mathcal{L}(y_t, Q(\hat{h}_t^{a_t} \mid \ell)).$$

In Signature-Q-Learning the parameters of the functional ℓ_k are updated towards this minimum by one-step-gradient descent. If $\mathcal{L}$ is given as the squared error loss $\mathcal{L}(x, y) = \frac{1}{2}(x-y)^2$, for $x, y \in \mathbb{R}$, the update amounts to

$$\ell_{k+1} \leftarrow \ell_k + \alpha_t \left(y_t - Q(\hat{h}_t^{a_t} \mid \ell_k)\right) \nabla_\ell Q(\hat{h}_t^{a_t} \mid \ell)\Big|_{\ell=\ell_k}, \tag{5.12}$$

with learning rate $\alpha_k \geq 0$ at update step k. After a total number of $M \in \mathbb{N}$ episodes and $K = MT$ update steps the algorithms has learned the approximate Q-function $Q(\cdot \mid \ell_K)$ as final estimate for the true optimal Q-functions $(Q_t^*)_{t \in [T]}$. We can then follow a policy $\pi^K \in \Pi^{\mathrm{HD}}$ that is greedy with respect to $Q(\cdot \mid \ell_K)$, by selecting the action at step $t \in [T-1]$ as

$$a_t = \pi^K(h_t) \in \arg\max_{a \in \mathcal{A}} Q(\hat{h}_t^{a} \mid \ell_K). \tag{5.13}$$

for any $h_t \in \mathcal{H}_t$. If the learned function $Q(\cdot \mid \ell_K)$ is a sufficiently good approximation of the true optimal Q-functions, then π^K is approximately optimal. Note, that the policy π^K can be considered a stationary in the sense that it applies the time-invariant decision rule $\arg\max_{a \in \mathcal{A}} \langle \ell, \cdot \rangle$ to any input element from $\bigcup_{t \in T} S_t^{(N)}(\hat{\mathcal{H}}_T^0 \times \mathcal{A}) \subset T^{(N)}(\mathbb{R}^{d+2})$. The full Signature-Q-Learning procedure is provided in Algorithm 2, and Remarks 5.5 comment on some aspects of it.

Remarks 5.5. (i) The update of the estimate $Q(\cdot \mid \ell_k)$ towards the target value y_t given by Equation (5.12), corresponds to the usage of the squared error loss $\mathcal{L}(x, y) = \frac{1}{2}(x-y)^2$. Other loss functions may be used, such the absolute error loss $\mathcal{L}(x, y) = |x - y|$ or the smooth-L_1 loss,

$$\mathcal{L}(x, y) = \begin{cases} \frac{1}{2}(x-y)^2, & \text{if } |x-y| < 1, \\ |x-y| - \frac{1}{2}, & \text{else,} \end{cases} \tag{5.14}$$

which combines the absolute loss with the squared loss in a continuously differentiable way. In the numerical experiments presented in Chapter 6, the smooth-L_1 is used.

(ii) At time $T-1$ of each episode, the Q-target value in Line 15 of Algorithm 2 is provided as the sum $r_{T-1} + r_T$. Inline with the handling of terminal rewards in episodic

Algorithm 2 Signature-Q-learning

Require: Number of episodes $M \in \mathbb{N}$, signature truncation order $N \in \mathbb{N}$, initial linear functional $\ell_0 \in T^{(N)}(\mathbb{R}^{d+2})$, loss function $\mathcal{L}$, learning rates $\alpha_{n,t}$, exploration rates $\varepsilon_{n,t}$, for $t \in [T-1]$ and $n \in \mathbb{N}$.

1: **for** episode $n = 1, 2, \ldots, M$ **do**
2: Observe o_0 and set $\hat{h}_0 = (0, o_0)$.
3: **if** basepoint = True **then**
4: Append basepoint to $\hat{h}_0$ at time step $t = -1$ (details in Section 5.3).
5: **end if**
6: Calculate $S_0^{(N)}(\hat{h}_0^a)$ for each $a \in \mathcal{A}$.
7: **for** time step $t = 0, 1, \ldots, T-1$ **do**
8: Sample action a_t according to ε-greedy policy based on current estimate ℓ_k:

$$a_t = \begin{cases} \text{selected from } \arg\max_{a\in\mathcal{A}} \langle \ell_k, S_t^{(N)}(\hat{h}_t^a)\rangle, & \text{with probability } 1-\varepsilon_{n,t}, \\ \text{sampled uniformly from } \mathcal{A}, & \text{with probability } \varepsilon_{n,t}. \end{cases}$$

9: Take action a_t, receive reward r_t and observe o_{t+1}.
10: **if** $t = T-1$ **then**
11: Receive terminal reward r_T.
12: **end if**
13: Update the time-augmented history $\hat{h}_{t+1} = (\hat{h}_t, a_t, t+1, o_{t+1})$.
14: Update the respective signature using Chen's identity,

$$S_{t+1}^{(N)}(\hat{h}_{t+1}^a) = S_t^{(N)}(\hat{h}_t^{a_t}) \otimes S_1^{(N)}(\hat{h}_{t:t+1}^a), \quad \text{for each } a \in \mathcal{A}.$$

15: Create the Q-target value as

$$y_t = \begin{cases} r_t + r_T & \text{if } t = T-1, \\ r_t + \max_{a\in\mathcal{A}} \langle \ell_k, S_{t+1}(\hat{h}_{t+1}^a)\rangle, & \text{else.} \end{cases}$$

16: Update the linear functional parameters via stochastic gradient descent with learning rate $\alpha_{n,t}$ and loss function $\mathcal{L}$,

$$\ell_{k+1} \leftarrow \ell_k + \alpha_{n,t} \nabla_\ell \mathcal{L}\Big(y_t, \langle \ell, S_t^{(N)}(\hat{h}_t^{a_t})\rangle\Big)\Big|_{\ell=\ell_k}$$

17: **end for**
18: **end for**

reinforcement learning tasks [58], no bootstrapping takes place at time T, since there are no more rewards available afterwards, and as such the last target value in each episode is given as $y_{T-1} = r_{T-1} + r_T$.

(iii) Algorithm 2 applies ε-greedy as exploration policy in Line 8. This exploration scheme is often used in online Q-Learning for MDPs and derives its name from the parameters $\varepsilon_{n,t} \in [0,1]$, that govern the exploration-exploitation trade-off. At each step t, action a_t is sampled uniformly from $\mathcal{A}$ with probability $\varepsilon_{n,t}$, and otherwise selected greedily based on the current estimate $Q(\cdot \mid \ell_k)$ as in Equation (5.13). Setting $\varepsilon_{n,t} = \varepsilon \in [0,1]$ for all $n \in \mathbb{N}$ and $t \in [T]$ amounts to a constant exploration rate and ensures that each action is sampled infinitely often if the number of episodes n tends to infinity. On the other hand, the exploration rates $\varepsilon_{n,t}$ may also be defined as a decreasing series approaching some value $\varepsilon_\infty \geq 0$ for $n \to \infty$ and $t \to T$, to ensure high exploration early during training, and a focus on exploitation later on when the learned approximate Q-functions are already more accurate. Possible decay schemes include exponential and linear decays, with examples displayed in Figure 5.1. The decay may take place over update steps or episode-wise where $\varepsilon_{n,t} = \varepsilon_n$ for all $t \in [T]$ and episode $n \in \mathbb{N}$. Note, that the decision about whether action a_t is sampled uniformly or not is made *for each* action. Since the objective of Signature-Q-Learning is to select actions which maximize future expected values *based on histories* instead of single observations or states, an alternative exploration approach could be to decide at the beginning of each episode if it includes uniformly sampled actions or not. This would provide the algorithm with a mix of histories including exploration, alternating with histories for which actions are exclusively selected greedily w.r.t. to the current estimate alone.

(iv) Instead of an ε-greedy exploration policy in Algorithm 2, one might use a softmax exploration policy π_τ, based on the current optimal Q-function estimate. Specifically, with current linear functional ℓ_k, and given history h_t, the probability of sampling action $a \in \mathcal{A}$ under π_τ is given by

$$\mathbb{P}^{\pi_\tau}[A_t = a \mid H_t = h_t] = \frac{\exp\Big(\tau^{-1}\Big\langle \ell_k, S_t^{(N)}(\hat{h}_t^a)\Big\rangle\Big)}{\sum_{\alpha \in \mathcal{A}} \exp\Big(\tau^{-1}\Big\langle \ell_k, S_t^{(N)}(\hat{h}_t^\alpha)\Big\rangle\Big)}, \tag{5.15}$$

where $\tau > 0$ is the exploration parameter, called temperature, that determines the exploration behaviour. For $\tau \to 0+$, the policy π_τ converges to the uniform distribution on $\mathcal{A}$, while for $\tau \to \infty$, the sampling policy becomes greedy w.r.t the current estimate and the action with the highest estimated expected reward is selected. Analogue to ε-greedy, decay schemes such as the ones displayed in Figure 5.1 may be employed to the temperature parameter τ to steer exploration behaviour during training.

(v) Algorithm 2 constitutes an online algorithm, where the estimates $Q(\cdot \mid \ell_k)$ are updated at each step while interacting with the environment, and, with ε-greedy as exploration policy, the updated estimate determines the action selection at the subsequent step. It is also possible to design the algorithm in an offline fashion, where first, a number of full histories $\{h_T^i\}_{i=0}^n, n \in \mathbb{N}$ are sampled by following some fixed exploration policy

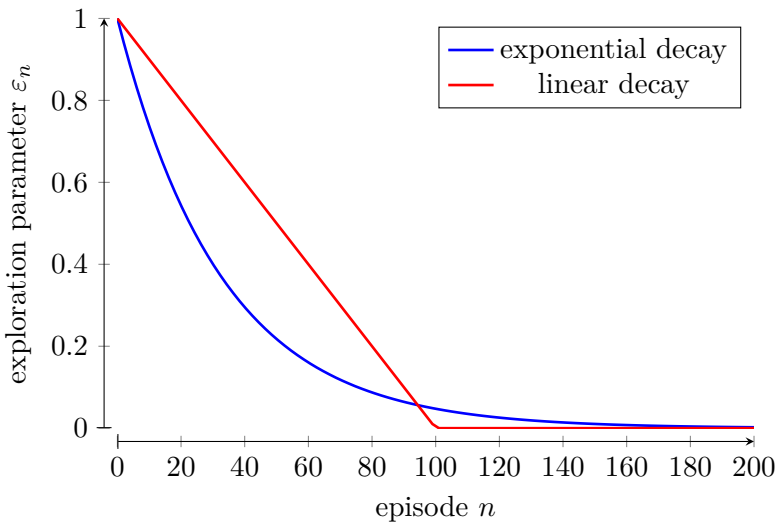


Figure 5.1: Two exemplary decay schemes for the parameters $\varepsilon_{n,t}$ of the ε-greedy exploration policy from Algorithm 2 are displayed. Parameter values are assumed to be fixed during episodes, i.e. $\varepsilon_n := \varepsilon_{n,0} = \varepsilon_{n,t}$ for all steps $t \in [T]$. For the starting value $\varepsilon_0 = 1$, the linear decay scheme reduces the value linearly to zero over 100 episodes, and leaves it fixed for the remaining 100 episodes. The exponential decay reduces the initial value by the constant factor 0.97 over 200 episodes.

π, and only afterwards the linear functional's parameters are learned, with updates determined by the sampled histories. The key difference in the offline version, is that the exploration policy π is not informed by any already learned estimates and as such, there is no exploitation component in the action selection when collecting histories to learn from at a later step. Depending on the environment, it may be hard to collect histories with high value only though random action selection. An example is the environment used in the experiment in Section 6.2, in which the algorithm must learn to reach a predetermined goal in as few steps as possible. If actions are selected, for example, solely by uniformly sampling the action space, the probability of reaching the goal is close to zero. Thus, the collected sample of histories does not contain any information about the goal, and in the following training phase, the algorithm likely fails to learn an approximate Q-function which is sufficiently close to the true optimal Q-functions.

5.3 Implementation of Approximate Q-Functions

Recall that $\mathcal{O} \subset \mathbb{R}^d$ and $\mathcal{A} \subset \mathbb{R}$ with $|\mathcal{A}| = m < \infty$, and a time-augmented, action-concatenated history $\hat{h}_t^a \in \mathcal{H} \times \mathcal{A}$ is the stream of data $\hat{h}_t^a = (i, o_i, a_i)_{i=0}^t$ is a stream of data with sample points in $\mathbb{R}^{d+2}$ for any $a_t \in \mathcal{A}$ and $\hat{h}_t \in \mathcal{H}_t$. As already mentioned in Section 2.2, the computation of the signature of linearly interpolated streams of data may be handled with various packages in Python [37, 45, 61]. To implement the linear functional

computationally on top of the signature computation, one may again exploit existing libraries, that provide machine learning functionalities. Popular Python libraries include the packages `PyTorch` [59] and `TensorFlow` [1]. The advantage of the usage of such packages is, that it readily provides access to back-propagation functionalities, gradient-based optimization algorithms such as ADAM [38] and methods to include the decay of hyperparameters in training as displayed in Figure 5.1. By identifying a linear functional $\ell \in T^{(N)}(\mathbb{R}^{d+2})$ as a parameter vector $\theta_\ell \in \mathbb{R}^{d_N}$, with d_N the dimension of $T^{(N)}(\mathbb{R}^{d+2})$, it can be implemented as single linear neural network layer, which takes as input the time-augmented histories' signature, again identified with a vector in $\mathbb{R}^{d_N}$, and outputs approximate Q-values for each action. Given $\hat{h}_t \in \mathcal{H}_t$, the approximate Q-values for $(\hat{h}_t^a)_{a\in\mathcal{A}}$ are then obtained by looping through actions and evaluating the vector product

$$Q(\hat{h}_t^a \mid \ell) = \theta_\ell^\top \cdot S^{(N)}(\hat{h}_t^a).$$

With this approach there is a single parameter vector θ_ℓ for all actions that is updated at each step, irrespective of the action that was chosen. Further, this requires the application of Chen's identity to compute the updated signature $S^{(N)}(\hat{h}_{t+1}^a)$ in Line 14, $|\mathcal{A}|$-times, before selecting the maximal action value.

A second approach is to use $\hat{h}_t$ as input to the linear functional, without the concatenation with next possible action at the end of the history, and have the linear functional output multiple approximate Q-values, each corresponding to one action. Since $\hat{h}_t$ contains an uneven number of actions and observations, by concatenating a dummy action[4] $\alpha \in \mathbb{R}$ at the beginning of $\hat{h}_t$ before $(0, o_0)$, we obtain the stream of data ${}^\alpha\hat{h}_t := (a_i, i, o_i)_{i=0}^t$, where $a_0 := \alpha$, again with sample points in $\mathbb{R}^{d+2}$. The signature of ${}^\alpha\hat{h}_t$ can readily be computed, and the linear functional ℓ is then implemented via the parameters $\Theta = (b, W)$, with $b \in \mathbb{R}^m$ and $W \in \mathbb{R}^{m\times d_N}$, corresponding to the usual bias vector and weight matrix of a linear neural network layer, which outputs the vector of approximate Q-values by

$$(Q(\hat{h}_t^{a_1} \mid \ell), \dots, Q(\hat{h}_t^{a_m} \mid \ell)) = \Theta\, S^{(N)}({}^\alpha\hat{h}_t)^\top = b + W\big(S^1({}^\alpha\hat{h}_t), \dots, S^N({}^\alpha\hat{h}_t)\big)^\top. \tag{5.16}$$

This approach avoids looping over actions and requires only one evaluation of the linear layer (5.16) to provide the Q-values for all actions. Note, that this implementation effectively amounts to one linear functional per action, which for action $a_i \in \mathcal{A}$ is given by the i-th bias vector component b_i and i-th weight matrix row $(W_{i,j})_{j=1,\dots,d_N}$.

The third approach, is analogue to the second, with the difference, that histories contain only observations and actions are discarded. This is possible since the action space at each step does not depend on the observation the environment is in and is inspired by the approximation of optimal Q-functions in MDPs by neural network, where often the input is the current state and the network outputs all action-values at once. A time-augmented history containing only observations has components $(t, o_t) \in \mathbb{R}^{d+1}$ at each step and its truncated signature of order $N \in \mathbb{N}$ is an element in $T(\mathbb{R}^{d+1})$. The implementation is then given by (5.16) with the only difference that the weight matrix's number of rows d_N is the

[4] Often $\alpha \in \mathcal{A}$ is possible, for example many environments permit a "do-nothing" action, as it is the case in the experiments in Chapter 6, where the agent simply observes the environment's transition and which constitutes a generic choice for α.

dimension of $T(\mathbb{R}^{d+1})$.

Finally, we comment on the handling of the signature computation at the beginning of an episode. At time $t = 0$ there is only one tuple available, depending on the approach it is given as either $\hat{h}_0^a = (0, o_0, a)$ for the first, ${}^{\alpha}\hat{h}_0 = (\alpha, 0, o_0)$ for the second, and $(0, o_0)$ for the third approach. Since analytically the signature of a continuous path at a single time point evaluates to zero, compare Definition 2.3 with $s = t$, one can simply set $S^N(\hat{h}_0^a) = \mathbf{1}$, with $\mathbf{1}$ the unit element in $T^N(\mathbb{R}^{d+2})$, and analogue for the other two initial tuples. In the first approach, this results in the approximate Q-value for each $a \in \mathcal{A}$ being given by the first component of θ_ℓ which implies that the algorithm is not able to differentiate between action at the beginning of episodes. For the other two approaches, the vector of approximate Q-values (5.16) at $t = 0$ is then given by the bias vector b. As such the algorithm can learn to differentiate actions at the start of an episode, this differentiation however, is static and can not depend on the information provided by the initial observation o_0. To be able to incorporate the information provided by o_0 at the start of an episode, we propose the addition of a basepoint before the initial tuple. In approach three, this basepoint would be given as tuple $(0, o_{-1})$ for some artificial observation $o_{-1} \in \mathbb{R}^d$, such that $t = 0$, there is already the stream $((0, o_{-1}), (0, o_0))$ available for the computation of the signature. A simple solution would be to set $o_{-1} = 0$, such that the Q-function approximation at $t = 0$ depends on the value of o_0 and the bias vector b. However, any other value is possible and the choice can be tailored to the environment, as in the experiment in Section 6.2. For the other two approaches a basepoint is provided analogously as $(0, o_{-1}, a_{-1})$ with some artificial action $a \in \mathbb{N}$ for the first approach, and$(\alpha, 0, o_{-1})$ for the second approach, respectively.

6 Numerical Experiments and Results

6.1 Github Repositories

The code for the experiments in this chapter can be found on GitHub: The first experiment in Section 6.2 is implemented in repository `SignatureQLearning-MountainCar`[1] and the second experiment in Section 6.3 in the repository `SignatureQLearning-Execution`[2]. Since the second experiment's implementation was more involved due to the integration of the market simulator ABIDES, detailed in Section 6.3.1, the experiments are provided as two distinct repositories. Both experiments are coded in Python 3.8. The main third-party packages used are `Signatory` [37] for the signature computations, the machine learning library `PyTorch` [59], the reinforcement learning API `Gymnasium` [74] and its predecessor OpenAI's `Gym` [10], and the market simulator ABIDES [12], which is described in more detail in Section 6.3.1. Python's version 3.8, is the latest version which is compatible with the last stable release of the package `Signatory` for Apple's Mac. The full list of package requirements and their versions can be found in the `requirements.txt` file in the respective repository. All training was performed on a Mac Book Pro with Intel(R) Core(TM) i7-6700HQ CPU @ 2.60GHz processor.

6.2 Mountain Car Experiment

In this section we test the Signature-Q-learning method given by Algorithm 2 in an environment modeled as partially observable Markov Decision Process. The environment is an adaptation of the Markovian reinforcement learning problem "Mountain Car", provided as part of the Python package `Gymnasium` [74]. The package is an open source reinforcement learning library that provides ready to use environments and a standard API for the communication with custom learning algorithm. Since in the adaptation the agent has only access to a partial observation, it has to use the history of observations to infer the hidden true state of the system and learn an approximate optimal Q-function.

6.2.1 The Environment

The "Mountain Car" problem consists of a car placed at the bottom of a 2-dimensional valley, with the starting position chosen uniformly over some area in the valley. The agent can accelerate the car in either direction and the goal is to reach the top of the hill located to the right hand side of the valley. An illustration of the setting is given in Figure 6.1. The agent's challenge is to learn a policy that drives the car up the left side of the valley

[1] https://github.com/phaelicks/SignatureQLearning-MountainCar
[2] https://github.com/phaelicks/SignatureQLearning

(potentially multiple times) first, in order to gain enough momentum while coming down from the left, to subsequently being able to reach the top right.

The dynamics in the original environment provided by the package `Gymnasium` follow a deterministic MDP. The state $s_t = (p_t, v_t)$ is given as the car's position along the x-axis p_t and its current velocity v_t, which takes values in the state space $\mathcal{S} = [-1.2, 0.6] \times [-0.07, 0.07]$. The initial position p_0 is sampled from $\mathcal{U}([-0.6, -0.4])$ while the initial velocity is set to $v_0 = 0$. The action space is $\mathcal{A} = \{0, 1, 2\}$ where actions correspond in increasing order to accelerating the car to the left, not accelerating and accelerating it to the right, respectively. Following the selection of action $a \in \mathcal{A}$, the environment transitions to the new state $s_{t+1} = (p_{t+1}, v_{t+1})$ according to the deterministic, Markovian dynamics,

$$v_{t+1} = v_t + (a - 1)f - \cos(3p_t)g, \tag{6.1}$$

$$p_{t+1} = p_t + v_{t+1}, \tag{6.2}$$

with acceleration force $f \equiv 0.001$ and gravity on slopes $g \equiv 0.0025$. The maximum episode length is $T = 200$ and an episode is solved successfully if $p_t \geq 0.5$ for some $t \leq 200$, which corresponds to the position of the flag in Figure 6.1.

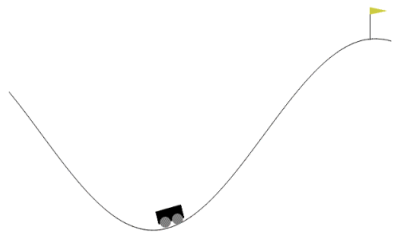


Figure 6.1: An illustration of Gymnasium's Mountain Car environment.[3] The goal is to strategically accelerate the car such that it reaches the flag on the mountain top to its right. Since its power is not sufficient to go up by just accelerating right, the agent has to learn a policy that first moves the car up the opposite hill and uses its momentum when going down again.

Observation and Action Space We modify the environment such that it becomes partially observable one, which justifies the need of using the environment's history for learning. Instead of having access to the full state s_t, we define an observation as the car's position $o_t = p_t$, taking values in the observation space $\mathcal{O} = [-1.2, 0.6]$, and the agent receives the time-augmented observation

$$\hat{o}_t = \left(1 - \frac{t}{T}, p_t\right) \in [0, 1] \times \mathcal{O},$$

where the time component is given as the normalized remaining episode time. The action space remains as in the original environment as $\mathcal{A} = \{0, 1, 2\}$ for the possibility to accelerate right, not accelerate and accelerate left. The environment still progresses according to the

dynamics in Equation (6.1), however since the agent only observes $\hat{o}_t$, the problem now a (deterministic) partially observable Markov decision process. In order to infer the hidden velocity and an (approximately) optimal policy, observation histories $\hat{h}_t = (\hat{o}_0, \dots, \hat{o}_t)$ may be used.

Rewards The objective is to reach the top right hill as quickly as possible. For the agent to learn a policy that does so, keeping an episode short must result in a higher reward. The original environment achieves this by assigning the reward $R_t(s_t, a_t) = -\mathbb{1}_{[-1.2,0.5)}(p_t)$ for any $t \in [T]$, which simply penalizes the agent for each position different to a goal position and serves as incentive to keep the length of an episode short. With Q-learning methods, however, it can be difficult to solve the problem with this specific reward structure. The agent has no knowledge about the existence of observations or trajectories thereof, such that the total reward per episode is greater than -200,[4] before a position $p_t \geq 0.5$ is reached for the first time. Until then, the randomly initialized Q-functions approximation, contains no information about histories with higher values. Since the exploration behaviour is based on the current approximation, it is extremely difficult to reach a goal position by accident. For this reason, we adapted the reward functions to incorporate some guidance for the agent, to

$$R_t(\hat{h}_t) = R_t(\hat{o}_t) := \mathbb{1}_{\{x<0.5\}}(p_t)\frac{p_t - 0.5}{T} + \mathbb{1}_{\{x\geq 0.5\}}(p_t)\, C\,(1 - \frac{t}{T})^2, \tag{6.3}$$

for $t \in [200]$ and with $C \geq 0$. Note, that the reward function depends on $\hat{h}_t$ only through the current time-augmented observation $\hat{o}_t$. The reward obtained is the sum of two components. The first is a running reward the agent receives as long as the goal has not been reached. It provides some guidance as to where the goal is located and at the same time serves as incentive to keep episode short, since it only takes negative values. At the same time, the agent must learn to accept negative rewards of higher absolute value when moving the car to the left, in order to increase the overall episode reward by managing to reach the goal earlier. It is normalized for stability. The second component is a terminal reward, which provides a positive signal if the goal is reached at time $t \leq T$, and which increases with smaller t. We include the terminal reward in the definition of the reward functions R_t since it depends on the variable time step at which the episode ends. In the experiments $C = 0.01$ was chosen.

As an alternative to setting the reward functions to (6.3), one could use the environment's original reward structure in conjunction with action repetition to increase to probability of the agent reaching the goal position for a first time. Instead of selecting an action at each step, fix $k \in \mathbb{N}$ (w.l.o.g. assume that $T \mod k = 0$) and select actions only at steps $0, k-1, 2k-1, \dots, T-k-1, T-1$. In between those times the action selected last is repeated, that is if $\tilde{a} \in \mathcal{A}$ is selected at time $t = nk - 1$ for some $n \in \{1, \dots, T/k\}$, we have $a_{t+i} = \tilde{a}$ for $i = 0, 1, \dots, k-1$.

[4] This corresponds to the undiscounted reward per episode if the goal has not been reached. Depending on the discount factor, the total reward the agent receives may be different.

6.2.2 Training Details

We perform training in three different set-ups. Set-up one is a single hyperparameter configuration, to emphasize that Signature-Q-Learning is able to successfully solve the problem. Set-up two studies the effect of different signature truncation orders, and set-up three the behaviour of the algorithm with different basepoint positions p_1.

Before providing details on the different set-ups in the paragraphs below, we describe the training configuration's parts that are mutual to all three. First, we consider only the time-augmented observation history. That is, the input to the approximate Q-function at time t is the stream $(\hat{o}_i)_{i=0}^t = (1 - i/T, p_i)_{i=0}^t$. The approximate Q-function is implemented as single linear layer, which outputs the Q-value for each one of the three actions simultaneously. Details on this implementation are given in Section 5.3. For the signature truncated at order $N \in \mathbb{N}$, this results in d_N trainable parameters for each action, with d_N the dimension of $T(\mathbb{R}^2)$, which consist of a weight matrix $W \in \mathbb{R}^{3\times(d_N-1)}$ and a bias vector $b \in \mathbb{R}^3$, cf. (5.16). The entries in W are initialized randomly from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ with $k = 6/(d_N^{-1} + 3)$, which corresponds to `PyTorch`'s uniform Xavier initialization [22], and the bias vector's components are set to $b_i = 0.1, i = 1, 2, 3$. Using histories containing only observations proved to be more stable in training, than the other two implementation approaches described in Section 5.3.

Further, the environment exhibits two possible terminations. The first one is due to reaching the maximum episode length $T = 200$, and the second due to reaching a goal position $p_t \geq 0.5$. Both are treated as environmental terminations and no bootstrapping is performed at either one, as proposed in [58]. The Q-target values at k-th update step are thus given as

$$y_t = \begin{cases} r_t, & \text{if } t = 200 \text{ or } p_t \geq 0.5, \\ r_t + \arg\max_{a\in\mathcal{A}} Q_t(\hat{h}_{t+1}^a \mid \ell_k), & \text{otherwise,} \end{cases}$$

where r_t is the sampled reward at time t according to the reward function (6.3). W.l.o.g. we assume there are no ties between actions such that the argmax indeed returns a single action. Otherwise, ties are broken up by sampling uniformly from the actions that are part of the tie.

Set-Up One: Fixed parameter configuration The first training set-up consists of a fixed hyperparameter configuration, which is detailed in Table 6.1. In total, 10 training runs are performed in set-up one. The hyperparameter configuration was found through an informal search. The discount factor of $\gamma = 0.99$ enabled stable learning behaviour in reducing the Q-function's horizon to roughly $1/(1 - \gamma) = 100$ steps. Regarding the handling of the signature computation at $t = 0$, see Section 5.3, the basepoint $\hat{o}_{-1} = (1, p_{-1}) = (1, -0.5)$ was used. This amounts to an artificial position $p_{-1} = -0.5$ before $t = 0$, exactly in the middle of the interval $[-0.6, -0.4]$ in which the car is placed randomly at the beginning of each episode. As detailed in Section 5.3, this choice should enable the algorithm to differentiate between initial observations o_0, or more precisely initial positions p_0, beyond the values provided by the bias vector. For the initial position $p_0 = -0.5$, the initial observation $\hat{o}_0$ is equal to the basepoint $\hat{o}_{-1}$ and the truncated signature at $t = 0$ evaluates to $\mathbf{1} \in T^{(N)}(\mathbb{R}^{d+1})$. In this case, the selection of the first action a_0 solely depends on the values in the bias

vector $b \in \mathbb{R}^3$. For any other $p_0 \in [-0.6, -0.4] \setminus \{-0.5\}$, the truncated signature is distinct from $\mathbf{1}$ and as such contains information about p_0. Other values for p_{-1} are considered in training set-up three, Section 6.2.2

The exploration policy used is ε-greedy. The learning rates and the exploration rates are decreased over training, with the decay schemes detailed in Table 6.2. Since the environment exhibits a clear goal for each episode, both parameter values are decreased only after successful episodes to facilitate learning and exploration when the number of successful episodes is still low. During the episode both values are held fixed. Using the example of the exploration rate, this means that $\varepsilon_n := \varepsilon_{n,0} = \varepsilon_{n,t}$ for all $t \in [T]$, and this value is only decreased to the next smaller value ε_{n+1}, if in the current episode the goal was reached, else it is left the same for the subsequent episode.

Table 6.1: The hyperparameter values used in training set-up one for the Mountain Car experiment. The parameters correspond to number of episodes, maximum episode length, discount factor, loss function, gradient descent optimizer, signature truncation order, and basepoint, in the order of columns. The smooth-L_1 loss function is provided by (5.14), the gradient descent optimizer Adam is introduced in [38].

Parameter	M_{train}	T	γ	$\mathcal{L}$	optimizer	N	$\hat{o}_{-1}$
Value	4000	200	0.99	smooth-L_1	Adam	5	$(1, -0.5)$

Table 6.2: Details on the decay schemes of the learning rates and ε-greedy exploration rate in the Mountain Car experiment.

Parameter	**Start value**	**End value**	**Decay epochs**	**Decay mode**
Learning rates $\boldsymbol{\alpha}_n$	0.01	10^{-5}	3000 successes	Linear
Exploration rates $\boldsymbol{\varepsilon}_n$	0.3	0.05	2000 successes	Linear

Set-Up Two: Truncation Orders The second training set-up investigates different signature truncation orders $N \in \mathbb{N}$, a hyperparameter distinct to Signature-Q-Learning. For each truncation order four training runs are performed, respectively. Apart from the truncation orders, other parameter values are the same as in set-up one and detailed in Table 6.1. The ε-greedy exploration rates ε_n are again decayed over successful episodes with the decay schemes detailed in Table 6.2. Distinct to set-up one, in set-up two the learning rate is held fixed at $\alpha = \alpha_n = 0.005$ for all episodes $m \in [M]$ in each training run. Since the observation history is only two dimensional, it is possible to truncate at a relatively high order without running into computational restrictions due to the exponential scaling of the truncated signature's dimension d_N. We compare truncation orders

$$N \in \{2, 3, 4, 5, 6, 7, 8\}.$$

We expect that the algorithm performs better with a higher truncation order in terms of higher rewards and number of successfully solved episodes.

Set-Up Three: Basepoints The third set-up investigates different basepoint configurations for handling the signature computation at $t = 0$, as detailed in Section 5.3. Specifically, we perform training runs with three different basepoints and without basepoint. For each configuration four training runs are performed, respectively. Each basepoint is given as a tuple $\hat{o}_{-1} := (1, p_{-1})$, with $p_{-1} \in \mathbb{R}$ an artificial position prior to $t = 0$. This tuple is added to the observation history, such that at $t = 0$, the agent receives the stream $(\hat{o}_{-1}, \hat{o}_0)$ to compute the signature. The value for p_{-1} is based on the interval $[-0.6, -0.4]$ from which p_0 is sampled uniformly in each episode, and we compare the values

$$p_{-1} \in \{-0.65, -0.5, -0.35\}.$$

In the case of no basepoint, at $t = 0$, the truncated signature is set to $\mathbf{1} \in T^{(N)}(\mathbb{R}^{d+1})$. Thus, its components apart form the first entry 1 are zero and only take non-zero values at $t = 1$ when there are two observations available. This results in the selection of action a_0 based solely on the bias vector $b \in \mathbb{R}$, since no information about the initial observation $\hat{o}$ is available to the agent through the signature.

Apart from the basepoint configuration, the remaining hyperparameters are again provided in Table 6.1. The ε-greedy exploration rates ε_n are again decayed over successful episodes with the decay schemes detailed in Table 6.2 and the learning rate held fixed at $\alpha = 0.005$ for all basepoint configurations and runs.

6.2.3 Testing Details

For each training run, we asses the learned approximate Q-function by performing a test run consisting of $M_{\text{test}} = 500$ episodes. Let $Q(\cdot \mid \ell_K)$ be the approximate Q-function from a training run, with ℓ_K the final linear functional of the signature approximation after a total of $K = TM_{\text{train}}$ update steps during training. In the following test run, actions are selected greedily with respect to $Q(\cdot \mid \ell_K)$. That is, the policy followed in testing at $t \in [T-1]$ is given as

$$\pi_t^{\text{test}}(h_t) := \arg\max_{a \in \mathcal{A}} Q(\hat{h}_t^a \mid \ell_K), \tag{6.4}$$

for any $h_t \in \mathcal{H}_t$, where we assume that there are no ties between actions, such that the action that maximizes the right hand side of (6.4) is unique[5]

For each test run, we report the test statistics in Table 6.3. We provide a brief description of their calculation. For test episode $n \in \{1, \dots, M_{\text{test}}\}$, let $r^{(n)}$ be the sum of undiscounted rewards obtained at each step in the episode. The mean cumulative, undiscounted episode reward $\bar{r}$ and the episode reward standard deviation σ_r are calculated as the sample mean and standard deviation of the sample $\{r^{(n)}\}_{n \in \{1,\dots,M_{\text{test}}\}}$. Analogously, let $m^{(n)} := \inf\{t \in [T] \mid p_t^{(n)} \geq 0.5\}$ be the length of episode n, with $p_t^{(n)}$ the car's position at step t in episode n, and the convention $\inf \varnothing := T$. The mean episode length $\bar{m}$ and the episode length standard

[5] If there happens to be a tie between action values in the experiment, an action may be selected by sampling uniformly from the set of actions involved in the tie.

deviation σ_m are calculated as the sample mean and sample standard deviation of the sample $\{m^{(n)}\}_{n\in\{1,\dots,M_{\text{test}}\}}$. Further, we have $\min_m := \min\{m^{(n)} : n \in \{1,\dots,M_{\text{test}}\}\}$ and $\max_m := \max\{m^{(n)} : n \in \{1,\dots,M_{\text{test}}\}\}$. For the mean first observation value $\bar{v}_0^*$, recall that in each episode the car's starting position p_0, which is the agent's initial observation o_0, is sampled from $\mathcal{U}[-0.6,-0.4]$. Let $o^{(n)} = p_0^{(n)} \in [-0.6,-0.4]$ be the initial position in test episode $n \in \{1,\dots,M_{\text{test}}\}$ of the current test run and $\hat{o}_0^{(n)} = (1, p_0^{(n)})$. We define $v_0^{*,n}$ as the value of the first observation $\hat{o}_0^{(n)}$ as provided by the final approximate Q-function $Q(\cdot \,|\, \ell_K)$, and $\bar{v}_0^*$ as the average over test episodes,

$$v_0^{*,n} := \max_{a\in\mathcal{A}} Q(\hat{o}_0^{(n)}, a \,|\, \ell_K), \qquad \text{and} \qquad \bar{v}_0^* := \frac{1}{M_{\text{test}}} \sum_{n=1}^{M_{\text{test}}} v_0^{*,n}. \tag{6.5}$$

Finally, the mean cumulative, discounted episode reward $\bar{r}_\gamma$, with discount factor $\gamma = 0.99$, is calculated as the sample mean over the sample $\{r_\gamma^{(n)}\}_{n\in\{1,\dots,M_{\text{test}}\}}$, where for each episode n, the cumulative, discounted episode reward $r_\gamma^{(n)}$ is given as as the sum

$$r_\gamma^{(n)} = \sum_{t=0}^{m^{(n)}} \gamma^t r_t^{(n)}, \tag{6.6}$$

with $r_t^{(n)}$ the reward obtained at step t in test episode n.

If the algorithm converged to the true optimal Q-functions, then $Q(\hat{o}_0, a \,|\, \ell_K)$ is an estimate of the true Q-value $Q_0^*(o_0, a)$ for any $(o_0, a) \in \mathcal{O} \times \mathcal{A}$. This implies that due to Equation (5.2), the first observation value $v_0^{*,n}$ is an estimate of the true optimal value $V_0^*(o_0^{(n)})$, and by its definition as sample mean in (6.5), $\bar{v}_0^*$ is an estimate of the expected optimal value at $t = 0$,

$$\bar{v}_0^* \approx \mathbb{E}[V_0^*(O_0)], \qquad \text{with } O_0 \sim \mathcal{U}[-0.6,-0.4].$$

In absence of an optimal policy we can use for comparison, however, we do not know whether the algorithm has converged only by looking at the value of $\bar{v}_0^*$. Still, it may offer an indication about convergence at least to *some* solution, if we compare it to $\bar{r}_\gamma$. Recall Definition 4.11 for the value functions $(V_t^\pi)_{t\in[T]}$ corresponding to a given policy $\pi \in \Pi^{\text{HR}}$. In the test run, actions are selected according to π^{test}, thus, $r_\gamma^{(n)}$ is a sample of $V_0^{\pi^{\text{test}}}(o_0^{(n)})$, and by its definition as sample mean, $\bar{r}_\gamma$ is an estimate of the expected initial value of policy π^{test},

$$\bar{r}_\gamma \approx \mathbb{E}[V_0^{\pi^{\text{test}}}(O_0)] \qquad \text{with } O_0 \sim \mathcal{U}[-0.6,-0.4].$$

If we obtain $\bar{r}_\gamma \approx \bar{v}_0^*$, we may conclude, that in this test run the algorithm has converged to some Q-functions. More specifically, the approximate Q-function correctly returns the action-values of the policy π^{test}.

Table 6.3: Description of the reported test statistics in the Mountain Car Experiment. Statistics are calculated for each performed test run and reported in Section 6.2.4.

$\overline{r}$	Mean undiscounted reward per episode
σ_r	Standard deviation of undiscounted reward per episode
$\overline{r}_\gamma$	Mean discounted reward per episode
succ.	Percentage of successful episodes in test run
$\overline{m}$	Mean episode length until termination
σ_m	Standard deviation of episode length until termination
$\min_m$	Minimum episode length until termination
$\max_m$	Maximum episode length until termination
$\overline{v}_0^*$	Mean first observation value (6.5)

6.2.4 Results

The following paragraphs present the results for the three training set-ups described in Section 6.2.2. Results are compared based on the statistics in Table 6.3 calculated from performed test runs.

Set-Up One: Fixed parameter configuration Table 6.4 provides a comparison of the different test runs performed for each training run in set-up with a fixed hyperparameter configuration. We note that, Signature-Q-Learning algorithm was able to consistently solve the problem in all runs, showing that the approach proposed in this work indeed works for a history-dependent problem given as the partially observable MDP at hand. In all 10 runs, the algorithm was able to learn a Q-function approximation that reaches the goal position in 100% of test episodes. The difference between the lowest and highest mean undiscounted episode reward reward $\overline{r}$ is 3.86% of the highest absolute value, indicating that for this specific hyperparameter configuration the algorithm is not sensitive to parameter initialization and the seed used to simulate the environment. This is also apparent from the mean *discounted* episode rewards $\overline{r}_\gamma$, which is the measure the algorithm optimizes for. Its sampled values are practically the same, agreeing up to a prevision of 10^2. At the same time the standard deviation of rewards in each run is relatively high, with all values in the interval $[-0.145, -0.165]$. This variability can be explained by large differences in episodes lengths. While the mean episode length $\overline{m}$ is very similar for all runs, the difference between the minimum $\min_m$ and maximum $\max_m$ episode lengths is more than 100 steps in all runs but one. This has a direct consequence on rewards, due to the reward structure (6.3), that is revealed by Figure 6.2. The figure shows episode lengths and obtained undiscounted episode rewards as a function of the starting position p_0 for test run 8. The plots show a clear split around $p_0 \approx -0.45$ with vastly different behaviour for smaller and larger starting positions for both statistics, which results in a distribution of episode rewards that is heavily clustered around two values. Although this causes high variance in both statistics, it is also an indication that the algorithm learned to differentiate starting positions and follows different action sequences depending on them, to increase rewards. This is in particular visible when comparing the result in set-up one, to the ones in the following set-up two

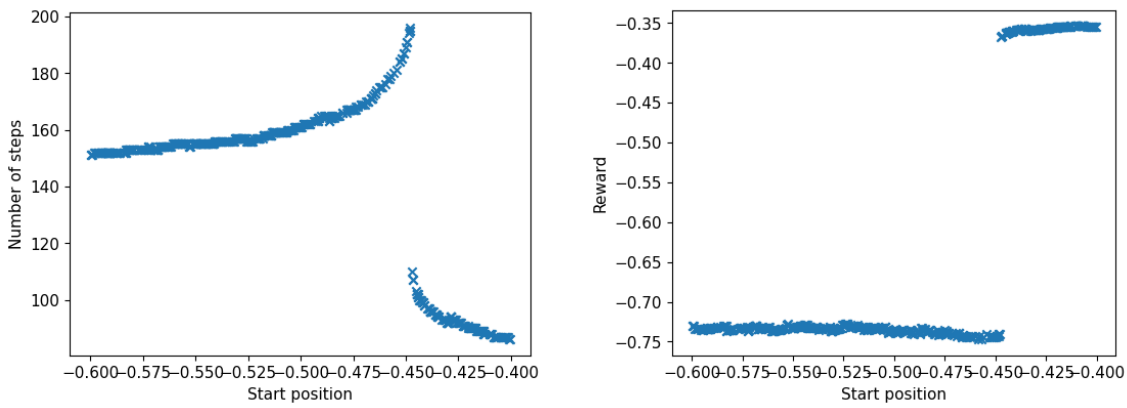
with the same truncation order $N = 5$, where a more homogeneous policy was learned that however delivers smaller overall episode rewards.

Table 6.4: The table displays test statistics for the Mountain Car experiment **set-up one** in Section 6.2.2, when following the obtained approximate Q-function from each training run for 500 test episodes. Bold values highlight the best result over runs: the highest reward, the lowest standard deviation, or the shortest episode length. We refrained from highlighting the column containing success rates, since for each run it is 100%, and also the column with initial optimal values, because the obtained values are not meaningful due to the overestimation of Q-values. $\bar{v}_0^*$ is calculated as in Equation (6.5) with the basepoint $\hat{o}_{-1} = (1, p_{-1}) = (1, -0.5)$ for the computation of the signature at time $t = 0$.

Run	$\bar{r}$	$\bar{r}_\gamma$	σ_r	succ.	$\bar{m}$	σ_m	$\min_m$	$\max_m$	v_0^*
1	−0.6475	−0.3538	0.1606	100%	144.79	29.98	**86**	199	267.67
2	−0.6638	−0.3590	**0.1456**	100%	147.44	27.10	87	197	263.70
3	−0.6546	−0.3563	0.1537	100%	145.72	28.45	**86**	**185**	263.12
4	−0.6591	−0.3570	0.1474	100%	147.04	28.00	87	195	268.42
5	−0.6514	−0.3558	0.1522	100%	144.61	27.13	87	193	269.61
6	−0.6580	−0.3562	0.1523	100%	147.29	28.98	**86**	199	272.03
7	−0.6603	−0.3577	0.1465	100%	146.88	**27.03**	87	196	271.70
8	−0.6431	−0.3523	0.1627	100%	144.28	30.23	**86**	199	267.23
9	**−0.6391**	**−0.352**	0.1644	100%	**142.54**	29.99	**86**	196	275.27
10	−0.6476	−0.3534	0.1635	100%	145.22	31.76	**86**	199	260.34

Figure 6.3 shows training performance in set-up one, averaged over the 10 performed training runs. It again highlights the variability in episode rewards (a), and in steps until termination (d), due to the heterogeneous actions sequences followed based on starting positions. Overall however, all runs exhibit a similar training behaviour and convergence to the same Q-functions approximation. The algorithms success of learning an approximation that solves the problem is apparent from Figure 6.3d, which visualized the mean end position for each training episode. It shows that after around 2000 episodes, all runs managed to reach a goal position of $p_t \geq 0.5$ in almost all episodes, and towards the end of the training runs in 100% of episodes.

One issue Signature-Q-Learning suffers from in this particular problem, is the overestimation of Q-values. Although the algorithm was able to consistently reach the goal position, the approximate Q-values for pairs (h_t, a) did not converge to their true values provided by the true optimal Q-functions $Q_t^*, t \in [T]$. This is apparent from the high values obtained for the mean first observation value $\bar{v}_0^*$. Recall, its definition from Equation (6.5), and that it should approximately match the cumulative, discounted reward $\bar{r}_\gamma$ for any run in which the approximate Q-function has converged. Clearly, for any test run we have $\bar{v}_0^* \not\approx \bar{r}_\gamma$ in Table 6.3. Additionally, due to the reward structure (6.3), the true expected first observation value $\mathbb{E}[V_0^*(O_0)]$ must take its value in the interval $[-0.735, 0]$. As such any sufficiently accurate estimate must fall into the same interval, however, clearly $\bar{v}_0^* \notin [-0.735, 0]$ for all test runs. The overestimation further results in high episode losses during training,



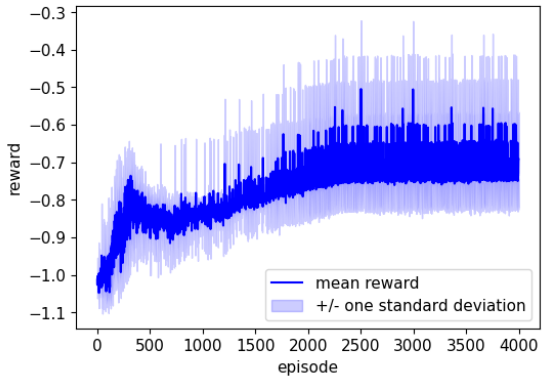
(a) Start position vs. steps till termination.

(b) Start position vs. loss per episode.

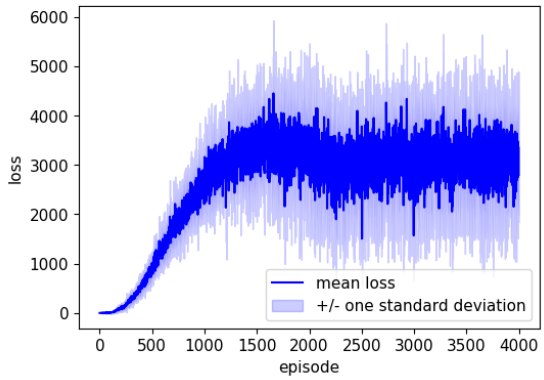
Figure 6.2: The figures displays scatter plots for test run 9 from set-up one in the Mountain Car experiment. It shows that the learned Q-function approximation differentiates between the car's starting positions $p_0 \in [-0.6, -0.4]$ and follows different action sequences based on them. Plot (a) displays the number of steps until termination as a function of the starting position p_0 for each episode from test run 9. Plot (b) displays the total undiscounted reward obtained as a function of the starting position p_0 for each episode from test run 9.

visible in Figure 6.3b. While the loss stabilized around the value 3000, its variability over episodes remains high, which may be due to the combination of overestimated Q-values, large differences in episodes lengths, and the fact that one set of parameters is used to approximation optimal Q-functions at all time-steps. As an heuristic example, consider an episode in which at $t = 86$ a goal position $p_{86} \geq 0.5$ is reached, which corresponds to the minimum episode length for most runs in Table 6.4. For simplicity, assume the loss function $\mathcal{L}$ in Algorithm 2 is the absolute loss $\mathcal{L}(x, y) = |x - y|$ for any $x, y \in \mathbb{R}$. The following reasoning, however, also applies to the smooth-L_1 loss (5.14) used in the experiment. Recall the definition of the Q-target value y_t from Equation (5.11) used to update the Q-functions approximation. As described in the training details Section 6.2.2 no bootstrapping is performed at terminations and the target value at $t - 1 = 85$ is given as $y_{85} = R_{85}(\hat{o}_{85}) = 0.016245$, for the reward function (6.3). In any other episode that does not terminate at $t = 86$, the target value additionally includes the maximum action value $\max_{a \in \mathcal{A}} Q(\hat{h}_{86}^{a} \,|\, \ell)$ at the next step $t = 86$, which presents an estimate of the maximal attainable cumulative discounted reward over potentially 100 or more remaining steps. An overestimation of this value results in vastly different errors for step $t = 85$, which in both cases is calculated as $\left|y_{85} - Q(\hat{h}_{85}^{a_{85}} \,|\, \ell)\right|$, to update the set same set of parameters ℓ. If during training short episodes alternate with significantly longer episodes, which is the case in this experiment as apparent from Figure 6.2a, the overestimation issue results in large errors at a number of steps.

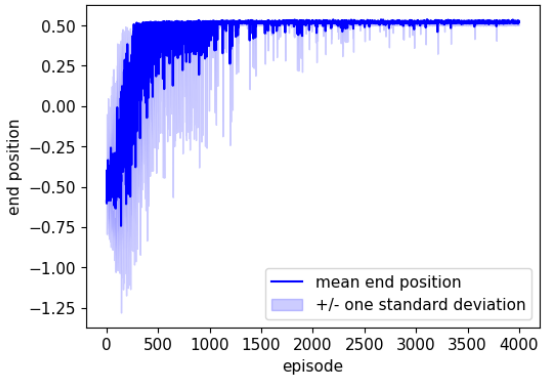
Overestimation of Q-values is a known issue for Q-Learning in MDPs [26, 77], especially in combination with function approximation as in Deep Q-Networks [54]. In [77], the authors argue that estimation errors, that emerge from the use of function approximation



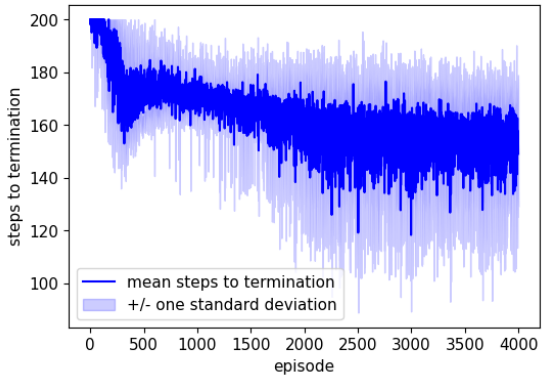
(a) Average reward per episode.



(b) Average loss per episode.



(c) Average end position per episode.



(d) Average steps till termination per episode.

Figure 6.3: Averaged training results of the Signature-Q-Learning algorithm in the Mountain Car experiment set-up one Section 6.2.2. 10 training runs were performed in total with the hyperparameter set-up detailed in Table 6.1. The sub-figures show the respective statistic as the mean over training runs for each episode. (a) shows the average cumulative, undiscounted reward obtained per episode. (b) the average total loss obtained per episode. (c) shows the average of the car's end position at the episode termination. (d) shows the average number of steps until the episode terminated, either because the car reached a goal position or the maximum number of steps $T = 200$.

or environmental noise, introduce an upwards bias in Q-function approximations, and they provide a lower bound for the error (Theorem 1 [77]). Their statement readily transfers to the history-based setting considered in the present work. Assume $|\mathcal{A}| > 1$, and consider a time $t \in [T]$ and history $h_t \in \mathcal{H}_t$, for which the true optimal Q-values are equal at $Q_t^*(h_t^a) = V_t^*(h_t)$ for all $a \in \mathcal{A}$, with V_t^* the true optimal value function (4.20) at t. Further consider some Q-function approximation provided by the linear functional ℓ, such that

$$\sum_{a\in\mathcal{A}} Q(\hat{h}_t^a \,|\, \ell) - V_t^*(h_t) = 0, \quad \text{and} \quad |\mathcal{A}|^{-1} \sum_{a\in\mathcal{A}} \big(Q(\hat{h}_t^a \,|\, \ell) - V_t^*(h_t)\big)^2 = C$$

for some $C > 0$. Then, we have the lower bound

$$\max_{a\in\mathcal{A}} Q(\hat{h}_t^a \,|\, \ell) \geq V_t^*(h_t) + \sqrt{\frac{C}{|\mathcal{A}| - 1}}.$$

While this lower bound decreases with the number of actions $|\mathcal{A}|$, the authors in [77] show that the actual overestimation typically increases with the size of $\mathcal{A}$. The usage of the maximum action value $\max_{a\in\mathcal{A}} Q(h_t, a \,|\, \ell)$ to approximate *expected* maximum action value from the optimal Q-iteration (5.4) in the bootstrapping procedure (5.12), further reinforces the overestimation and propagates the error through estimates, see also [26]. An overestimation however, does not have to result in a poor policy, as argued in [77], and which is also apparent in the experiment at hand. If the overestimation occurs uniformly over state-action pairs, in our case history-action pairs, the relative action preferences, and in this way maximizing actions, are maintained.

To overcome the issue of overestimation of Q-values in MDPs, [26] proposed the *Double Q-Learning* algorithm in a tabular setting, which was later extended to the approximation of Q-functions with deep neural networks, resulting in Double Deep-Q-Networks [77]. The principle behind Double Q-learning is to disentangle the agent's action selection from the estimation of the selected action's value through the target value y_t. Two distinct Q-function approximation either in tabular or parametrized form are used, one for each of the task, and alternately updated with the encountered environmental transitions. In MDPs, this proofed to avoid the overestimation of Q-values and improve performance, especially in conjunction with function approximation [77]. The same principle could be used here, to investigate whether it mitigate the overestimation of Q-values, by using one linear functional with parameters $\ell_k^{(1)}$ to select the action at each step, and another functional with parameters $\ell_k^{(2)}$ to create the target value y_t.

Set-Up Two: Truncation Orders Table 6.5 shows the test results for training set-up two with different truncation orders. As expected, the performance increased with higher truncation order N, however only up to a certain order. For $N = 2$, Signature-Q-Learning did not learn a Q-function approximation that solves the problem consistently in any run and obtained rewards are generally lower than with higher truncation orders. The results for $N = 3$ are similarly poor, with the exception of run 4, which performed best in a number of criteria, indicated by values highlighted in bold. It obtained the highest mean reward over truncation orders not only in run 4 but over all runs, as well as the overall shortest

mean episode length, corresponding shortest standard deviation and smallest maximum episode length. However, the outperformance compared to successful runs of higher order is very narrow. Further, considering that only one out of four runs was able to achieve 100% success rate, indicates that the order $N = 3$ is still too low for consistent successful training.

For truncation orders $N \geq 4$ Signature-Q-Learning achieved a 100% success rate in all test runs. While performance still increases from truncation order $N = 4$, with $N = 5$ exhibiting slightly higher mean reward and shorter mean episode length, higher orders yielded no further improvement. For $N \geq 5$, no truncation order emerges as clearly best performing, with all reported statistics taking very similar values regardless the value of N. The only exception to this is the mean first observation value v_0^*, which increases with the truncation order, indicating a growing overestimation of Q-values with larger $N \in \mathbb{N}$. This effect may caused by the larger number of parameters with higher truncation order together with the fixed learning rate $\alpha = 0.005$. Overall, the results show, that in the Mountain Car environment, a low truncation order of $N = 5$, which results is sufficient. Note, that for $N = 5$, the Q-function approximation, implemented as described in the training details in Section 6.2.2, contains in total $|\mathcal{A}| \cdot d_N = 189$ trainable parameters.

In terms of the policy derived from the approximate Q-function, all successful truncation orders $N \geq 5$ learned the same solution. This is apparent from the remarkably similar values for mean episode length and corresponding standard deviation as well as minimum and maximum episode length. It is evident, that for this specific hyperparameter set-up and basepoint configuration, Algorithm 2 is robust in Signature truncation order $N \in \mathbb{N}$.

For truncation order $N = 5$ in set-up two, the only difference to set-up one in the training configuration is the the use of a fixed rate $\alpha = 0.005$ in set-up two. This resulted in a different Q-function approximation and thus, also in a different policy, which is apparent from comparing the statistics reported for $N = 5$ in Table 6.5 for set-up two, to the ones in Table 6.4 for set-up one. While the success rate for all involved runs is 100%, there is a clear difference in obtained reward and episode characteristics. Notable, the policy followed in set-up two, results in a much smaller range of episode lengths, with the difference between minimum and maximum length roughly 40 steps, compared to over 100 steps in set-up one. This more homogeneous policy w.r.t to episode lengths however, comes at the cost of a lower reward.

Set-Up Three: basepoints Table 6.6 reports the test results for training set-up three. It shows that the different basepoint configurations have a large effect on Signature-Q-Learning performance and the policy derived from the learned Q-function approximations. In contrast to the truncation orders in set-up two, there usually is a clear basepoint configuration which performed best w.r.t to a certain test statistic.

In terms of obtained reward, a basepoint with artificial position $p_{-1} = -0.35$ performed best in all four runs by a significant margin. On average, the mean reward obtained is 8.75% higher than for the artificial position $p_{-1} = -0.5$, which achieved the second highest reward in all but one runs. Notably, the reward with $p_{-1} = -0.35$ is also larger than in set-up one, with the tuned hyperparameter setting, showing the effect the basepoint configuration has on performance. Further, this configuration also exhibits the shortest mean episode

Table 6.5: The table displays test statistics for the Mountain Car experiment **set-up two** in Section 6.2.4, when following the obtained approximate Q-function from each training run for 500 test episodes. Bold values highlight the best result over truncation orders N for each run: highest reward, lowest standard deviation, highest successes rate or shortest episode length. Note, that truncation order $N = 3$ resulted in the lowest standard deviation σ_r for Run 1, however, this configuration is excluded from the ranking since the goal position was not reached in any episode. Further, we refrained from highlighting the column containing mean first observation values $\bar{v}_0^*$, due to the overestimation of those values, and from reporting the cumulative, discounted reward $\bar{r}_\gamma$, since its comparison with $\bar{v}_0^*$ would be superfluous. $\bar{v}_0^*$ is calculated as in Equation (6.5) with the basepoint $\hat{o}_{-1} = (1, p_{-1}) = (1, -0.5)$ for the computation of the signature at time $t = 0$.

N	Run	$\bar{r}$	σ_r	succ.	$\bar{m}$	σ_m	$\min_m$	$\max_m$	$\bar{v}_0^*$
2	1	−1.0515	0.0677	8.0%	197.41	**8.86**	164	200	12.82
	2	−0.6981	0.1213	91.6%	151.03	24.79	**86**	200	10.00
	3	−0.7930	0.0480	71.6%	173.17	20.33	148	200	7.01
	4	−0.9572	0.0736	74.8%	176.74	14.91	163	200	8.51
3	1	−0.9472	0.0197	0.0%	200.00	0.00	200	200	1.23
	2	−0.7457	0.1239	82.4%	152.62	28.35	132	200	2.96
	3	−0.8994	0.0925	43.8%	183.03	19.28	157	200	0.49
	4	**−0.6801**	0.0239	**100.0%**	**145.29**	**8.63**	147.24	**164**	139.97
4	1	−0.6948	0.0224	**100.0%**	148.98	9.35	135	171	251.46
	2	−0.6926	0.0214	**100.0%**	148.40	**9.01**	135	170	248.97
	3	−0.6918	0.0225	**100.0%**	148.24	9.39	133	170	255.19
	4	−0.6925	**0.0221**	**100.0%**	148.24	9.26	135	170	240.24
5	1	−0.6907	0.0239	**100.0%**	147.64	9.89	130	171	307.49
	2	−0.6894	0.0217	**100.0%**	147.21	9.22	133	170	283.08
	3	**−0.6841**	0.0244	**100.0%**	**145.46**	**9.21**	**130**	170	302.91
	4	−0.6896	**0.0221**	**100.0%**	147.74	9.60	133	168	285.18
6	1	−0.6887	0.0237	**100.0%**	**146.98**	9.66	**129**	170	317.27
	2	**−0.6872**	**0.0205**	**100.0%**	**146.90**	**9.10**	132	**166**	296.99
	3	−0.6884	0.0226	**100.0%**	147.62	9.98	131	**168**	291.95
	4	−0.6899	0.0229	**100.0%**	147.43	9.63	131	170	322.98
7	1	−0.6884	0.0230	**100.0%**	147.02	9.78	131	169	329.42
	2	−0.6895	0.0231	**100.0%**	147.67	9.86	131	168	312.88
	3	−0.6878	**0.0211**	**100.0%**	147.02	9.25	132	**168**	313.78
	4	−0.6875	0.0233	**100.0%**	146.70	10.12	**129**	169	319.66
8	1	**−0.6879**	**0.0210**	**100.0%**	147.42	9.63	130	**167**	310.34
	2	−0.6890	0.0225	**100.0%**	147.32	9.34	130	169	323.90
	3	−0.6891	0.0219	**100.0%**	147.55	9.58	131	**168**	313.16
	4	−0.6847	0.0242	**100.0%**	145.72	9.60	130	169	321.06

length in all but one run, and the shortest minimum episode length, with the algorithm reaching a goal position in 84 steps in the fastest case. At the same time however, the learned Q-function estimates do not solve the problem consistently, with success rates up to two percentage points lower than 100%. The failure to solve all test episodes, yields the highest variability in obtained rewards and episode lengths of any configuration, visible from the reported corresponding standard deviations. This indicates that the policy followed, strongly differentiates between starting positions, managing to reach the goal very quickly for some, and not at all for others. This is clearly visible in Figure 6.4c, where there is a clear cut in episode length as a function of the starting position for the episodes in test run 1 at around $p_0 \approx -0.4875$. Interestingly, this cut is closer to the middle of the starting interval $[-0.6, -0.4]$, than the one in Figure 6.2a from set-up one, although there a basepoint position of -0.5 was used.

The basepoint position p_{-1} appeared as the least performing with the reported statistics over the performed runs showing no consistent picture. While two runs managed to achieve a 100% success rate, the corresponding rewards are lower than with positions $p_{-1} \in \{-0.35, -0.5\}$. Further, in the successful runs, this configuration did not differentiate between starting positions, but followed a policy that resulted in a linear decrease in the number of steps until termination, with larger $p_0 \in [-0.6, -0.4]$, visible in Figure 6.4a. This linear relationship is reversed for rewards, with a linear decrease in obtained rewards the larger p_0.

For basepoint position $p_{-1} = -0.5$ and the configuration with no basepoint, denoted as *None* in Table 6.6 the algorithm solved the problem consistently, achieving 100% success rate in all four runs, respectively. Setting $p_{-1} = -0.5$ yields the same training set-up as in set-up two and the corresponding values in Table 6.6 are consistent with the ones reported in Table 6.5 for the previous set-up. Compared to the other basepoint configurations, it the obtained rewards are the second highest in all but run 3, and it delivered a success rate of 100% in all four runs. The learned policy is qualitatively the same as with the previous basepoint position, apparent from the similarity of Figure 6.4b to the one for $p_{-1} = -0.65$.

Using no basepoint for the computation of the signature at $t = 0$, yielded a perceptibly different policy to the other configurations. While this configuration yielded the lowest rewards of all basepoint configurations, it was also the most consistent, with the the value of $\bar{r}$ falling in the interval $[-0.7, -0.71]$ in each run, and an absolute difference of 0.006 between the largest and smallest mean cumulative reward. Furthermore, it yielded the lowest standard deviation of mean rewards, being below 0.0155 in each run. The learned policies follow the same action sequence, regardless of the starting position p_0. Achieving a 100% success rate in all runs, the largest difference between minimum and maximum episode length is 7 steps, resulting in a standard deviation of less than 1.3 steps in each run. This effect is consistent expectations, due to the translation invariance of the signature if histories without a fixed basepoint are used as input. The homogeneous distribution of episode lengths is also visible in Figure 6.4d, which shows distinctly different picture than for the other configurations. Using no basepoint further resulted in the lowest maximum episode length of all configurations, where in the longest episode the car reached a goal position at $t = 153$.

In summary, the results from set-up three show that the basepoint configuration can have a large effect on the algorithms performance and learned Q-function approximation,

with each configuration resulting in different policies. Except for p_{-1}, the results within a configuration are robust, converging to the same Q-function approximation in all four runs. Further, the findings are consistent with the theory w.r.t incorporation information about the initial observation. For a fixed basepoint, the Q-function approximation contains this information and is able to differentiate between starting positions, resulting in varying action sequences depending on p_0. Without a basepoint, this information is lost and the translation invariance of the signature maintained in this case, yields a policy that does not differentiate between start, but follows the same action sequence regardless of the value p_0. Finally, the results in Table 6.6 indicate some trade-off between higher reward and a consistent success rate. The high rewards with $p_{-1} = -0.35$ come at the cost of high variance and a below 100% success rate, while for the homogeneous and low variance policy obtained with no basepoint, one has to accept the lowest rewards of all tested configuration, and with the performance of basepoint position $p_{-1} = -0.5$ falling somewhere in the middle of the other two. Depending on the specific criteria one wants to optimize in a problem, a certain basepoint configuration can be selected.

6.3 Simple Execution Experiment

In the second experiment we test the Signature-Q-Learning algorithm in an environment, in which the agent has to solve a simple execution problem, by trading in a single stock, simulated financial market. The environment provides an (approximately) optimal policy as benchmark, which albeit not analytically available, offer the possibility to assess the algorithm's learning performance. Before detailing the environment and its components, we introduce the agent-based market simulator, which is used to generate the underlying stock market to trade in.

6.3.1 The Market Simulator ABIDES

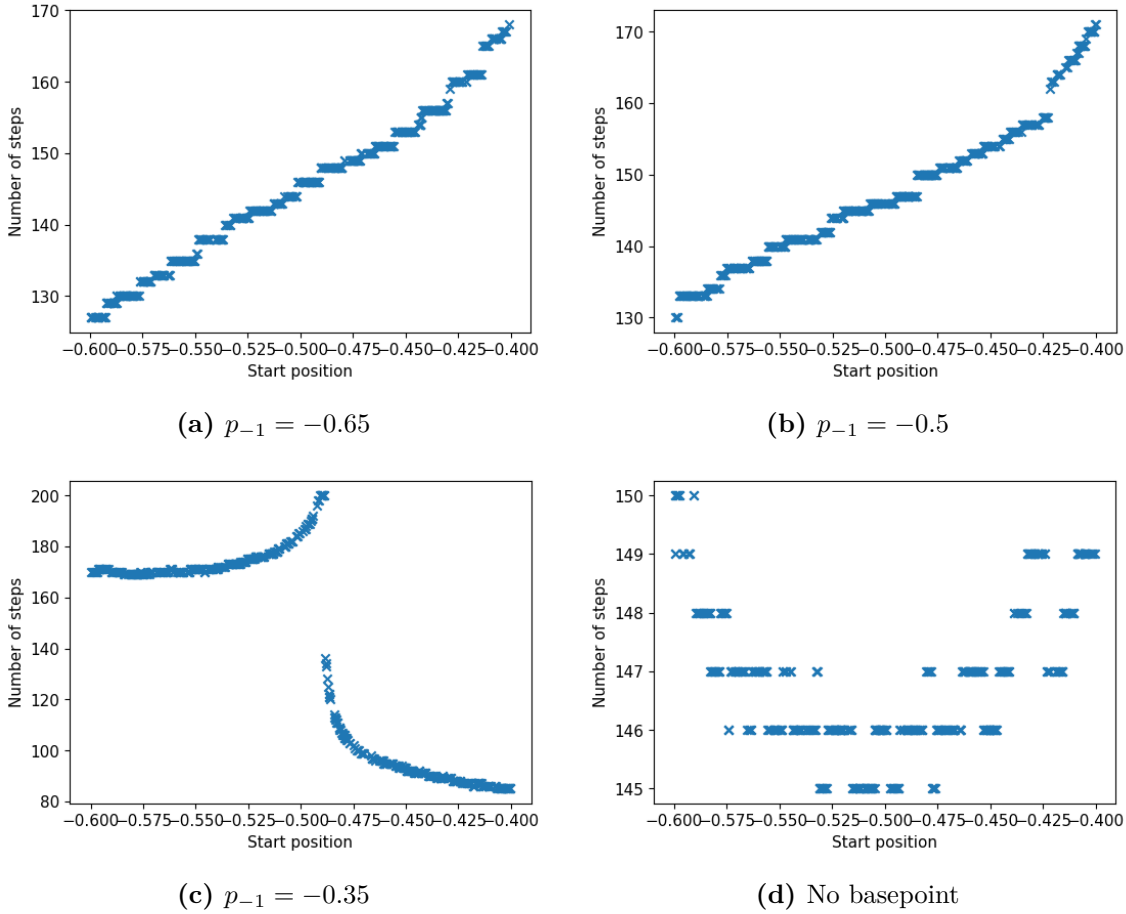
The simple execution problem's environment relies on the discrete event simulator ABIDES [12], and more specifically on it's extension ABIDES-Gym [3], to simulate a stock market. We describe the simulator and its extension, and provide details on the generation of realistic financial market environments.

ABIDES stands for *Agent-Based Interactive Discrete Event Simulation*. It is an open-source market simulator developed to support research in agent-based modeling and simulation within financial markets. At its core, ABIDES is a discrete event simulation engine, which manages the timing of events within the market. Instead of the market's state being updated at fixed steps, time progresses by processing those events sequentially, and the simulation moves forward by executing events in order of their specific timestamps. This approach is particularly efficient for market simulations, where events like order submissions, trades, and price changes happen at irregular intervals. The simulated market consists of a set of trading agents and an exchange agent.[6] Trading agents are autonomous entities that represent market participants, such as traders or institutional investors. Each trading agent has its own strategy and decision-making process, which can range from simple rule-based

[6] In general, those agents are not reinforcement learning agents in the sense of Section 3.1

Table 6.6: The table displays test statistics for the Mountain Car experiment **set-up three** in Section 6.2.4, when following the obtained approximate Q-function from each training run for 500 test episodes. Bold values highlight the best result over basepoint positions p_{-1} for each run: highest reward, lowest standard deviation, most successes or shortest episode length. We refrained from highlighting the column containing mean first observation values $\bar{v}_0^*$, due to the overestimation of those values, and from reporting the cumulative, discounted reward $\bar{r}_\gamma$, since its comparison with $\bar{v}_0^*$ would be superfluous. $\bar{v}_0^*$ is calculated as in Equation (6.5) with basepoints $\hat{o}_{-1} = (1, p_{-1})$ where p_{-1} is set to the respective value in the first column.

p_{-1}	Run	$\bar{r}$	σ_r	succ.	$\bar{m}$	σ_m	$\min_m$	$\max_m$	$\bar{v}_0^*$
−0.65	1	−0.6934	0.0340	**100.0**%	145.51	10.79	127	168	246.53
	2	−0.8456	0.0495	65.8%	184.16	11.67	172	200	44.74
	3	−0.6554	0.1116	98.6%	**139.86**	22.51	90	200	237.59
	4	−0.7017	0.0378	**100.0**%	145.02	10.57	127	170	272.35
−0.5	1	−0.6905	0.0232	**100.0**%	147.36	9.65	130	171	307.97
	2	−0.6844	0.0262	**100.0**%	145.64	9.52	130	164	305.54
	3	−0.6873	0.0239	**100.0**%	146.80	9.12	129	163	296.40
	4	−0.6888	0.0234	**100.0**%	147.00	9.12	130	171	301.21
−0.35	1	**−0.6163**	0.2203	98.0%	**138.16**	41.13	**85**	200	314.15
	2	**−0.6343**	0.2210	99.4%	**142.62**	40.47	**86**	200	310.75
	3	**−0.6269**	0.2220	98.4%	141.79	41.03	**86**	200	326.89
	4	**−0.6327**	0.2189	98.2%	**141.46**	41.47	**84**	200	330.34
None	1	−0.7019	**0.0122**	**100.0**%	146.76	**1.26**	145	**150**	335.42
	2	−0.7039	**0.0144**	**100.0**%	147.50	**1.23**	145	**151**	305.56
	3	−0.7072	**0.0153**	**100.0**%	148.81	**1.27**	146	**153**	307.00
	4	−0.7079	**0.0154**	**100.0**%	149.03	**1.26**	147	**152**	310.69



(a) $p_{-1} = -0.65$

(b) $p_{-1} = -0.5$

(c) $p_{-1} = -0.35$

(d) No basepoint

Figure 6.4: The figure illustrates the effect of different basepoint configurations on the learned policy in the Mountain Car experiment. Each subplot shows the number of steps until termination as a function of the starting position for all episodes in test run 1, for the specific basepoint position p_{-1} and for no basepoint. Plot (a) shows the results for the basepoint position $p_{-1} = -0.65$. Note, that test run 1 is one of two test runs that achieved a 100% success rate for this configuration. Figure (b) shows the results for basepoint position $p_{-1} = -0.5$, with a 100% success rate in test run 1. Figure (c) shows the results for basepoint position $p_{-1} = -0.35$, with a 98% success rate in test run 1. Plot (d) shows the results without a basepoint, with a 100% success rate in test run 1.

logic (e.g. trend-following or market-making with a fixed spread) to more complex algorithms. The exchange agent functions as the central market infrastructure, handling the matching of orders, keeping an order book, and executing trades. It mimics the behavior of real-world exchanges, following protocols like NASDAQ's ITCH and OUCH, used for order book management and trade execution, giving rise to a realistic order processing. In particular, no assumptions about order queue positions for limit orders and order cancellations have to be taken, as opposed to running simulations based on historical data, compare e.g. [70]. Another of ABIDES' key features is its ability to simulate network latency between agents to reflect real-world conditions, especially in high-frequency scenarios.

ABIDES-Gym is an extension of ABIDES, designed to integrate its simulation capabilities with the modular OpenAI Gym framework. It provides a standardized interface to apply reinforcement learning algorithms to different market scenarios, enabling the training and evaluation of learning agents, in a simulated environment that mimics the complexities of real-world financial markets. Each simulation within ABIDES-Gym is represented as Gym environment, where reinforcement learning agents interact with the market by sending actions (e.g. placing or canceling orders) and receiving observations (e.g. market prices, order book states) along with corresponding rewards in a continuous feedback loop. The design separates the underlying market simulation, driven by ABIDES, from the agent's learning algorithm, enabling an easy use of custom methods or third-party reinforcement learning libraries, as demonstrated in [3]. This provides a controlled environment for training RL agents and evaluating their performance in executing trades, managing risk, and maximizing profits.

Trading Agents The ABIDES-Gym framework provides pre-specified market configurations that set up and start the desired market environment with exchange agent settings, and a population of trading agents. To any selected configuration a RL agent is added through the Gym interface, employing for example Signature-Q-Learning or any other method to learn a trading strategy. The configuration may be adapted by changing the trading agent population or the exchange settings, to generate different market conditions. At the same time, more than one learning agent could be added through the Gym interface to create a multi-agent or adversarial RL environment. The following types of trading agents may constitute a set of market participants:

- *Noise agents*, also referred to as *Zero Intelligence* agents [12], wake up once during the trading day and place one limit buy or sell order in the market. Wake up time, order type are determined randomly, as is the order size, drawn from some distribution.
- *Value agents*, also referred to as *Heuristic Belief Learning* agents [12], maintain a believe of the stock's fundamental value, and trade when they think the stock is mispriced, placing a limit buy order if the mid-price is below, and a limit sell order if it is above their believe, respectively. The order size is determined stochastically. An agent's belief is updated in a Bayesian manner, based on noisy observations of some extrinsic fundamental time series representing the stock's fundamental value (described in more detail below in this section).
- *Momentum agents* wake up at fixed time intervals throughout the trading day and

follow a simple momentum strategy. At each wake up they compare the average of the past 50 mid-price observations, $\bar{p}_{50}$, with the average of the past 20 observations $\bar{p}_{20}$. If $\bar{p}_{20} \geq \bar{p}_{50}$, an agent assumes the stock's price will increase, and it places a limit buy order, otherwise a limit sell order, assuming the price will fall.

- *POV Market Maker agents* provide liquidity to the market, through placing limit order pairs at fixed distances to, and outside some window around the current mid-price. The window and distance between price levels is determined at each wake up and based on the observed spread in the market. POV stands for *percentage of volume* and refers to the orders' size which is given as a percentage of the overall volume traded since the last wake up time.

Fundamental Value Time Series An important feature of the simulation is the utilization of an external time series, that defines the fundamental value of the stock. It can be thought of as a consensus valuation derived from all available information at a given time, reflecting the "true" stock price. No market participant (with the exception of the exchange agent) has access to this true fundamental value. Instead, a trading agent is, at best, only able to receive noisy observations of it. An example is the above introduced Value agent, whose fundamental value belief is updated after it obtained such a noisy observation. Crucially, there is only one fundamental value time series per stock in the simulation, and it is not affected by actions taken by trading agents. In the ABIDES simulation, the fundamental value time series is modeled as a mean-reverting stochastic process $X = (X_t)_{t\in[0,T]}$, with $T \in \mathbb{R}$ the simulation's terminal time. Its dynamics are given by

$$\mathrm{d}X_t = -\gamma(X_t - \mu)\,\mathrm{d}t + \sigma\,\mathrm{d}W_t + \mathrm{d}Y_t\,, \quad X_0 = x_0, \tag{6.7}$$

where $W = (W_t)_{t\in[0,T]}$ is a standard Brownian motion, $Y = (Y_t)_{t\in[0,T]}$ a compound Poisson process independent of W, $x_0 \in \mathbb{R}$ the initial value, and $\mu \in \mathbb{R}$, $\sigma > 0$ and $\gamma \in (0, 1]$ are constants. Over time, the process tends to its long term mean μ at the constant mean reversion rate γ. The volatility σ determines how strong the process fluctuates around its tendency to the mean. Without the $\mathrm{d}Y_t$ term, Equation (6.7) would be standard Ornstein-Uhlenbeck process. Its addition introduces jumps to the trajectory. The compound Poisson Y with rate $\lambda > 0$ given as

$$Y_t = \sum_{i=1}^{N_t} J_n, \tag{6.8}$$

where, $(N_t)_{t\in[T]}$ is a homogeneous Poisson counting process with rate λ, and the jumps $(J_n)_{n\in\mathbb{N}}$ are i.i.d. random variables. The distribution of J_1 is the mixture of the two normal distributions with means $\mu_J \in \mathbb{R}$ and $-\mu_J$, common variance $\sigma_J^2 > 0$, and mixing parameter $p = 1/2$. Importantly, the parameters satisfy $|\mu_J| \gg 0$ and $\sigma_J^2 \gg \sigma^2$, resulting in a bimodal distribution with zero mean, which introduces large jumps relative to the continuous fluctuations from the Ornstein-Uhlenbeck component. In a financial context, the Brownian motion term $\sigma\,\mathrm{d}W_t$ models continuous, small movements around the fundamental time series' mean due to high-frequency market interactions, while the compound Poisson term $\mathrm{d}Y_t$ resembles mega-shocks to the stock price, caused by infrequent arrival of extrinsic information. With a variance σ_J^2 significantly larger than σ^2, these shocks are able to

sustainably shifts the consensus valuation of the stock. Note, that while the jump component Y is part of the fundamental value time series as implemented in the ABIDES simulator, in the pre-specific market configuration used in the experiment and detailed in Section 6.3.2, the value for the jump intensity λ is set so small such that practically there are no jumps the trading window.

In general, the simulation advances at the nanoseconds time-scale, where events can occur at any nanosecond time-step during the simulation time. However, activity of single agents is set-up such that it rather occurs at the order of seconds, resembling a more retail-like behaviour and resulting in a sparser distribution of events during the simulation. Due to the continuous time process (6.7) used as fundamental value time series and the simulation progressing event-wise instead of in fixed time-intervals, arbitrary time intervals with no activity may be skipped without any computational cost. Further details on background agents to populate a simulation, and the construction and necessity of a exogenous fundamental value time series are provided in [12, 11], and in the ABIDES github repository's wiki.[7] Details on the specific market configuration used in the simple execution environment are given in the following section.

6.3.2 The Environment

The problem set-up is as follows: The agent faces a simple execution problem, wherein it starts a trading session with positive inventory of the stock traded in the market. Its objective is simply to sell all its holdings until the end of the session. The observations the agent receives only contain information about is current inventory and the remaining time. The transition dynamics however, depend on the mid-price and order flow, both not available to the agent, which makes the environment partially observable. The agent can trade by posting limit orders in both directions, and additionally it may choose not to trade at all. The reward it receives will be a quadratic function of its current inventory, with the magnitude determined by the change in mid-price. The rewards functions are designed in such a way that it penalizes the agent for holding (large) absolute inventory values. To maximize cumulative expected rewards, the agent has to learn to reduce its inventory as quickly as possible towards zero, and subsequently stop trading. This provides a clear optimal baseline policy for the agent, through which Signature-Q-Learning's performance may be assessed. Following the framework of ABIDES-Gym [3], the environment is implemented as a custom Gym environment, on top of the underlying market simulation. In the rest of this section details on the simulated market configuration and the environment's components are given.

Market Configuration The market configuration used in the simple execution environment is provided by the `rmsc04` configuration file from the ABIDES-Gym github repository.[8] It consists of one Exchange Agent, two POV Market Maker agents, 102 Value agents, 12 Momentum agents, and 1000 Noise agents. Together, this population of trading agents ensures rich market dynamics, mimicking real world environments. All agents trade one single stock, whose fundamental value time series is modeled by the stochastic process (6.7),

[7] https://github.com/abides-sim/abides/wiki

[8] https://github.com/jpmorganchase/abides-jpmc-public. The configuration file `rmsc04` is located in the repository sub-directory `/abides-markets/abides-market/configs`.

Table 6.7: The parameter values for the ABIDES fundamental value time series (6.7) in Simple Execution experiment. The values are taken from the configuration file `rmsc04` used in the experiment to set-up the market, which can be found in the ABIDES-Gym github repository. Note, that the simulation advances on the nanosecond time-scale, and that due to the small jump intensity λ effectively there are no jumps in the trading window.

γ	μ	σ^2	x_0	λ	μ_J	σ_J^2
$1.67 \cdot 10^{-16}$	10^5	$2.5 \cdot 10^{-10}$	10^5	$2.77778 \cdot 10^{-18}$	10^3	$5 \cdot 10^4$

with its parameter values given in Table 6.7. Note, that while the fundamental value time series' jump component Y is present in the implementation, due to the small value for the jump intensity λ there are practically no jumps in the trading window[9], and the process (6.7) is effectively a standard Ornstein-Uhlenbeck process. Note, that the fundamental value time series is given in USD cents, such that $\mu = 10^5$ corresponds to a mean value of USD 1000. An episode consists of the simulation of a 35 minutes of trading. To sufficiently populate the order book, the reinforcement learning agent's first wake up is five minutes into the session, which is defined as step $t = 0$ at which it receives its first observation o_0 and the Signature-Q-Learning algorithm is invoked.[10] During the remaining 30 minutes of an episode, the agent wakes up every ten seconds of the simulation to receive a new observation and select an action, resulting in $T = 180$ steps until termination. No transaction costs are incurred in trading. Order priority and queue position as well as market impact and time delays are handled automatically through the ABIDES simulation.

Observation Space For $t \in [T]$, let $i_t \in \mathbb{Z}$ denote the amount of stock the agent currently holds. An inventory $i_t < 0$ means that the agent is short $|i_t|$ units of the stock. Furthermore, we restrict the maximal inventory the agent is allowed to hold in either direction to $|i_t| \leq i_{\max}$ for any $t \in [T]$, where $i_{\max} \in \mathbb{N}$ will be a multiple of the fixed order size the agent trades, see below in the description of the action space. Observations are defined as the current inventory as percentage of the maximal inventory $o_t := i_t / i_{\max}$ taking values in $\mathcal{O} := [-1, 1]$. At each $t \in [T]$, the agent receives the time-augmented observation

$$\hat{o}_t := \left(1 - \frac{t}{T}, o_t\right) = \left(1 - \frac{t}{T}, \frac{i_t}{i_{\max}}\right) \in [0, 1] \times \mathcal{O},$$

where the time-component is the normalized, remaining time in the episode. Each episode starts at some initial observation $\hat{o}_0 = (1, o_0) = (1, i_0/i_{\max})$ with the initial inventory $i_0 \in [-i_{\max}, i_{\max}] \cap \mathbb{Z}$. In the specific environment configuration given in Table 6.9 we set i_0 at a fixed positive value, such that the agent has to learn to liquidate its position over

[9] The probability of at least one jump within the 35 minutes simulated trading interval is $1 - \exp(-\lambda N) \approx \lambda N \approx 9.72 \times 10^{-8}$, with $N = 3.5 \times 10^{10}$ nanoseconds in the window. Even if a full trading day of 8 hours is considered, this probability only increases to about 2.34×10^{-6}.

[10] The environment is implemented such that one can define an additional interval at the beginning of an episode, in which the agent merely receives observations, but is not allowed to trade. This may be useful if for example price or order-book features are included in the observations, such that the first trade decision is informed by their trajectories in this observation interval.

the trading window. Note, however, that this choice is arbitrary and that i_0 could also be sampled according to some initial distribution μ on $[-i_{\max}, i_{\max}] \cap \mathbb{Z}$. Note further, that although the agent does not observe any other market variable, such as the mid-price or order book statistics, the observation transitions still depend on them, making the environment partially observable. More specifically, assume the agent holds i_t units of stock and wants to reduce its holdings to $i < i_t$ by posing a limit order of size $s = i_t - i$. The probability that at $t+1$ the agent indeed observes $i_{t+1} = i$ depends on two things: First, the best bid price in the order book has to match or exceed the agent's limit sell price during the time interval $[t, t+1]$. And second, the order has be entirely executed, since a partial fill would yield a new inventory $i_{t+1} > i_t - i$.

Action Space The action space $\mathcal{A}$ is provided in Table 6.8, and it consists of posting a limit sell order, posting a limit buy order, and not trading. The limit prices p_t^{bid} and p_t^{ask} are the best bid and best ask price available at time $t \in [T]$, which corresponds to the first price level in the order book. This guarantees a high probability that the order will at least partially be filled. If one side of the order book is empty, the respective limit price is set to the price of the last transaction in the market. The agent is restricted to one single order of fixed size $\rho \in \mathbb{N}$ at each step, which satisfies $\rho \leq i_{\max}$ and $(i_{\max} \mod \rho) = 0$. Any existing orders of the agent in the order book are cancelled before a new order is submitted to the market. To satisfy the constraint $|i_t| \leq i_{\max}$ for all $t \in [T]$, whenever $|i_t| + \rho \geq i_{\max}$, and the agent selects the action that would further increase $|i_t|$, Action 2 (Do nothing) is selected instead.

Table 6.8: The action space in the simple execution the execution environment. The prices p_t^{bid} and p_t^{ask} correspond to the best bid and best ask price available in the order book at time t, respectively.

Action 0	Post limit buy order at price p_t^{bid} of size ρ.
Action 1	Post limit sell order at price p_t^{ask} of size ρ.
Action 2	Do nothing

Rewards The policy we want the agent to learn is to reduce its absolute inventory towards zero as quickly as possible, then refrain from trading until the end of the episode. Thus, the reward functions should discourage the agent from holding large absolute inventory values and provide an incentive for keeping its holdings close to zero, or more specifically, at a value that satisfies $|i_t| \leq \rho/2$ with $\rho \in \mathbb{N}$ the agent's fixed order size. A detailed reasoning for this is provided in the description of the baseline policy in Section 6.3.3.

For any $t \in [T]$, let $m_t := (p_t^{bid} + p_t^{ask})/2$ be the mid-price at step t, with p_t^{bid} and p_t^{ask} the best bid and ask price in the order book at step t, respectively, and define the absolute mid-price change at $t \in [T] \setminus \{0\}$ as $\Delta m_t := |m_t - m_{t-1}|$.[11] Then, for all $t \in [T-1]$ and

[11] All prices here are assumed to be in USD terms. Recall, that prices in the simulation are given as integers representing USD cents, thus a simple division by 100 does the trick, compare Section 6.3.1

triplets $(h_t, a_t, o_{t+1}) \in \mathcal{H}_t \times \mathcal{A} \times \mathcal{O}$ the reward function is defined as

$$R_t(h_t, a_t, o_{t+1}) := \Delta m_{t+1} \left[\left(\frac{\rho/2}{i_{\max}} \right)^2 - o_{t+1}^2 \right], \tag{6.9}$$

with $o_{t+1} = i_{t+1}/i_{\max}$.[12] The reward signal's qualitative behaviour is controlled by the inventory the agent holds at $t+1$, which resulted from its trading decision at t and the market events in between, and its magnitude determined by the absolute mid-price change Δm_{t+1}. Whenever $|i_{t+1}| > \rho$, the signal is negative, and it decays quadratically to zero for $|i_{t+1}| \to \rho/2$. For values $|i_{t+1}| > \rho/2$, the reward signal is positive with its maximum at for $i_{t+1} = 0$. Figure 6.5 illustrates the reward functions behaviour for inventory values around the interval $[-\rho/2, \rho/2]$, and this specific design will become more apparent with the optimal baseline policy provided in Section 6.3.3

To provide an additional inventive to end an episode with $i_T \in [-\rho, \rho]$, the terminal reward function is defined similarly as

$$R_T(h_T) := C \left[\left(\frac{\rho/2}{i_{\max}} \right)^2 - o_T^2, \right], \tag{6.10}$$

for some fixed scaling factor $C \geq 0$ and with terminal observation $o_T = i_T/i_{\max}$ in the history h_T.

Environment Parameters The parameter values provided in Table 6.9 define the specific instance of the environment used in the simple execution experiment. The agent starts an episode with $i_0 = 700$, which corresponds to an inventory of 70% of the maximal allowed value $i_{\max} = 1000$, thus $\hat{o}_0 = (1, 0.7)$. With the fixed order size $\rho = 50$, the agent would need 14 orders in total to reduce its holdings to zero if each order was fully filled. Note, that this is seldom the case in the market simulation. The scaling factor for the terminal reward function is set to $C = 0.7$, which was found to yield balanced training.

Table 6.9: The environment parameter values that define the specific instances of the observation space, the action space, and the reward functions used in the Simple Execution experiment.

i_0	$i_{\max}$	ρ	C
700	1000	50	0.7

6.3.3 Baseline Policy

Albeit an optimal policy is not analytically available, the Simple Execution environment offers a deterministic, (possibly approximately) optimal policy, which serves as baseline. At

[12] Note, that R_t is defined as a function of $o_{t+1} = i_{t+1}/i_{\max}$ only (compare also Remark 4.7). While technically it is also a function of Δm_{t+1}, we omit this dependence since the mid-price information is hidden from the agent, highlighting again the partial observability it faces.

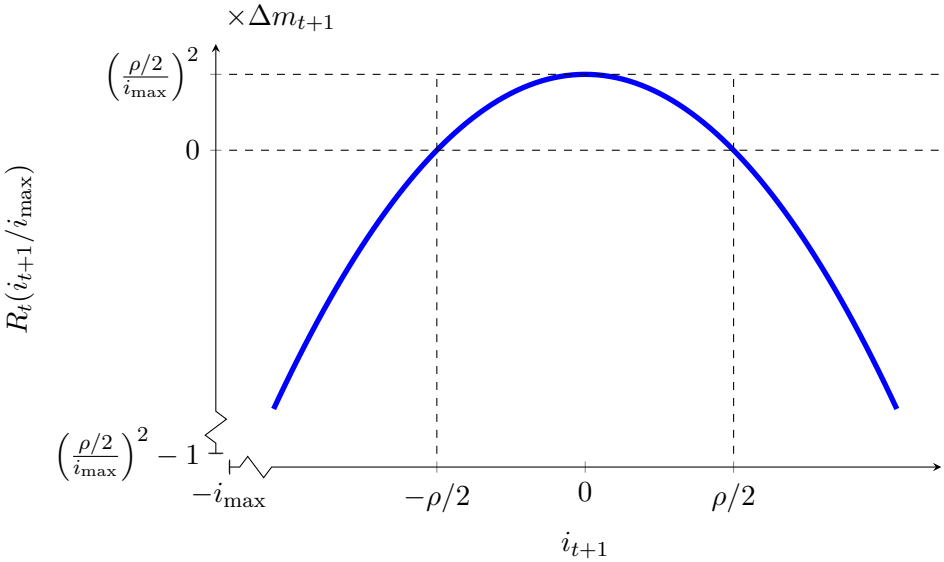


Figure 6.5: The figure shows the behaviour of the reward functions $R_t, t \in [T-1]$ from Equation (6.9), for inventory values symmetrically around zero. Specifically, the agent receives a positive reward for $i_{t+1} \in [-\rho/2, \rho/2]$, with ρ the fixed order size it can trade, and a negative reward otherwise. The maximum value is always attained at $i_{t+1} = 0$, and the functions decay quadratically in both directions to its minima at $|i_{+1}| = i_{\max}$. Note, that the functions' magnitude is determined by the absolute absolute mid-price change Δm_{t+1}, as indicated by the y-axis scaling.

any $t \in [T-1]$, this *optimal baseline policy* π^b is defined as

$$\pi_t^b(h_t) = \pi_t^b(o_t) := \begin{cases} \text{Action 0 – limit buy,} & \text{if } i_t < \rho/2, \\ \text{Action 1 – limit sell,} & \text{if } i_t > \rho/2, \\ \text{Action 2 – do nothing,} & \text{else,} \end{cases} \tag{6.11}$$

for any history $\hat{h}_t \in \hat{\mathcal{H}}_t$ with observation $\hat{o}_t = (1 - t/T, i_t/i_{\max})$ at step t and arbitrary starting inventory $i_0 \in [-i_{\max}, i_{\max}] \cap \mathbb{Z}$. When following π^b, the agent reduces its absolute inventory value $|i_t|$ as quickly as possible with a series of limit buy orders (action 0), respectively, limit sell orders (action 1), depending on the sign of the starting inventory. As soon as its absolute inventory is at or below 50% of the fixed order size ρ, the agent stops trading until the end of the episode (action 2). Note, that due to the fixed order size ρ, when following π^b with arbitrary starting inventory $i_0 \in [-i_{\max}, i_{\max}] \cap \mathbb{Z}$, there almost[13] always exists a $t \in [T]$, such that

$$i_t \in I_{\rho/2} := \left[-\frac{\rho}{2}, \frac{\rho}{2}\right],$$

which is considered an optimal inventory value, in the sense that an additional limit order of size ρ in either direction would result in an inventory i_{t+1} with $|i_{t+1}| \geq \rho/2 \geq |i_t|$, if the order is fully filled. As such, the agent has no further inventive to continue trading as soon as $i_t \in I_{\rho/2}$. Note, that we do not claim π^b to be strictly optimal in the sense of Definition 4.14, since in absence of observation transition probabilities and specifically, order execution dynamics, we do not know whether a different behaviour around absolute inventory values close to $\rho/2$ is better.[14] Nevertheless, since π^b is very close to optimal, we use it as a proxy for a truly optimal policy, and as benchmark to asses the performance of any policy learned by Signature-Q-Learning.

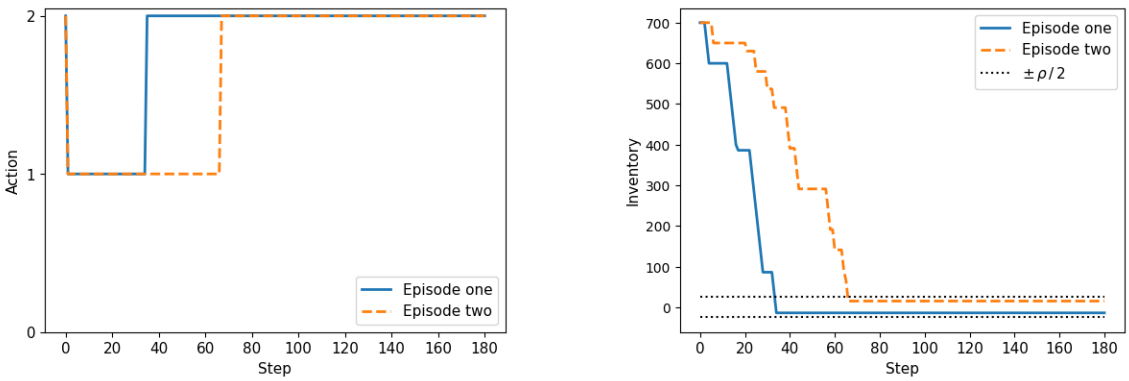
The specific environment configuration used in the experiment, given in Table 6.9, starts with a positive initial inventory $i_0 = 700$. Thus, the optimal baseline policy amounts to posting limit sell orders (action 1) until $i_t \leq \rho/2$ for some $t \in [T]$, and subsequently doing nothing (action 2) until the episode terminates. Figure 6.6 shows two typical action sequences and the resulting inventory trajectories when following π^b in the environment specified in Table 6.9. It is clearly visible that the policy only selects action 1, until the inventory falls within the interval $I_{\rho/2}$, indicated by the dotted lines, after which it switches to action 2 and stops trading. The specific time step $t \in [T]$ at which $i_t \in I_{\rho/2}$ depends on how quickly the limit sell orders are filled in the respective episode, which is apparent by the two different inventory trajectories.

The policy π^b will be run for $M_b = 2000$ baseline episodes. Denote the average, cumula-

[13] Here, *almost* means, save the extreme cases where there is less than $i_0 - \rho/2$ units of the agent's total posted volume executed during the whole trading window. A highly unrealistic scenario where i_0 must amount to a large fraction of the total traded volume in the market over the whole trading window, and negligible for the market configuration detailed in Section 6.3.2.

[14] Consider the example, of the probability of the event, that less than half the volume ρ posted in an order is filled between steps t and $t+1$, being consistently higher than for its complement at any time step t. In this case, a policy that continues to trade when $|i_t|$ is slightly below $\rho/2$, would on average yield a inventory values closer to zero than π^b, and thus, overall higher rewards.

tive reward obtained per episode as $\overline{r}_b$. Since π^b is (approximately) optimal, the value $\overline{r}_b$ constitutes an empirical estimate of the true optimal value $V_0^*(o_0)$ for the initial observation $o_0 = i_0/i_{\max}$. To assess the convergence of Signature-Q-Learning, for any learned approximate Q-function $Q(\cdot|\ell_K)$, we can compare the estimate $\overline{r}_b$ to: (a) the average, cumulative reward per episode when following the greedy-policy with respect to $Q(\cdot|\ell_K)$ for a number of test episodes, and (b) the estimate for $V_0^*(o_0)$ provided by $\max_{a\in\mathcal{A}} Q(\hat{o}_0, a|\ell_K)$.



(a) Baseline policy action sequences.

(b) Baseline policy inventory trajectories.

Figure 6.6: Display of typical (a) action sequences, and (b) inventory trajectories for two episodes when following the optimal baseline policy π^b from Equation (6.11) in the Simple Execution experiment with configuration given in Table 6.9. The dotted lines in (b) correspond to the boundaries of interval $I_{\rho/2} = [-\rho/2, \rho/2]$ for fixed order size $\rho = 50$.

6.3.4 Training Details

Table 6.10 provides the Signature-Q-Learning hyperparameter configuration used for training in the simple execution environment, with the values found through an informal search. In total, 10 training runs are performed. As in the Mountain Car experiment, we use only the time-augmented observation history as input to the approximate Q-function, which at time t is the stream $(\hat{o}_i)_{i=0}^t = (1 - i/T, i_t/i_{\max})_{i=0}^t$. The approximate Q-function is implemented as single linear neural network layer, with the identity map as activation function, which outputs the Q-value for each of the three actions simultaneously. This amounts to three linear functionals of the signature, one for each of the actions to estimate their action-values given any history. For truncation order $N = 7$, we obtain $d_N = 255$ trainable parameters for each action, with d_N the dimension of $T^{(N)}(\mathbb{R}^2)$, which are collected in a weight matrix $W \in \mathbb{R}^{3\times(d_N-1)}$ and a bias vector $b \in \mathbb{R}^3$, compare Equation (5.16) and Section 5.3 for details on this implementation.

The entries in W are initialized randomly, where each entry is sampled uniformly from the interval $(-\sqrt{k}, \sqrt{k})$ with $k = 6/(d_N + 3)$, which corresponds to `PyTorch`'s uniform Xavier initialization [22]. The bias vector's entries are initialized as $b_i = 0.01$ for $i = 1, 2, 3$. A basepoint of $o_{-1} = (0, 0)$ was used for computing the truncated signature at $t = 0$, compare Section 5.3. Note, that since the initial observation is deterministic, the truncated

signature's value at $t = 0$ is the same in each episode. Nevertheless, since we include a basepoint, all of the truncated signature's components contribute to the selection of the first action a_0, as opposed to using no basepoint, in which case only the bias vector would contain non-zero entries that determine the choice of a_0.

Table 6.10: The training hyperparameter values in the Simple Execution experiment. In column order the parameters correspond to number of episodes, episode length, discount factor, loss function, gradient descent optimizer, signature truncation order, and basepoint. The smooth-L_1 loss function is provided by (5.14), the gradient descent optimizer Adam is introduced in [38].

M_{train}	T	γ	$\mathcal{L}$	optimizer	N	$\hat{o}_{-1}$
3000	180	1	smooth-L_1	Adam	7	$(0, 0)$

The exploration policy used is ε-greedy. Exploration rates and learning rates are decayed during training, with the decay schemes detailed in Table 6.2. Contrary to the Mountain Car experiment, decay happens after each single episode. During an episode, the values of both parameter values are held fixed, that is $\varepsilon_n := \varepsilon_{n,0} = \varepsilon_{n,t}$ and $\alpha_n := \alpha_{n,0} = \alpha_{n,t}$ for all $t \in [T-1]$, and episode $n \in \{1, \dots, M_{\text{train}}\}$. We do not discount rewards in the simple execution environment and set $\gamma = 1$. This results in the Q-functions horizon being the full episode length and sampled reward values are propagated throughout all steps.

Table 6.11: Details on the decay schemes of the learning rates and ε-greedy exploration rate in the Simple Execution experiment. End values are rounded and correspond to start value multiplied by the decay factor $M_{\text{train}} = 3000$ times.

Parameter	**Start value**	**Decay factor**	**End value**	**Decay mode**
Learning rates α_n	5×10^{-5}	0.999	2.4856×10^{-6}	Exponential
Exploration rates ε_n	1	0.997	1.2175×10^{-4}	Exponential

6.3.5 Testing Details

As a benchmark, we run the optimal baseline policy π^b defined in Equation (6.11) for $M_b = 2000$ episodes. Then, for each training run, we test the learned approximate Q-function by performing a test run of $M_{\text{test}} = 2000$ episodes. Let $Q(\cdot \mid \ell_K)$ be the approximate Q-function obtained from a training run, with ℓ_K the final linear functional after a total of $K = T \times M_{\text{train}}$ update steps during training. In the following test run, actions are selected greedily with respect to $Q(\cdot \mid \ell_K)$, such that for any $t \in [T-1]$, the policy followed in testing is given as

$$\pi_t^{\text{test}}(h_t) := \arg\max_{a \in \mathcal{A}} Q(\hat{h}_t^a \mid \ell_K), \tag{6.12}$$

for any $h_t \in \mathcal{H}_t$, where we assume there are no ties between actions, such that the arg max is well-defined.[15] For each test run, we report the test statistics in Table 6.12. Note, that there is no discounting of rewards since $\gamma = 1$. For test episode $n \in \{1, \dots, M_{\text{test}}\}$, let $r^{(n)}$ be the sum of (undiscounted) sampled rewards obtained over the episode. The mean cumulative episode reward $\overline{r}$ and the episode reward standard deviation σ_r are calculated as sample mean and standard deviation over $\{r^{(n)}\}_{n \in \{1,\dots,M_{\text{test}}\}}$. Analogously, let $i_T^{(n)} \in [-i_{\max}, i_{\max}] \cap \mathbb{Z}$ be the inventory the agent holds at terminal time T in test episode n. The mean terminal inventory $\overline{i_T}$ and the terminal inventory standard deviation σ_{i_T} are calculated as sample mean and sample standard deviation over $\{i_T^{(n)}\}_{n \in \{1,\dots,M_{\text{test}}\}}$, with $\min_{i_T}$ and $\max_{i_T}$ the minimal and maximal value in the sample, respectively. The first observation value v_0^* is the value of the initial observation $\hat{o}_0 = (1, o_0) = (1, 0.7)$ as provided by the final estimate $Q(\cdot \,|\, \ell_K)$, calculated as

$$v_0^* = \max_{a \in \mathcal{A}} Q(\hat{o}_0, a \,|\, \ell_K). \tag{6.13}$$

Since in the simple execution environment the initial observation is deterministic, v_0^* takes the same value in each test episode, as opposed to the initial values $v_0^{*,n}$ in (6.5) from the Mountain Car experiment. Statistics for the optimal baseline policy are calculated analogously, with $\overline{r}_b$ and σ_{r_b} referring to the episode rewards' sample mean and standard deviation in the baseline run, respectively.

For each run, the value v_0^* offers an indication about whether the algorithm has converged, by comparing v_0^* to the average episode reward $\overline{r}$ from following the test policy π^{test}, and to the average episode reward $\overline{r}_b$ obtained from the optimal baseline policy (6.11). Recall Definitions 4.11 and 4.15 for the value functions $(V_t^\pi)_{t \in [T]}$ corresponding to a given policy $\pi \in \Pi$, and the optimal value functions $(V_t^*)_{t \in [T]}$. Since π^b is (approximately) optimal, $\overline{r}_b$ is a sample-based estimate of $V_0^*(o_0)$ with $o_0 = 0.7$, that is the optimal value over an entire episode for initial observation o_0. In a test run actions are selected according to π^{test}, thus, $\overline{r}$ is an estimate of $V_0^{\pi^{\text{test}}}(o_0)$. Assume that the algorithm has perfectly converged to the true optimal Q-functions. Then π^{test} is optimal and we have

$$\overline{r}_b \approx V_0^*(o_0) = V_0^{\pi^{\text{test}}}(o_0) \approx \overline{r}.$$

In general we expect $\overline{r} < \overline{r}_b$ in any test run, with $\overline{r}_b - \overline{r} \approx 0$. Furthermore, for any $(o_0, a) \in \mathcal{O} \times \mathcal{A}$, we have the equality $Q(\hat{o}_0, a \,|\, \ell_K) = Q_0^*(o_0, a)$, and by Equations (5.2) and (6.13) we obtain

$$\overline{r}_b \approx V_0^*(o_0) = \max_{a \in \mathcal{A}} Q_0^*(o_0, a) = \max_{a \in \mathcal{A}} Q(\hat{o}_0, a \,|\, \ell_K) = v_0^*$$

This implies, that for any test run the fact that $\overline{r}_b \approx v_0^* \approx \overline{r}$ is an indication that the algorithm has converged to the optimal Q-functions.

In the case that $Q(\cdot|\ell_K)$ is not optimal, then we distinguish two cases. In the first case we have we get $v_0^* \approx \overline{r}_b$ but $\overline{r} < \overline{r}_b$ and $\overline{r} < \overline{v}_0^*$. This implies that $Q(\cdot|\ell_K)$ correctly represents the true Q-function Q_0^* at $t = 0$, and from this also the true value V_0^*, however there is some deviation at later steps $t > 0$, resulting in a lower average reward $\overline{r}$ from following π^{test}

[15] If there happens to be a tie between actions in the experiment, we select by sampling uniformly from the set of actions involved in the tie.

than from π^b. In the second case, we get $v_0^* \approx \overline{r}$ but $\overline{r} < \overline{r}_b$ and $v_0^* < \overline{r}_b$. This implies, that while the algorithm was not able to learn the true optimal Q-functions, it still converged to some sub-optimal Q-functions, and $Q(\cdot \mid \ell_K)$ correctly representing the action -values of the policy π^{test}. That $v_0^* \approx \overline{r}$ need not be the case in general is clear from the results of the experiment in Section 6.2, where the approximate Q-function vastly overestimates Q-values, however it produces action sequences that solve the problem.

Table 6.12: Description of the reported statistics in the Simple Execution experiment for the test and baseline runs. The statistics are calculated for each performed test or baseline run with the values are reported in Section 6.3.6. The subscript b indicates that the respective statistic corresponds to the baseline runs. Recall that $\rho = 50$ corresponds to the minimal order size.

$\overline{r}$ / $\overline{r}_b$	Mean reward obtained per episode for test / baseline runs
σ_r / σ_{r_b}	Standard deviation of reward per episode for test / baseline runs
m / m_b	Median reward obtained per episode for test / baseline runs
$\overline{i_T}$	Mean terminal inventory per episode
σ_{i_T}	Standard deviation of terminal inventory per episode
$\min_{i_T}$	Minimum terminal inventory in test run
$\max_{i_T}$	Maximum terminal inventory in test run
v_0^*	First observation value (6.13) (only for test runs)
$\%I_\rho$	Percentage of episodes in a test run with $i_T \in I_\rho = (-\rho, \rho]$
$\%I_{\rho/2}$	Percentage of episodes in a test run with $i_T \in I_{\rho/2} = (-\rho/2, \rho/2]$

6.3.6 Results

First, the results from running the optimal baseline policy are given, then we describe the Signature-Q-Learning algorithm's training behaviour, and subsequently results from testing the obtained approximate Q-functions are presented.

Baseline Results Table 6.13 displays the results from running the optimal baseline policy π^b for $M_b = 2000$ episodes. By design, the mean terminal inventory $\overline{i_T}$ is approximately zero and in each episode i_T falls within the interval $(-\rho/2, \rho/2]$ with minimal order size $\rho = 50$, as indicated by the minimum and maximum terminal inventory values over episodes. The histogram and boxplot in Figure 6.7 show a unimodal and left-skewed distribution of the episode rewards, which explains the median episode reward being larger than the mean. The sample's Fisher-Pearson coefficient of skewness is -2.32, indicating moderate skewness. The mean episode reward $\overline{r}_b = -0.0884$ will serve as benchmark for policies learned by Signature-Q-Learning. The mean episode reward's 95% confidence interval is $(-0.08763, -0.08117)$, which corresponds to a deviation of $\pm 3.83\%$ from the sample mean $\overline{r}_b$. Its calculation is based on the central limit theorem with bounds given by

$$\overline{r}_b \pm t_{M_b-1,0.975} \frac{\sigma_b}{\sqrt{M_b}},$$

with an estimated standard error of the sample mean $\sigma_b/\sqrt{M} = 0.00165$, and where $t_{M_b-1,0.975}$ is the 0.975-quantile of the Student's t distribution with $M_b - 1 = 1999$ degrees of freedom. Due to the large sample size and moderate skewness, this confidence interval calculation is deemed appropriate. Additionally, a 95% confidence interval based on bootstrapping with replacement was computed, with the interval's bounds agreeing with the ones presented above up to an precision of 10^{-4}. Values are rounded to 5 decimal points.

Table 6.13: Reward and inventory statistics received when following the optimal baseline policy π^b from Equation (6.11) in the Simple Execution environment for $M_b = 2000$ episodes. Values are rounded to five and two decimal points, respectively.

$\overline{r}_b$	σ_{r_b}	m_b	$\overline{i_T}$	σ_{i_T}	$\min_{i_T}$	$\max_{i_T}$
−0.08440	0.07359	−0.06188	−0.22	13.10	−24	25

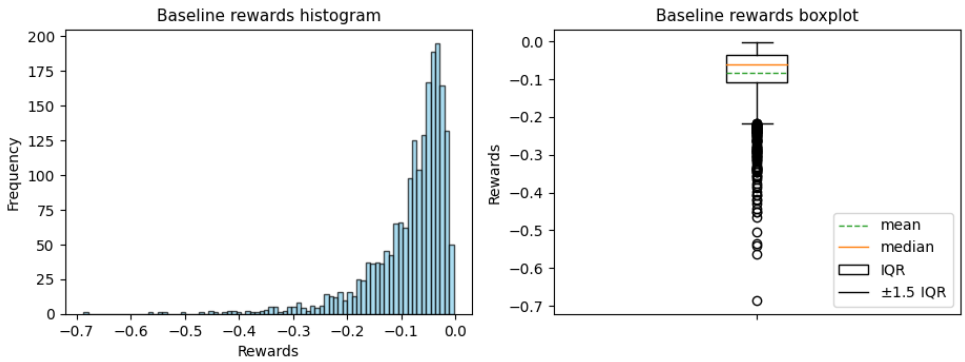


Figure 6.7: Histogram (left) and boxplot (right) of episode rewards when following the optimal baseline policy π^b for $M_b = 2000$ episodes.

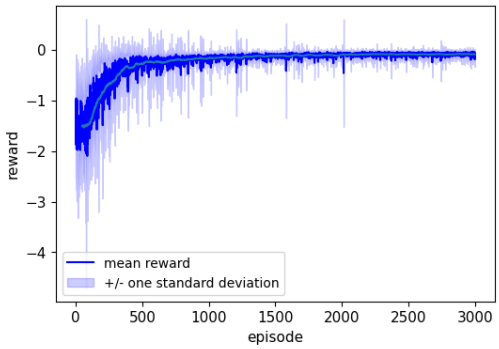
Training Results Analysing Signature-Q-Learning's during training shows robust learning, with all ten runs exhibiting a similar and stable training behaviour. This is evident from Figure 6.8, which displays the average cumulative reward, the average cumulative loss and the average terminal inventory i_T per episode during training. Each statistic is averaged over the ten training runs for each training episode. The standard deviation, represented by the shaded areas around the mean values, becomes small over the training phase, indicating that the ten runs converged similarity. Especially the average loss shows no variation at end of training, meaning that each run converged to a stable approximate Q-function. Rewards increase quickly in the first third of the training phase and tend towards a value slightly below zero. Moreover, Figure 6.8c shows that on average, the agent learns to end the trading window with a terminal inventory i_T close to zero, a characteristic that is expected from an (approximately) optimal policy.

Further evidence for the algorithm's convergence is provided by the evolution of the initial observation value v_0^* during training. Recall the definition of v_0^* from Equation (6.13), as the value of the initial observation $\hat{o}_0 = (1, 0.7)$ provided by the learned approximate Q-function

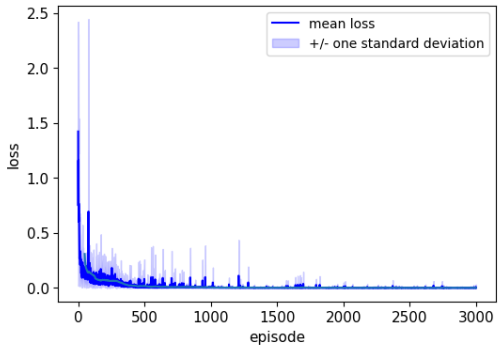
at the end of training. Analogously, for each training episode $n \in \{1, \dots, 3000\}$, we can calculate $v_0^*(n)$ as the value of $\hat{o}_0 = (1, 0.7)$ with the *current* estimate $Q(\cdot|\ell_{k(n)})$, where $\ell_{k(n)}$ is the corresponding linear functional at the end of training episode n, after update step $k(n)$. Figure 6.9a shows the evolution of $v_0^*(n)$ during training for each run, together with their sample mean and standard deviation calculated over runs for each episode. Clearly, $v_0^*(n)$ converges towards the same value for each run, and importantly, the limit value corresponds to the average episode reward $\overline{r}^* = -0.0844$ from following the optimal baseline policy. Since $\overline{r}^*$ is an estimate of the true optimal value V_0^*, convergence of $v_0^*(n)$ provides us with numerical evidence, that Signature-Q-Learning indeed successfully learned the true optimal Q-function Q_0^*. A closer look at the last third of the training episodes in Figure 6.9b shows the average $v_0^*(n)$ values tending to $\overline{r}_b = -0.0844$ with decreasing standard deviation towards the end. Additionally, also the average episode reward roughly settles around $\overline{r}_b$ indicating convergence, albeit it fluctuating with high variability due to the rewards depending on the mid-price trajectory in each episode.

Figure 6.10 offers a more qualitative insight, which again highlights the successful training. It shows the trajectories of selected actions $(a_t)_{t\in[T]}$ and resulting inventories $(i_t)_{t\in[T]}$ for all runs in a selected number of training episodes. Starting from a random behaviour in the first training episode due to the exploration parameter's value $\varepsilon_0 = 1$, in all runs Signature-Q-Learning is able to improve it's action selection as training progresses, until in the last training episode the trajectories clearly resemble the ones generated from following the optimal baseline policy, displayed in Figure 6.6. This shows that in all but one training run the baseline strategy is learned in the end. Only the action Run 5 deviates slightly from an optimal action trajectory by selecting action 0 (limit buy) around step 20. Note, however, that this may be due to the ε-greedy policy followed, since even in the last training episode the probability of selecting an action randomly is positive.

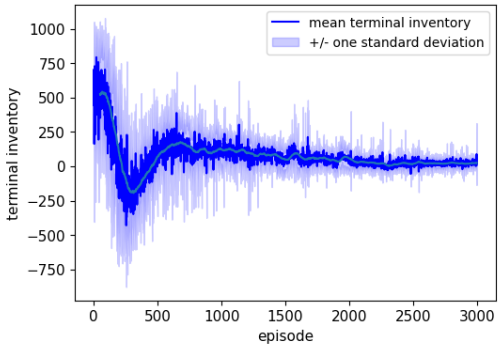
Overall, Signature-Q-Learning demonstrated robust training and exhibited convergence to the truly optimal Q-functions represented by the optimal baseline policy π_b. Recall that in the Mountain Car environment in Section 6.2 episode losses and first observation values diverged in training, despite the algorithm successfully solving the problem, and it was hypothesis, that this is a consequence of the varying episode lengths. The difference to training behaviour observed for the Simple Execution environment which features a fixed episode length, might indeed support this hypothesis, however more analysis is needed.



(a) Average reward per episode.

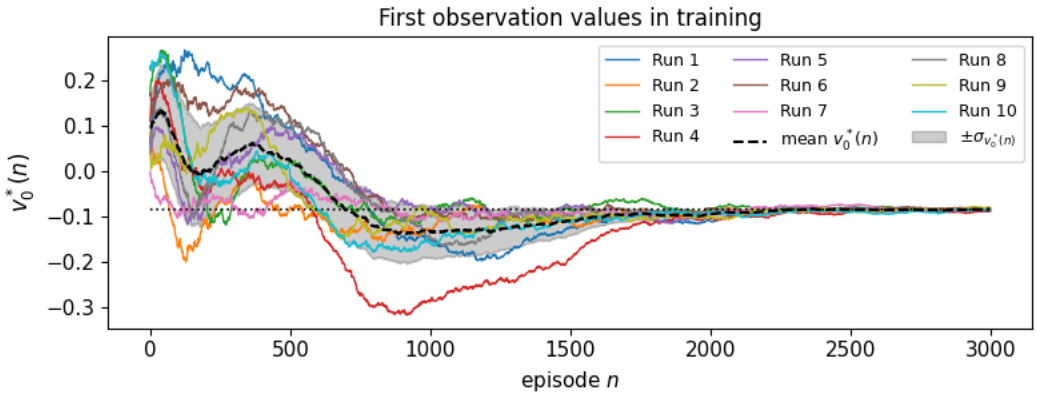


(b) Average loss per episode.

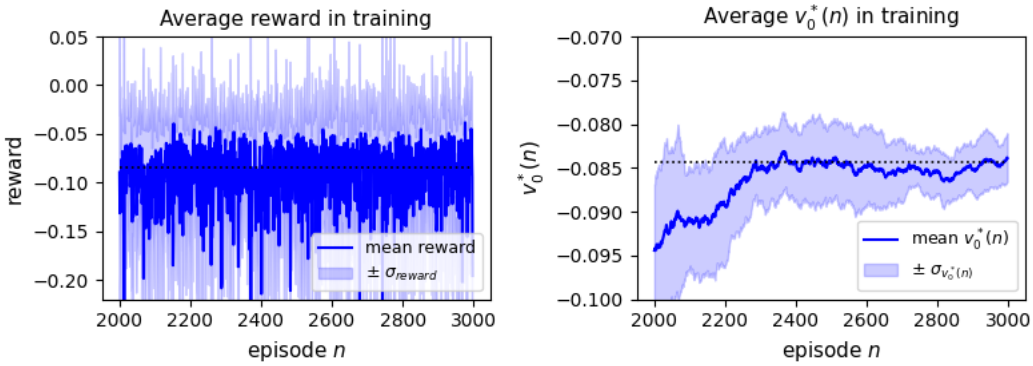


(c) Average terminal inventory i_T per episode.

Figure 6.8: Averaged training results of the Signature-Q-Learning algorithm in the Simple Execution experiment. 10 training runs were performed in total, each with 3000 episodes and with the hyperparameter set-up detailed in Table 6.10. The sub-figures show the respective statistic per episode averaged over training runs. (a) shows the cumulative, undiscounted reward obtained per episode. (b) shows the average total loss obtained per episode. (c) shows the average inventory held at terminal time $T = 180$ per episode.



(a) First observation values v_0^* during training for each run. Sample mean and standard deviation is calculated at each episode over all runs. The dotted line corresponds to the average baseline episode reward $\overline{r}_b = -0.0844$.



(b) Comparison of average reward (**left**) and average first observation value v_0^* (**right**) in the last 1000 training episodes. The dotted black line in each subplot corresponds to the mean baseline episode reward $\overline{r}_b = -0.0844$.

Figure 6.9: The figures illustrate the development of the first observation value $v_0^*(n)$ as a function of the training episode n, and compares it to the average episode reward.

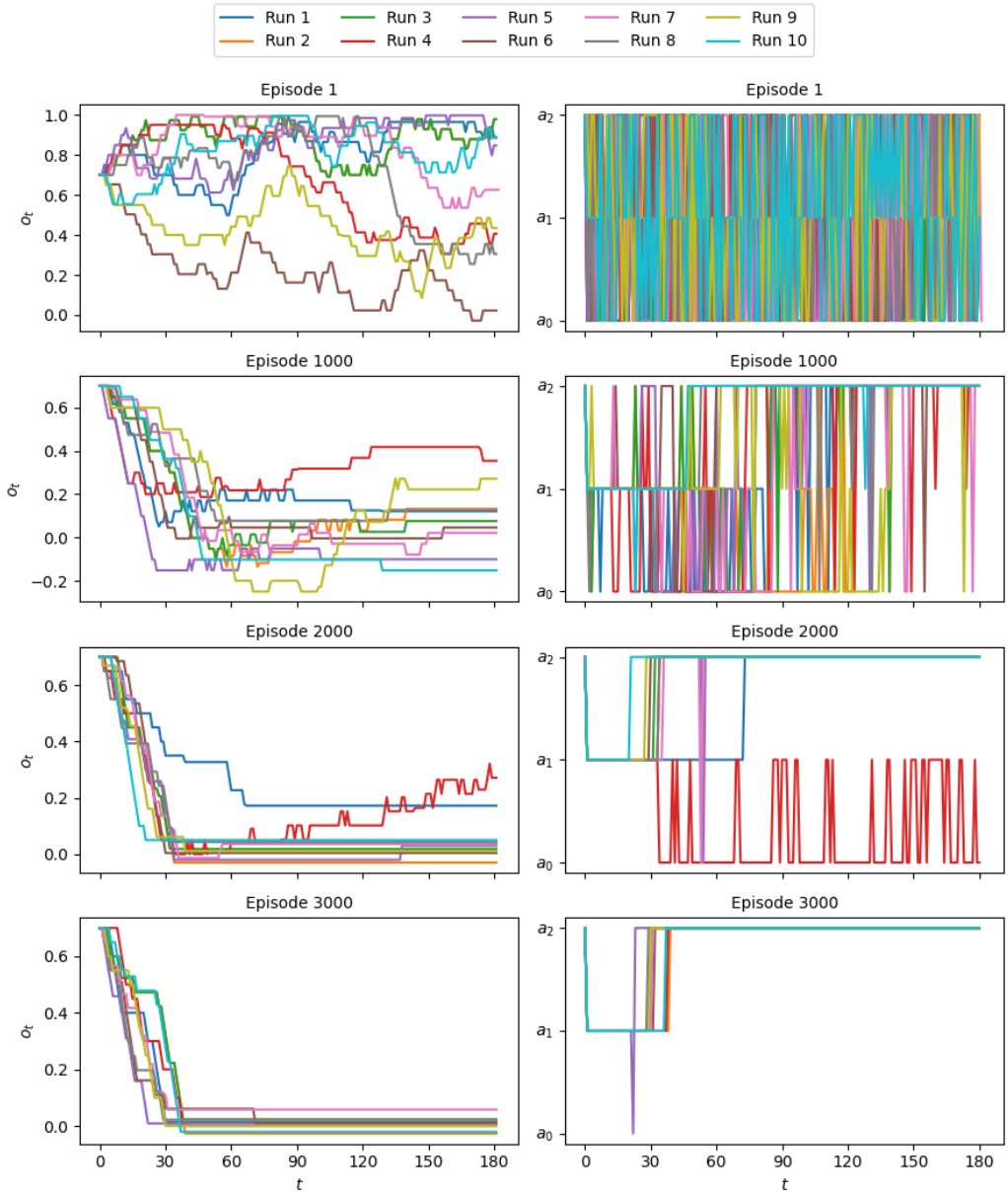


Figure 6.10: The figure shows trajectories of observations $o_t = i_t/i_{\max}$ (**left column**) and actions a_t (**right column**) for $t \in [T]$, with the number of episode steps $T = 180$, and for all performed runs in a number of training episodes: At the start of training in episode 1 (first row), in episode 1000 (second row), in episode 2000 (third row), and at the end of training in episode 3000 (fourth row). The labels of the x-axis of the graphs in the bottom row apply analogously to the graphs in the rows above, and correspond to the time step $t \in [T]$ of an episode.

Testing Results Each of the ten obtained approximate Q-function from training is tested for $M_{\text{test}} = 2000$ episodes. Results regarding rewards obtained in the test runs are displayed in Table 6.14. The first thing to note is, that all runs performed similarity in testing in terms of mean episode reward, indicating that for this specific hyperparameter configuration the algorithm is not sensitive to parameter initialization and the seed used to simulate the environment. The values obtained for each statistic are of the same order of magnitude for all test runs as well as for the baseline run. Test run 4 stands out as best performing run in terms of the episode rewards' mean and median and variation, measured by theirs standard deviation. The largest absolute difference in mean reward between any two runs is 8.68×10^{-3}, which corresponds to roughly 10.5% of the best run's mean value. Notably, despite the apparent convergent behaviour seen in training, the mean episode reward is consistently lower than the mean baseline episode reward of $\overline{r}_b = -8.44 \times 10^{-2}$. The minimal absolute difference is obtained in run 4, and the maximal absolute difference in run 7, with values provided in Table 6.15. Furthermore, the variability in episode rewards is high, with the standard deviations of the same order of magnitude as the means. This is consistent with the values obtained for the baseline policy from Table 6.13. The boxplots of episode rewards in Figure 6.11 show a left-skewed distribution for each test run, similar to the one obtained for the baseline policy, albeit with heavier left tails, indicated by larger interquartile ranges and lower outlier values for most runs. The seed to initialized the environment used in the baseline run was different to any test run to avoid correlation in results due to the same pseudo-random sampling behaviour.

Table 6.14: Reward statistics obtained from testing the approximate Q-function from each training run for $M_b = 2000$ test episodes in the Simple Execution experiment. Bold values highlight the best result over test runs: highest reward, lowest standard deviation, lowest v_0^* value. v_0^* is calculated as in Equation (6.13) for the initial observation $\hat{o}_0 = (1, 0.7)$ and with the basepoint $\hat{o}_{-1} = (0, 0)$ for the truncated signature's computation at time $t = 0$. Values are rounded to five decimal points.

Run	$\overline{r}$	σ_r	m	v_0^*
1	-9.290×10^{-2}	8.177×10^{-2}	-6.769×10^{-2}	-8.817×10^{-2}
2	-9.323×10^{-2}	9.122×10^{-2}	-6.439×10^{-2}	-8.194×10^{-2}
3	-9.392×10^{-2}	8.452×10^{-2}	-6.847×10^{-2}	-8.479×10^{-2}
4	$\mathbf{-8.628 \times 10^{-2}}$	$\mathbf{7.814 \times 10^{-2}}$	$\mathbf{-6.092 \times 10^{-2}}$	-8.777×10^{-2}
5	-9.237×10^{-2}	8.296×10^{-2}	-6.766×10^{-2}	-8.242×10^{-2}
6	-9.220×10^{-2}	8.553×10^{-2}	-6.430×10^{-2}	-8.474×10^{-2}
7	-9.496×10^{-2}	8.317×10^{-2}	-6.797×10^{-2}	$\mathbf{-8.163 \times 10^{-2}}$
8	-9.418×10^{-2}	7.865×10^{-2}	-7.004×10^{-2}	-8.403×10^{-2}
9	-9.385×10^{-2}	8.327×10^{-2}	-6.841×10^{-2}	-7.945×10^{-2}
10	-9.456×10^{-2}	8.177×10^{-2}	-6.848×10^{-2}	-8.528×10^{-2}

The first observation value v_0^* provides another estimate of the true optimal $V_0^*(o_0)$, and a means of judging the algorithms performance. With the exception of run 4, v_0^* is consistently larger than $\overline{r}$, estimating the expected cumulative episode reward $V_0^*(o_0)$ higher, than what

Table 6.15: Absolute differences between the various $V_0^*(o_0)$ estimates obtained in training and testing in the Simple Execution experiment.

Difference	Minimum		Maximum	
	Run	Value	Run	Value
$\lvert\overline{r} - \overline{r}_b\rvert$	4	1.880×10^{-3}	7	1.055×10^{-2}
$\lvert v_0^* - \overline{r}\rvert$	4	1.489×10^{-3}	9	1.440×10^{-2}
$\lvert v_0^* - \overline{r}_b\rvert$	6	3.334×10^{-4}	9	4.949×10^{-3}

was actually obtained from following the learned approximate Q-function in a test run. The range of absolute differences between the two sampled estimates is given in Table 6.15, with the minimal value again in run 4. The training results already indicated that the values v_0^* approached the sampled average baseline episode reward $\overline{r}_b = -8.44 \times 10^{-2}$, which is visible Figure 6.9, and this is confirmed by the testing results. Again with the exception of run 4, v_0^* is consistently closer to the average baseline episode reward $\overline{r}_b$ than $\overline{r}$ from the respective test run. In particular, the minimal and maximal absolute difference $|v_0^* - \overline{r}_b|$ over runs, are an order of magnitude smaller than the difference between remaining estimate combinations, respectively, which is apparent from Table 6.15. As already mentioned in Section 6.3.1, the fact that v_0^* provides a better approximation to the sampled $\overline{r}_b$ than $\overline{r}$ in each run, is a sign the algorithm learned an approximate Q-function $Q(\cdot \mid \ell_K)$ which estimates the true optimal Q-function Q_0^* at $t = 0$, and as such $V_0^* \approx r_b$, sufficiently well, however there is still exist discrepancies to the true Q-functions Q_t^* at some later time steps $t > 0$. Furthermore, for seven out of the then test runs, v_0^* falls within the 95%-confidence interval for the mean baseline episode reward, given by $(-8.763, -8.117) \times 10^{-2}$.

Table 6.16 displays statistics about terminal inventories from the ten test runs. On average, the learned approximate Q-functions resulted in policies that ended episodes with an inventory close to zero: For half of the test runs $\overline{i_T}$ lies within $I_{\rho/2}$, meaning that, on average, the terminal inventory is optimal in the sense that a further, fully filled trade in either direction would yield an inventory value further away from zero, as explained in Section 6.3.3. For the other half, $\overline{i_T}$ still lies within I_ρ, indicating sub-optimality, in the sense that the inventory could be brought closer to zero with further trades. Further, terminal inventory values exhibit some variability, which is apparent from their sample standard deviation σ_{i_T}, and the wide range between $\min_{i_T}$ and $\max_{i_T}$. Since a larger absolute terminal inventory translates to a lower reward, this explains the variability in episode rewards, reported in Table 6.14, as well as the lower mean episode reward obtained in test runs, when compared to the one obtained with the baseline policy, reported in Table 6.15. Notably, the best run in terms of rewards, test run 4, does not have the lowest mean terminal inventory, but ends episodes most consistently with i_T in the intervals $I_{\rho/2}$ and I_ρ, respectively. In general, the percentages of episodes with $i_T \in I_{\rho/2}$ are lower, with more than 50% of episode in three test runs. The share of episodes with $i_T \in I_\rho$ however, increases greatly for all runs with all but one above 70%. Still, the number of episodes with sub-optimal i_T larger than the fixed order size $\rho = 50$, is above 11%, even for runs 4 and 6, which are best in terms of highest mean episode reward $\overline{r}$ and lowest mean terminal

inventory $\overline{i_T}$.

Figure 6.13 displays the action trajectories for all episodes for each test run, and Figure 6.13 shows the corresponding inventory trajectories. Both reveal mixed qualitative behaviour in the test runs. Most notably, in most runs there is still a deviation from the optimal action trajectory in Figure 6.6a provided by the baseline policy. Only the action sequences of runs 7 and 10 are ... however, in both runs, the policy tends to oversell ending with negative inventories smaller than $-\rho$, visible in the corresponding inventory trajectory plots. Action sequences of runs 5 and 6 are close to the baseline trajectories, but in both the policy occasionally chooses to post a limit buy (action 0) before switching to do nothing. Nevertheless, they perform better in terms of mean rewards and percentage of runs in I_ρ and $I_{\rho/2}$, respectively, and run 6 even exhibits the smallest standard deviation of terminal inventories $\sigma_{i_T} = 24.8$, which is visible in inventory trajectories too. In Test run 4 the policy overshoots $-\rho$ by some extend and subsequently starts to post limit buy orders (action 0) to bring the inventory back closer to zero, a behaviour which resulted in the highest mean episode reward. The test run with the lowest mean terminal inventory, run 9, exhibits a similar behaviour, however, instead of switching to limit buy orders in the middle of the episode it does so at the end in a try to increase its inventory closer to zero. In general, we observe some erratic behaviour towards the end of the episodes in half of the runs, an effect likely due to the terminal inventory reward, which encourages the policy to bring the inventory closer to zero before receiving the reward.

In summary, upon observing the test runs' action trajectories the overall policies appear not as robust as the training behaviour has indicated. Notably, in training runs, the learned initial values v_0^* indeed approached the overall baseline episode reward given by $\overline{r}_b$, indicating their convergence to the initial values provided by the true optimal Q-function Q_0^*. Test results, however, hint at the fact that Q-values later during the episode might not have converged sufficiently. This is visible from the fact that mean episode rewards $\overline{r}$ obtained in test runs are below mean episode rewards $\overline{r}_b$ from baseline runs, and from the action selection in test runs, which shows the desired behaviour to some degree, however, still deviates from π^b. Approaches to improve the learned Q-functions, and bring action selection closer to the optimal baseline policy, include longer training, a systematic search for suitable training hyperparameters, and the tuning of the reward structure by balancing running and terminal rewards through parameter C in (6.10).

Table 6.16: Terminal inventory statistics obtained from testing the approximate Q-function from each training run for $M_b = 2000$ test episodes in the Simple Execution experiment. Bold values highlight the best result over test runs: smallest absolute inventory values, lowest standard deviation, highest percentage points. Values are rounded to one decimal point.

Run	$\overline{i_T}$	σ_{i_T}	$\min_{i_T}$	$\max_{i_T}$	$\%I_{\rho/2}$	$\%I_{\rho}$
1	37.5	43.4	−76	501	34.9	75.6
2	13.0	44.0	−257	110	41.6	80.5
3	37.9	38.5	−51	164	41.8	68.2
4	−13.9	29.4	−123	132	**65.4**	**88.8**
5	14.4	38.8	−62	188	52.2	83.2
6	28.7	**24.8**	**−33**	**120**	48.4	82.3
7	34.9	33.3	−201	169	33.1	72.4
8	34.5	42.4	−50	202	43.9	70.4
9	**5.4**	39.8	−279	132	53.2	85.1
10	22.8	36.8	−185	136	39.3	78.8

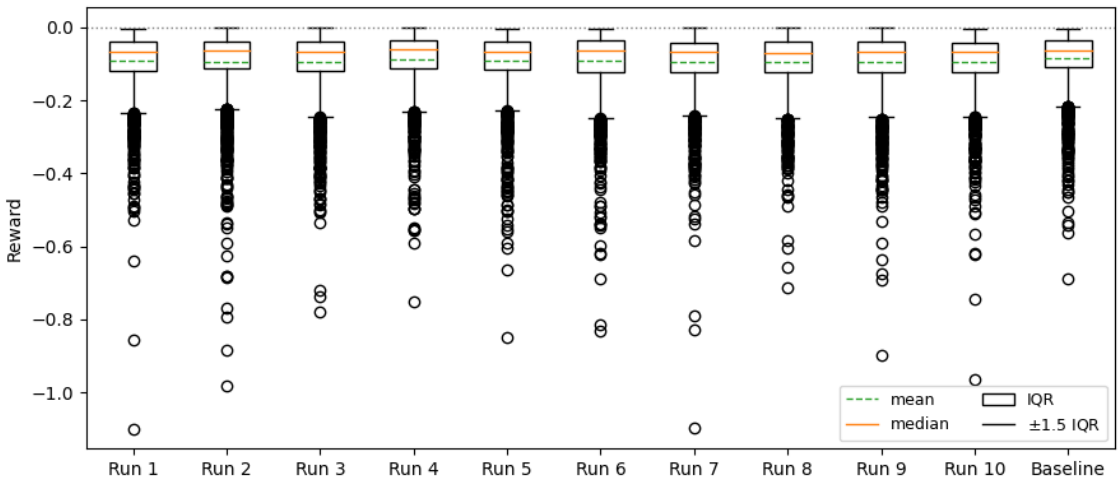


Figure 6.11: Boxplot of the episode rewards in the Simple Execution environment obtained in the ten test runs and the baseline run with $M_{\text{test}} = M_b = 2000$ episodes.

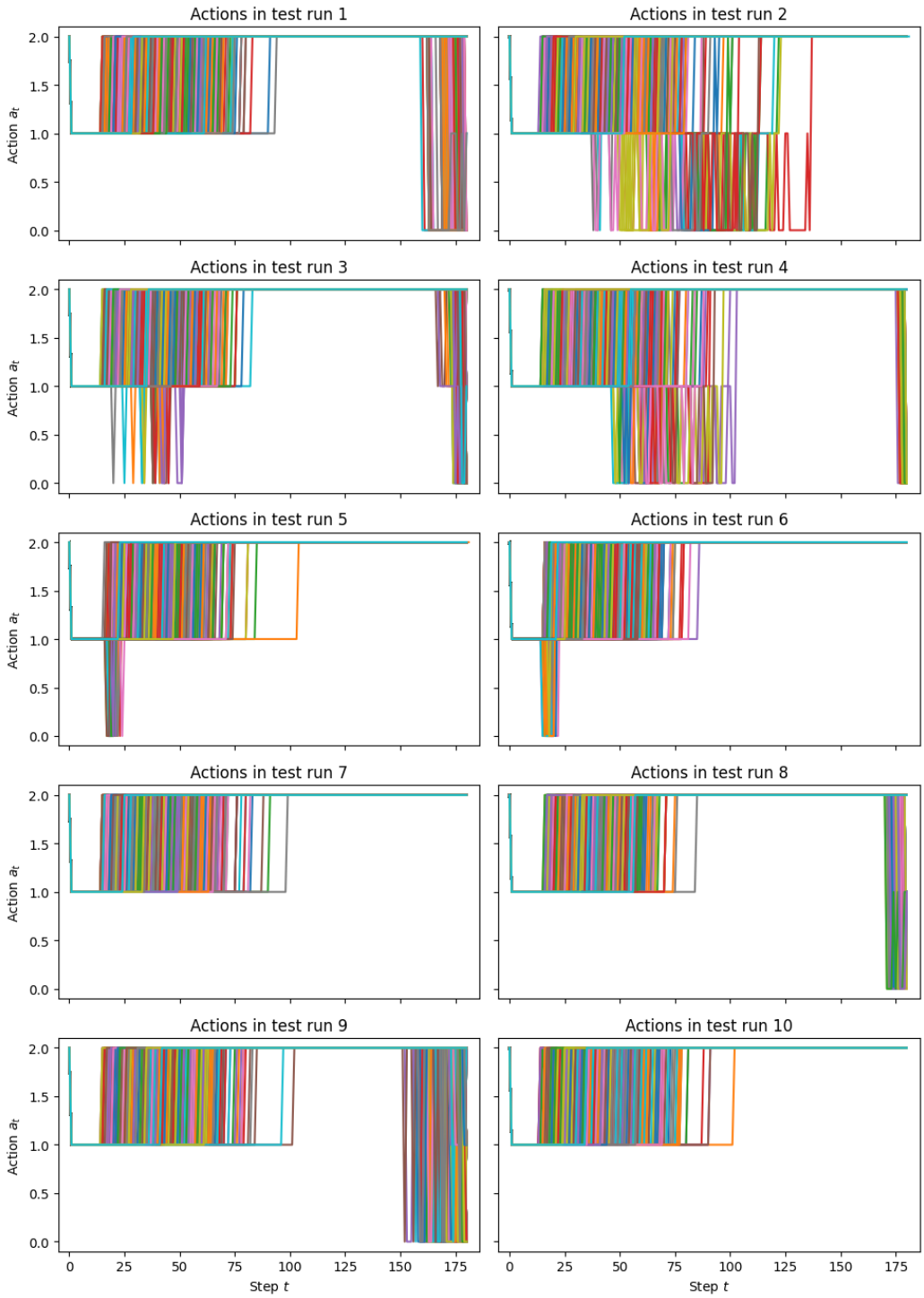


Figure 6.12: Inventory trajectories in test runs. The dotted lines correspond to the inventory values $\{-50, 0, 50\}$, respectively, with $\rho = 50$ the fixed order size in the environment. Subfigures in a column share their x-axis, and plots in a row share their y-axis.

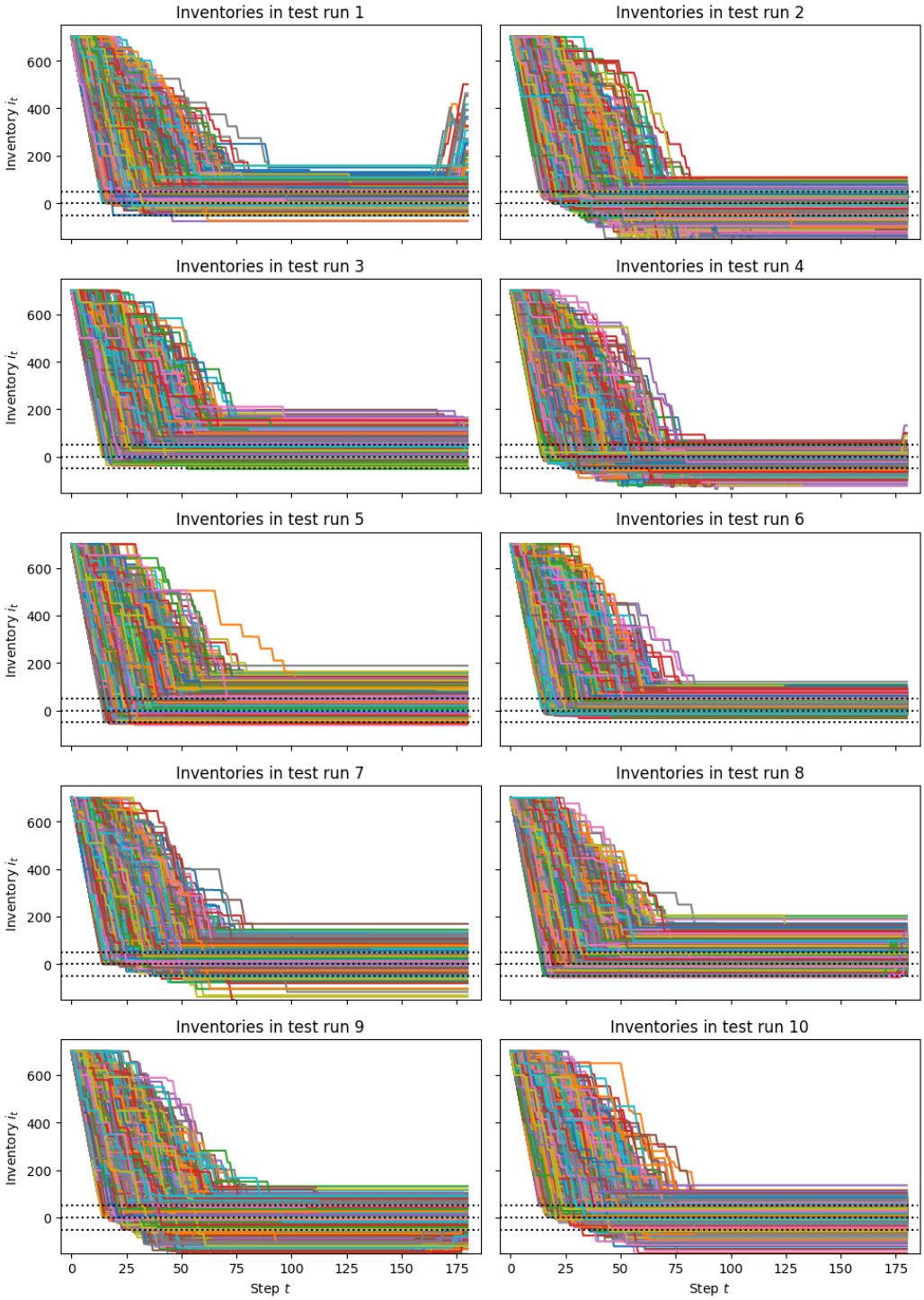


Figure 6.13: Inventory trajectories in test runs. The dotted lines correspond to the inventory values $\{-50, 0, 50\}$, respectively, with $\rho = 50$ the fixed order size in the environment. Subfigures in a column share their x-axis, and plots in a row share their y-axis.

A Appendix

A.1 Stochastic Kernels

Some results on stochastic kernels are collected. They are used to construct the components of the decision processes treated in this thesis. The contents in this section mainly follow the exposition in [40]. Exact references are given with the respective statements in the following.

Definition A.1 (Definition 8.25 in [40])**.** Let $(\Omega_1, \mathcal{F}_1)$ and $(\Omega_2, \mathcal{F}_2)$ be two measurable spaces. A *stochastic kernel* κ from $(\Omega_1, \mathcal{F}_1)$ to $(\Omega_2, \mathcal{F}_2)$ is defined as the map $\kappa : \Omega_1 \times \mathcal{F}_2 \to [0, 1]$ such that

(i) for any fixed $A \in \mathcal{F}_2$, the map $\omega \mapsto \kappa(\omega, A)$ is $\mathcal{F}_1$-measurable,

(ii) for any fixed $\omega \in \Omega_1$, the map $A \mapsto \kappa(\omega, A)$ is a probability measure on $\mathcal{F}_2$.

The probability measure defined by some ω in (ii) will be denoted as $\kappa(\cdot \,|\, \omega) := \kappa(\omega, \cdot)$ and we also write $\kappa(\mathrm{d}y \,|\, \omega)$, such that $\kappa(A \,|\, \omega) = \kappa(\omega, A)$ is the probability of event A under this measure. If it is clear from the context, we will say κ is a stochastic kernel from Ω_1 to Ω_2 without explicitly mentioning the corresponding σ- algebras.

In the literature, a kernel κ is often referred to as *Markov* kernel or *transition* kernel. We do not use the term *Markov* here to avoid ambiguities when defining the more general history-based decision processes and we will use the term transition kernel explicitly for the more specific kernels in the definition of decision processes.

Similar to product measures we can define the product of stochastic kernels.

Theorem A.2 (Theorem 14.25 in [40])**.** *Let $(\Omega_i, \mathcal{F}_i), i = 0, 1, 2$ be measurable spaces, κ_0 a stochastic kernel from $(\Omega_0, \mathcal{F}_0)$ to $(\Omega_1, \mathcal{F}_1)$ and κ_1 a stochastic kernel from $(\Omega_0 \times \Omega_1, \mathcal{F}_0 \otimes \mathcal{F}_1)$ to $(\Omega_2, \mathcal{F}_2)$. Then, the* ***product*** *of κ_0 and κ_1, defined as the map*

$$\begin{aligned} \kappa_0 \otimes \kappa_1 : \Omega_0 \times (\mathcal{F}_1 \otimes \mathcal{F}_2) &\to [0, \infty), \\ (\omega, A) &\mapsto \int_{\Omega_1} \int_{\Omega_2} \mathbb{1}_A(\omega_1, \omega_2)\, \kappa_1((\omega_0, \omega_1), \mathrm{d}\omega_2)\, \kappa_0(\omega_0, \mathrm{d}\omega_1), \end{aligned} \tag{A.1}$$

is well-defined and a stochastic kernel from $(\Omega_0, \mathcal{F}_0)$ to $(\Omega_1 \times \Omega_2, \mathcal{F}_1 \otimes \mathcal{F}_2)$.

Corollary A.3. *Let $(\Omega_1, \mathcal{F}_1, \mu)$ be a probability space and $(\Omega_2, \mathcal{F}_2)$ a measurable space. Let κ be a stochastic kernel from Ω_1 to Ω_2. Then there exists a unique probability measure $\mu \otimes \kappa$ on $(\Omega_1 \times \Omega_2, \mathcal{F}_1 \otimes \mathcal{F}_2)$ defined on measurable rectangles as*

$$\mu \otimes \kappa\,(A_1 \times A_2) := \int_{A_1} \kappa(\omega_1, A_2)\, \mu(\mathrm{d}\omega_1). \tag{A.2}$$

Proof. This follows directly from Theorem A.2 if we consider μ as a stochastic kernel not depending on the Ω_0-coordinate and set $\kappa_0(\omega_0, \cdot\,) = \mu$ and $\kappa_1 = \kappa$. □

As for ordinary product measures, there is a analogue of Fubini's theorem for stochastic kernels.

Theorem A.4 (Theorem 14.32 in [40])**.** *Assume the setting of Corollary A.3 and let the function $f : \Omega_1 \times \Omega_2 \to \overline{\mathbb{R}}$ be measurable with respect to the product σ-algebra $\mathcal{F}_1 \otimes \mathcal{F}_2$. If both or either of the conditions $\int f^+ \,\mathrm{d}\mathbb{P} \otimes \kappa < +\infty$ and $\int f^- \,\mathrm{d}\mathbb{P} \otimes \kappa < +\infty$ hold, then*

$$\int_{\Omega_1 \times \Omega_2} f \,\mathrm{d}(\mu \otimes \kappa) = \int_{\Omega_1} \int_{\Omega_2} f(\omega_1, \omega_2) \kappa(\omega_1, \mathrm{d}\omega_2) \mu(\mathrm{d}\omega_1).$$

To ease notation in the following, for measurable spaces $(\Omega_k, \mathcal{F}_k), k \in \mathbb{N}$ we set

$$(\Omega^{(i,j)}, \mathcal{F}^{(i,j)}) := \Big(\prod_{k=i}^{j} \Omega_k, \bigotimes_{k=i}^{j} \mathcal{F}_k \Big), \quad \text{for } i, j \in \mathbb{N} \text{ with } i \leq j, \tag{A.3}$$

and with elements $\omega_{i:j} := (\omega_i, \omega_{i+1}, \ldots, \omega_j) \in \Omega^{(i,j)}$.

Corollary A.5. *For $n \in \mathbb{N}$, let $(\Omega_i, \mathcal{F}_i), i = 0, 1 \ldots, n$ be measurable spaces and let κ_i be a stochastic kernel from $(\Omega^{(0,i)}, \mathcal{F}^{(0,i)})$ to $(\Omega_{i+1}, \mathcal{F}_{i+1})$ for $i = 0, \ldots, n-1$. Then, the recursion $\kappa^{(0,i)} := \kappa^{(0,i-1)} \otimes \kappa_i$ defines a stochastic kernel from $(\Omega_0, \mathcal{F}_0)$ to $(\Omega^{(1,i+1)}, \mathcal{F}^{(1,i+1)})$ for each $i = 0, \ldots, n-1$. Furthermore, if μ is a probability measure on $(\Omega_0, \mathcal{F}_0)$ then, $\mathbb{P}_i := \mu \otimes \kappa^{(0,i-1)}$, defines a probability measure on $(\Omega^{(0,i)}, \mathcal{F}^{(0,i)})$, for each $i = 0, 1, \ldots, n$, with the convention $\mathbb{P}_0 \equiv \mu$.*

Proof. This follows by inductively applying Theorem A.2 as well as from Corollary A.3. □

The family of probability measures $(\mathbb{P}_i)_{i=0}^n$ defined in Corollary A.5 are by construction projective, by which we mean that for any $i, j, k \in \{1, \ldots, n\}$ with $i, j \geq k$, and for any $A \in \mathcal{F}^{(0,k)}$, we have

$$\mathbb{P}_i[A \times \Omega_{k+1} \times \cdots \times \Omega_i] = \mathbb{P}_j[A \times \Omega_{k+1} \times \cdots \times \Omega_j].$$

In the setting of Corollary A.5, we define the *canonical probability space* given the measurable spaces $(\Omega_i, \mathcal{F}_i)$, the stochastic kernels κ_i and initial distribution μ, as the product space

$$(\Omega, \mathcal{F}, \mathbb{P}) := (\Omega^{(0,n)}, \mathcal{F}^{(0,n)}, \mathbb{P}_n). \tag{A.4}$$

The measure $\mathbb{P}$ is the unique probability measure on the probability space (A.4) that satisfies

$$\mathbb{P}[A \times \Omega_{k+1} \times \cdots \times \Omega_n] = \mathbb{P}_k[A],$$

for any $k \in \{0, 1, \ldots n\}$ and $A \in \mathcal{F}^{(0,k)}$. For $i \in \{0, 1, \ldots, n\}$, let the i-th coordinate map be denoted as $X_i(\omega) := \omega_i$ for $\omega \in \Omega$. The *canonical process* $X := (X_i)_{i \in \{0,1,\ldots,n\}}$ is a stochastic process on $(\Omega, \mathcal{F})$, and by construction, the finite dimensional distributions of X under $\mathbb{P}$

are determined by the kernels and the initial probability measure,

$$\mathbb{P}_{X_0} = \mu, \quad \text{and} \quad \mathbb{P}_{X_0,X_1,\dots,X_i} = \mu \otimes \kappa^{(0,i-1)} \quad \text{for } i \in \{1,\dots,n\}, \tag{A.5}$$

where $\mathbb{P}_{X_0,X_1,\dots,X_j}$ is the push-forward measure of $\mathbb{P}$ under $X_0, X_1, \dots, X_j$.

Stochastic kernels define regular conditional distributions [40, Definition 8.28]. With the stochastic process X constructed as above, the kernel κ_i from Corollary A.3 corresponds to a regular conditional distribution of X_{i+1}, given $X_{0:i} = (X_0, X_1, \dots, X_i)$. This mean that it determines the transition dynamics of X at point i, and for $\omega_{0:i} \in \Omega^{(0,i)}$ and $F \in \mathcal{F}_{i+1}$,

$$\mathbb{P}[X_{i+1} \in F \,|\, X_{0:i} = \omega_{0:i}] = \kappa_i(\omega_{0:i}, F). \tag{A.6}$$

This allows us to express conditional expectations of functions of X_{i+1}, given $X_{0:i} = \omega_{0:i}$, as integrals with respect to the kernel κ_i. Fix $i \in \{0, 1, \dots, n-1\}$ and consider an $\mathcal{F}_{i+1}$-measurable function $f : \Omega_{i+1} \to \mathbb{R}$ which satisfies $\mathbb{E}[|f(X_{i+1})|] < \infty$. Then, the conditional expectation of $f(X_{i+1})$ provided that $X_{0:i} = \omega_{0:i}$ can be expressed as

$$\mathbb{E}_{\kappa_i(\mathrm{d}x|\omega_{0:i})}[f(X_{i+1})] := \mathbb{E}[f(X_{i+1}) \,|\, X_{0:i} = \omega_{0:i}] = \int_{\Omega_{i+1}} f(x)\, \kappa_i(\mathrm{d}x \,|\, \omega_{0:i}). \tag{A.7}$$

The notations $\mathbb{E}_{\kappa_i(\mathrm{d}x|\omega_{0:i})}$ or equivalently $\mathbb{E}_{\kappa_i(\cdot|\omega_{0:i})}$, if the integration variable is clear from the context, to indicate that the conditional expectation is determined by the kernel κ_i, as integral w.r.t. to the measure $\kappa(\cdot|\omega_{0:i})$ for inputs $\omega_{0:i}$. If the function f is instead given as an $\mathcal{F}^{(0,i+1)}$-measurable function $f : \Omega^{(0,i)} \times \Omega_{i+1} \to \mathbb{R}$ that satisfies $\mathbb{E}[|f(X_{0:i}, X_{i+1})|] < \infty$, then given $\omega_{0:i} \in \Omega^{(0,i)}$, we define again

$$\mathbb{E}_{\kappa_i(\mathrm{d}x|\omega_{0:i})}[f(\omega_{0:i}, X_{i+1})] := \mathbb{E}[f(X_{0:i}, X_{i+1}) \,|\, X_{0:i} = \omega_{0:i}] = \int_{\Omega_{i+1}} f(\omega_{0:i}, x)\, \kappa_i(\mathrm{d}x \,|\, \omega_{0:i}). \tag{A.8}$$

Furthermore, the map $\kappa^{(i,n-1)}$ defined recursively by the product of kernels

$$\kappa^{(i,n-1)} := \kappa_i \otimes \kappa^{(i+1,n-1)} \quad \text{for } i = n-1, n-2, \dots, 1, 0, \tag{A.9}$$

is a stochastic kernel from $(\Omega^{(0,i)}, \mathcal{F}^{(0,i)})$ to $(\Omega^{(i+1,n)}, \mathcal{F}^{(i+1,n)})$ for each $i \in \{0, 1, \dots, n-1\}$. Note that $\kappa^{(0,n-1)}$ obtained through the recursion (A.9) coincides with the one obtained from the recursion in Corollary A.5. For each $i \in \{0, 1, 2, \dots, n-1\}$, the kernel $\kappa^{(i,n)}$ is a regular conditional distribution of $X_{i+1:n}$ given $X_{0:i}$. For a $\mathcal{F}$-measurable function $f : \Omega \to \mathbb{R}$, which satisfies $\mathbb{E}[|f(X_{0:n})|] < \infty$, analogue to (A.7), we can express conditional expectations given $X_{0:i} = \omega_{0:i}$ as integral w.r.t. to $\kappa^{(i,n)}$ as

$$\mathbb{E}[f(X_{0:n}) \,|\, X_{0:i} = \omega_{0:i}] = \int_{\Omega^{(i+1,n)}} f(\omega_{0:i}, x_{i+1:n})\, \kappa^{(i,n)}(\mathrm{d}x_{i+1:n} \,|\, \omega_{0:i}) \tag{A.10}$$

for any $\omega_{0:i} \in \Omega^{(0,i)}$, and where $x_{i+1:n} = (x_{i+1}, \dots, x_n)$ are the integration variables and $\omega_{0:i}$ is the input to the kernel $\kappa^{(i,n)}$. Since $\kappa^{(i,n)} = \kappa_i \otimes \kappa^{(i+1,n)}$, application of Theorem A.4

with the measure $\mu = \kappa_i(\cdot|\omega_{0:i})$ and the stochastic kernel κ defined as

$$\begin{aligned}\kappa : \Omega_i \times \mathcal{F}^{(i,n)} &\to [0,1] \\ (\omega_i, F) &\mapsto \kappa(\omega_i, F) := \kappa^{(i,n)}(F \,|\, \omega_{0:i}, \omega_i)\end{aligned}$$

yield the equivalence of the integral of Equation (A.10) to

$$\begin{aligned}\int_{\Omega_{i+1}} \int_{\Omega^{(i+2,n)}} f(\omega_{0:i}, x_{i+1}, x_{i+2:n})\, \kappa^{(i+1:n)}(\mathrm{d}x_{i+2:n} \,|\, \omega_{0:i}, x_{i+1})\, \kappa_i(\mathrm{d}x_{i+1} \,|\, \omega_{0:i}) \\ = \int_{\Omega_{i+1}} \mathbb{E}\big[f(X_{0:n}) \,|\, X_{0:i} = \omega_{0:i}, X_{i+1} = x_{i+1}\big]\, \kappa_i(\mathrm{d}x_{i+1} \,|\, \omega_{0:i}),\end{aligned} \tag{A.11}$$

where the inner integral is simply the conditional expectation of $f(X_{0:n})$ given $X_{0:i} = \omega_{0:i}$ and $X_{i+1} = x_{i+1}$ as in (A.10). Property Equation (A.11) is of particular importance to show recursions for value functions in decision processes, which are defined as conditional expectations.

The exposition in this section shows that, given an initial probability space and a finite sequence of stochastic kernels with associated measurable spaces, there exists a canonical probability space and a discrete-time stochastic process on that space, whose transition dynamics are given by the a priori fixed stochastic kernels.[1] This ensures the existence of the decision processes that model reinforcement learning environments in Sections 3.2 and 4.1.

We conclude this section with two lemmas.

Lemma A.6. *For measurable spaces $(\Omega_i, \mathcal{A}_i), i = 1, 2$ let κ be a stochastic kernel from Ω_1 to Ω_2 and let $g : \Omega_1 \times \Omega_2 \to \overline{\mathbb{R}}$ be measurable w.r.t. $\mathcal{A}_1 \otimes \mathcal{A}_2$. Then, the function $\lambda : \Omega_1 \to \overline{\mathbb{R}}$ given by*

$$\lambda(\omega_1) := \int_{\Omega_2} g(\omega_1, \omega_2)\, \kappa(\mathrm{d}\omega_2 \,|\omega_1) \tag{A.12}$$

is well-defined and $\mathcal{F}_1$-measurable if $f \geq 0$, or if for each $\omega_1 \in \Omega_1$ both or either of the conditions

$$\int_{\Omega_2} g^+(\omega_1, \omega_2)\, \kappa(\mathrm{d}\omega_2|\omega_1) < \infty \quad \textit{and} \quad \int_{\Omega_2} g^-(\omega_1, \omega_2)\, \kappa(\mathrm{d}\omega_2|\omega_1) < \infty, \tag{A.13}$$

holds. Specifically, maps of the form

$$\omega_{0:i} \mapsto \mathbb{E}[f(X_{0:n})|X_{0:i} = \omega_{0:i}], \tag{A.14}$$

for conditional expectations as in Equation (A.10), are $\mathcal{F}^{(0:i)}$-measurable, with $\mathcal{F}^{(0:i)}$ defined in Equation (A.3).

Proof. A proof for the first case $f \geq 0$ is provided in [40, Lemma 14.23]. For general f note that f^+ and f^- are non-negative, measurable functions and as such it follows form the first case that the integrals in (A.13) are well- defined. The claim follows then from

[1] This proposition also holds true for countably many stochastic kernels according to the *Ionesca Tulca* theorem [40, Theorem 14.35].

$f = f^+ - f^-$ and the assumption that at least one of the integral (A.13) is finite. Finally, the measurability of the map (A.14) follows from its representation as integral w.r.t. $\kappa^{(i,n)}(\cdot|\omega_{0:i})$ in Equation (A.10). □

Lemma A.7. *For $n \in \mathbb{N}$, let $(\Omega_i, \mathcal{F}_i), i = 1, \dots, n$ be measurable spaces, and consider the stochastic kernels κ_i for $i = 0, 1 \dots, n-1$ and the probability measures $\mathbb{P}_i$ for $i = 0, 1, \dots, n$ from Corollary A.5. Let $f : \Omega \to \overline{\mathbb{R}}$ be a measurable function on the associated canonical probability space (A.4), with both or either of the conditions $\int_\Omega f^+ \,\mathrm{d}\mathbb{P} < +\infty$ or $\int_\Omega f^- \,\mathrm{d}\mathbb{P} < +\infty$ satisfied. Then,*

$$\int_\Omega f \,\mathrm{d}\mathbb{P} = \int_{\Omega_0} \int_{\Omega_1} \cdots \int_{\Omega_n} f(\omega_0, \omega_1, \dots, \omega_n)\, \kappa_{n-1}(\mathrm{d}\omega_n \,|\, \omega_{0:n-1}) \\ \dots \kappa_1(\mathrm{d}\omega_2 \,|\, \omega_{0:1})\, \kappa_0(\mathrm{d}\omega_1 \,|\, \omega_0)\, \mu(\mathrm{d}\omega_0) \tag{A.15}$$

Proof. The statement follows from inductively applying Theorem A.4 to $\mathbb{P} = \mu \otimes \kappa^{(0,n-1)}$. Since μ is a probability measure on $(\Omega_0, \mathcal{F}_0)$ and $\kappa^{(0,n-1)}$ is a stochastic kernel from Ω_0 to $\Omega^{(1,n)}$, it follows from Theorem A.4 that

$$\int_\Omega f \,\mathrm{d}\mathbb{P} = \int_{\Omega_0} \int_{\Omega^{(1,n)}} f(\omega_0, \omega_{1:n})\, \kappa^{(0,n-1)}(\mathrm{d}\omega_{1:n} \,|\, \omega_0)\, \mu(\mathrm{d}\omega_0).$$

Now assume that for $i = 1, \dots, n-2$ it holds that

$$\int_\Omega f \,\mathrm{d}\mathbb{P} = \int_{\Omega_0} \cdots \int_{\Omega_i} \int_{\Omega^{(i+1,n)}} f(\omega_0, \dots, \omega_i, \omega_{i+1:n})\, \kappa^{(i,n-1)}(\mathrm{d}\omega_{i+1:n} \,|\, \omega_{0:i}) \\ \times \kappa_{i-1}(\mathrm{d}\omega_i \,|\, \omega_{0:i-1}) \dots \kappa_0(\mathrm{d}\omega_1 \,|\, \omega_0)\, \mu(\mathrm{d}\omega_0) \tag{A.16}$$

By definition $\kappa^{(i,n-1)} = \kappa_i \otimes \kappa^{(i+1,n-1)}$, and anew application of Theorem A.4 with the stochastic kernel $\kappa^{(i+1,n-1)}$ and the probability measure $\kappa_i(\cdot|\omega_{0:i})$ with fixed input $\omega_{0:i} \in \Omega^{0,i}$, yields for the innermost integral in (A.16),

$$\int_{\Omega^{(i+1,n)}} f(\omega_0, \dots, \omega_i, \omega_{i+1:n})\, \kappa^{(i,n-1)}(\mathrm{d}\omega_{i+1:n} \,|\, \omega_{0:i}) \\ = \int_{\Omega_{i+1}} \int_{\Omega^{(i+2,n)}} f(\omega_0, \dots, \omega_i, \omega_{i+1}, \omega_{i+2:n})\, \kappa^{(i+1,n-1)}(\mathrm{d}\omega_{i+2:n} \,|\, \omega_{0:i}, \omega_{i+1}) \kappa_i(\mathrm{d}\omega_{i+1} \,|\, \omega_{0:i}). \tag{A.17}$$

Plugging (A.17) into (A.16) proves the induction step. A similar statement has been proved in [6, Proposition 7.28]. □

A.2 Measurable Selection

A measurable selection theorem is presented, based on [29, Lemma 16.1.15]. Let $(\Omega, \mathcal{F})$ be some measurable space and $\mathcal{A} \subset \mathbb{Z}$ with $|\mathcal{A}| < \infty$. Given a function $g : \Omega \times \mathcal{A} \to \mathbb{R}$, we call

the function $\pi : \Omega \to \mathcal{A}$ a maximizer of g, if

$$\pi(\omega) \in \arg\max_{a \in \mathcal{A}} g(\omega, a), \quad \text{for all } \omega \in \Omega.$$

Theorem A.8. *Let $g : \Omega \times \mathcal{A} \to \mathbb{R}$ be $\mathcal{F} \otimes \mathcal{B}_{\mathcal{A}}$-measurable. Then, for each $\omega \in \Omega$, there exists a smallest and a largest maximizer of g and the map $\omega \mapsto g^*(\omega) := \max_{a \in \mathcal{A}} g(\omega, a)$ is $\mathcal{F}$-measurable.*

Proof. This follows literally by setting $(S, \mathfrak{S}) = (\Omega, \mathcal{F})$ as well as $D(s) = A = \mathcal{A}$ for each $s \in S$ in [29, Lemma 16.1.15]. □

List of Figures

3.1 Reinforcement learning in a nutshell 19

5.1 Illustration of exploration parameter decay schemes 55

6.1 Illustration of the Mountain Car environment 60
6.2 Test results scatterplots in the Mountain Car experiment 68
6.3 Averaged training results of set-up one in the Mountain Car experiment . . 69
6.4 Effect of different basepoint configurations in the Mountain Car experiment. 76
6.5 Reward functions in the Simple Execution experiment 83
6.6 Action and inventory trajectories from baseline policy 85
6.7 Distribution of episode rewards from baseline policy 89
6.8 Averaged training results in the Simple Execution experiment. 91
6.9 First observation value during training in the Simple Execution environment 92
6.10 Inventory and action trajectories in training in the Simple Execution experiment 93
6.11 Boxplots of episode rewards from testing in the Simple Execution experiment 97
6.12 Inventory trajectories in test runs 4 and 6 in the Simple Execution experiment 98
6.13 Inventory trajectories in test runs 4 and 6 in the Simple Execution experiment 99

List of Tables

6.1 Training parameter values in set-up one in the Mountain Car experiment . 63
6.2 Parameter decay schemes in the Mountain Car experiment 63
6.3 Description of reported test statistics in the Mountain Car Experiment . . 66
6.4 Test statistics in the Mountain Car experiment for training set-up one . . . 67
6.5 Test statistics in the Mountain Car experiment for training set-up two . . . 72
6.6 Test statistics in the Mountain Car experiment for training set-up three . . 75
6.7 Parameter values for ABIDES' fundamental value time series 80
6.8 Action space in the Simple Execution experiment 81
6.9 Environment parameter values in the Simple Execution experiment 82
6.10 Training parameter values in the Simple Execution experiment 86
6.11 Parameter decay schemes in the Simple Execution experiment 86
6.12 Description of reported test statistics in the Simple Execution Experiment 88
6.13 Baseline policy results in the Simple Execution experiment. 89
6.14 Test statistics for rewards in the Simple Execution experiment. 94
6.15 Differences between $V_0^*(o_0)$ estimates in the Simple Execution environment 95
6.16 Test statistics for terminal inventories in the Simple Execution experiment 97

Bibliography

[1] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems. Online: https://www.tensorflow.org/, 2015. Software available from tensorflow.org. [Cited on page 56.]

[2] E. Akyildirim, M. Gambara, J. Teichmann, and S. Zhou. Applications of signature methods to market anomaly detection. ArXiv preprint arXiv:2201.02441a, 2022. [Cited on page 4.]

[3] S. Amrouni, A. Moulin, J. Vann, S. Vyetrenko, T. Balch, and M. Veloso. Abides-gym: Gym environments for multi-agent discrete event simulation and application to financial markets. In *Proceedings of the Second ACM International Conference on AI in Finance*, ICAIF '21, New York, NY, USA, 2022. Association for Computing Machinery. [Cited on pages 8, 74, 77, and 79.]

[4] C. Bayer, P. Hager, S. Riedel, and J. Schoenmakers. Optimal stopping with signatures. ArXiv preprint arXiv:2105.00778, 2021. [Cited on pages 4, 11, and 17.]

[5] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: an evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47(1):253–279, 05 2013. [Cited on page 30.]

[6] D. P. Bertsekas and S. E. Shreve. *Stochastic Optimal Control: The Discrete-Time Case*. Athena Scientific, 1996. [Cited on pages 33 and 105.]

[7] S. Bhatnagar and K. Lakshmanan. Multiscale q-learning with linear function approximation. *Discrete Event Dynamic Systems*, 26(3):477–509, 2016. [Cited on page 7.]

[8] H. Boedihardjo, X. Geng, T. Lyons, and D. Yang. The signature of a rough path: Uniqueness. *Advances in Mathematics*, 293:720–737, 2016. [Cited on page 16.]

[9] P. Bonnier, P. Kidger, I. P. Arribas, C. Salvi, and T. Lyons. Deep signature transforms. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2019. Curran Associates Inc. [Cited on pages 5, 11, and 17.]

[10] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. OpenAI Gym. ArXiv preprint, arXiv:1606.01540, 2016. [Cited on page 59.]

[11] D. Byrd. Explaining agent-based financial market simulation. ArXiv preprint arXiv:1909.11650, 2019. [Cited on page 79.]

[12] D. Byrd, M. Hybinette, and T. H. Balch. Abides: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, pages 11–22, New York, NY, USA, 2020. Association for Computing Machinery. [Cited on pages 8, 59, 74, 77, and 79.]

[13] D. Carvalho, F. S. Melo, and P. Santos. A new convergent variant of Q-learning with linear function approximation. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 19412–19421. Curran Associates, Inc., 2020. [Cited on pages 7 and 30.]

[14] K.-T. Chen. Integration of paths, geometric invariants and a generalized baker- hausdorff formula. *Annals of Mathematics*, 65:163, 1957. [Cited on pages 4 and 15.]

[15] K.-T. Chen. Iterated path integrals. *Bulletin of the American Mathematical Society*, 83:831–879, 1977. [Cited on pages 4 and 15.]

[16] C. Cuchiero, G. Gazzani, and S. Svaluto-Ferro. Signature-based models: Theory and calibration. *SIAM Journal on Financial Mathematics*, 14(3):910–957, 2023. [Cited on pages 4, 11, and 14.]

[17] C. Cuchiero, F. Primavera, and S. Svaluto-Ferro. Universal approximation theorems for continuous functions of càdlàg paths and Lévy-type signature models. *Finance and Stochastics*, 29(2):289–342, 2025. [Cited on page 14.]

[18] M. Daswani, P. Sunehag, and M. Hutter. Q-learning for history-based reinforcement learning. In C. S. Ong and T. B. Ho, editors, *Proceedings of the 5th Asian Conference on Machine Learning*, volume 29 of *Proceedings of Machine Learning Research*, pages 213–228, Australian National University, Canberra, Australia, 13–15 Nov 2013. PMLR. [Cited on page 6.]

[19] E. A. Feinberg and A. Shwartz. *Handbook of Markov Decision Processes*. Springer US, 2002. [Cited on page 26.]

[20] P. K. Friz and A. Shekhar. General rough integration, Lévy rough paths and a Lévy–Kintchine-type formula. *The Annals of Probability*, 45(4):2707–2765, 2017. [Cited on page 14.]

[21] F. Garcia and S. M. Ndiaye. A learning rate analysis of reinforcement learning algorithms in finite-horizon. In *Proceedings of the Fifteenth International Conference on Machine Learning*, ICML '98, pages 215–223, San Francisco, CA, USA, 1998. Morgan Kaufmann Publishers Inc. [Cited on page 7.]

[22] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In Y. W. Teh and M. Titterington, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9

of *Proceedings of Machine Learning Research*, pages 249–256, Chia Laguna Resort, Sardinia, Italy, 13–15 May 2010. PMLR. [Cited on pages 62 and 85.]

[23] M. Hairer and D. Kelly. Geometric versus non-geometric rough paths. *Annales de l'Institut Henri Poincaré, Probabilités et Statistiques*, 51(1):207 – 251, 2015. [Cited on page 11.]

[24] B. Hambly and T. Lyons. Uniqueness for the signature of a path of bounded variation and the reduced path group. *Annals of Mathematics*, 171(1):109–167, 2010. [Cited on page 16.]

[25] B. Hambly, R. Xu, and H. Yang. Recent advances in reinforcement learning. ArXiv preprint arXiv:2112.04553, 2022. [Cited on page 1.]

[26] H. Hasselt. Double q-learning. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23. Curran Associates, Inc., 2010. [Cited on pages 68 and 70.]

[27] M. Hausknecht and P. Stone. Deep recurrent Q-learning for partially observable mdps. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents (AAAI-SDMIA15)*, Arlington, Virginia, USA, November 2015. [Cited on page 6.]

[28] N. Heess, J. J. Hunt, T. P. Lillicrap, and D. Silver. Memory-based control with recurrent neural networks. ArXiv preprint arXiv: 1512.04455, 2015. [Cited on pages 3, 6, and 32.]

[29] K. Hinderer, U. Rieder, and M. Stieglitz. *Dynamic Optimization*. Springer International Publishing, 2016. [Cited on pages 2, 5, 21, 22, 26, 33, 34, 38, 105, and 106.]

[30] M. Hutter. Feature reinforcement learning: Part I: Unstructured MDPs. *Journal of Artificial General Intelligence*, 1:3–24, oct 2009. [Cited on pages 6 and 33.]

[31] M. Hutter. Feature reinforcement learning: Part II: Structured MDPs. *Journal of Artificial General Intelligence*, pages 71–86, jun 2009. [Cited on page 6.]

[32] M. Hutter. Extreme state aggregation beyond markov decision processes. *Theoretical Computer Science*, 650:73–91, 2016. Algorithmic Learning Theory. [Cited on pages 6 and 33.]

[33] T. Jaakkola, M. I. Jordan, and S. P. Singh. Convergence of stochastic iterative dynamic programming algorithms. In *Proceedings of the 6th International Conference on Neural Information Processing Systems*, NIPS '93, pages 703–710, San Francisco, CA, USA, 1993. Morgan Kaufmann Publishers Inc. [Cited on page 6.]

[34] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1):99–134, 1998. [Cited on pages 5 and 31.]

[35] J. Kalsi, T. Lyons, and I. P. Arribas. Optimal execution with rough path signatures. *SIAM Journal on Financial Mathematics*, 11(2):470–493, 2020. [Cited on pages 4 and 17.]

[36] M. Kaltenbäck. *Fundament analysis*. Berliner Studienreihe zur Mathematik. Heldermann, Lemgo, 2014. [Cited on page 50.]

[37] P. Kidger and T. Lyons. Signatory: differentiable computations of the signature and logsignature transforms, on both CPU and GPU. In *International Conference on Learning Representations*, 2021. https://github.com/patrick-kidger/signatory. [Cited on pages 18, 55, and 59.]

[38] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In Y. Bengio and Y. LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. [Cited on pages 56, 63, and 86.]

[39] F. J. Kiraly and H. Oberhauser. Kernels for sequentially ordered data. *Journal of Machine Learning Research*, 20(31):1–45, 2019. [Cited on page 51.]

[40] A. Klenke. *Probability Theory: A Comprehensive Course*. Springer International Publishing, 2020. [Cited on pages 101, 102, 103, and 104.]

[41] V. Konda and J. Tsitsiklis. Actor-critic algorithms. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. [Cited on page 21.]

[42] A. Krishnamurthy, A. Agarwal, and J. Langford. Pac reinforcement learning with rich observations. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. [Cited on page 31.]

[43] C. Li, X. Zhang, L. Liao, L. Jin, and W. Yang. Skeleton-based gesture recognition using several fully connected layers with path signature features and temporal transformer module. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, AAAI'19/IAAI'19/EAAI'19. AAAI Press, 2019. [Cited on page 4.]

[44] Q. Liu, A. Chung, C. Szepesvari, and C. Jin. When is partially observable reinforcement learning not scary? In P.-L. Loh and M. Raginsky, editors, *Proceedings of Thirty Fifth Conference on Learning Theory*, volume 178 of *Proceedings of Machine Learning Research*, pages 5175–5220. PMLR, 02–05 Jul 2022. [Cited on page 5.]

[45] T. Lyons. The esig python package. https://esig.readthedocs.io/en/latest/, 2002. [Online; accessed 2-October-2023]. [Cited on pages 18 and 55.]

[46] T. Lyons and A. D. McLeod. Signature methods in machine learning. ArXiv preprint arXiv:2206.14674v4, 2023. [Cited on page 11.]

[47] T. Lyons, S. Nejad, and I. P. Arribas. Non-parametric pricing and hedging of exotic derivatives. *Applied Mathematical Finance*, 27(6):457–494, 2020. [Cited on page 4.]

[48] T. J. Lyons. Differential equations driven by rough signals. *Revista Matemática Iberoamericana*, 14(2):215–310, 1998. [Cited on page 4.]

[49] T. J. Lyons, M. Caruana, and T. Lévy. *Differential Equations Driven by Rough Paths*, volume 1908 of *Lecture Notes in Mathematics*. Springer Berlin, Heidelberg, 05 2007. [Cited on pages 4, 11, 12, 14, and 15.]

[50] H. R. Maei, C. Szepesvári, S. Bhatnagar, and R. S. Sutton. Toward off-policy learning control with function approximation. In *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ICML'10, pages 719–726, Madison, WI, USA, 2010. Omnipress. [Cited on page 6.]

[51] S. J. Majeed and M. Hutter. On q-learning convergence for non-markov decision processes. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI-18*, pages 2546–2552. International Joint Conferences on Artificial Intelligence Organization, 7 2018. [Cited on pages 7 and 33.]

[52] F. S. Melo, S. P. Meyn, and M. I. Ribeiro. An analysis of reinforcement learning with function approximation. In *Proceedings of the 25th International Conference on Machine Learning*, ICML '08, pages 664–671, New York, NY, USA, 2008. Association for Computing Machinery. [Cited on pages 6 and 30.]

[53] M. Min and R. Hu. Signatured deep fictitious play for mean field games with common noise. ArXiv preprint arXiv:2106.03272, 2021. [Cited on page 4.]

[54] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. [Cited on pages 1, 5, 6, 7, 30, 31, and 68.]

[55] B. Ning, F. H. T. Lin, and S. Jaimungal. Double deep Q-learning for optimal execution. ArXiv preprint arXiv:1812.06600v2, 2020. [Cited on page 34.]

[56] OpenAI. Chatgpt. https://openai.com/index/chatgpt/, 2022. [Online; accessed 16-July-2025]. [Cited on page 1.]

[57] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Gray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. In A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, editors, *Advances in Neural Information Processing Systems*, 2022. [Cited on page 1.]

[58] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev. Time limits in reinforcement learning. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4045–4054, 10–15 Jul 2018. [Cited on pages 36, 54, and 62.]

[59] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer. Automatic differentiation in pytorch. https://api.semanticscholar.org/CorpusID:40027675, 2017. [Cited on pages 56 and 59.]

[60] M. L. Puterman. *Markov decision processes: discrete stochastic dynamic programming*. Wiley Series in Probability and Mathematical Statistics: Applied Probability and Statistics. John Wiley & Sons, Inc., New York, 2005. A Wiley-Interscience Publication. [Cited on pages 1, 2, 5, 21, 26, and 34.]

[61] J. F. Reizenstein and B. Graham. Algorithm 1004: The iisignature library: Efficient calculation of iterated-integral signatures and log signatures. *ACM Transactions on Mathematical Software*, 46(1), 03 2020. [Cited on pages 18 and 55.]

[62] M. Riedmiller. Neural fitted Q iteration – first experiences with a data efficient neural reinforcement learning method. In *Proceedings of the 16th European Conference on Machine Learning*, ECML'05, pages 317–328, Berlin, Heidelberg, 2005. Springer-Verlag. [Cited on pages 7 and 30.]

[63] H. Robbins and S. Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400 – 407, 1951. [Cited on page 28.]

[64] J. Sadighian. Deep reinforcement learning in cryptocurrency market making. ArXiv preprint arXiv: 1911.08647, 2019. [Cited on page 5.]

[65] J. Sadighian. Extending deep reinforcement learning frameworks in cryptocurrency market making. ArXiv preprint arXiv: 2004.06985, 04 2020. [Cited on page 5.]

[66] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In F. Bach and D. Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897, Lille, France, 07–09 Jul 2015. PMLR. [Cited on page 21.]

[67] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. ArXiv preprint arXiv:1707.06347v2, 2017. [Cited on page 21.]

[68] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018. [Cited on page 1.]

[69] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017. [Cited on page 1.]

[70] T. Spooner, J. Fearnley, R. Savani, and A. Koukorinis. Market making via reinforcement learning. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '18, pages 434–442, Richland, SC, 2018. International Foundation for Autonomous Agents and Multiagent Systems. [Cited on page 77.]

[71] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018. [Cited on pages 2, 6, 19, and 21.]

[72] C. Szepesvári. *Algorithms for Reinforcement Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Springer, Cham, 1 edition, 07 2010. [Cited on page 21.]

[73] C. Szepesvári and W. D. Smart. Interpolation-based q-learning. In *Proceedings of the Twenty-First International Conference on Machine Learning*, ICML '04, page 100, New York, NY, USA, 2004. Association for Computing Machinery. [Cited on page 6.]

[74] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. de Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. J. S. Tan, and O. G. Younis. Gymnasium, version 0.28.1. Python library, 2023. [Cited on pages 8 and 59.]

[75] J. N. Tsitsiklis. Asynchronous stochastic approximation and Q-learning. *Machine Learning*, 16(3):185–202, sep 1994. [Cited on page 6.]

[76] H. van Hasselt, Y. Doron, F. Strub, M. Hessel, N. Sonnerat, and J. Modayil. Deep reinforcement learning and the deadly triad. ArXiv preprint arxiv:1812.02648, 2018. [Cited on page 6.]

[77] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, AAAI'16, pages 2094–2100, 2016. [Cited on pages 68 and 70.]

[78] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019. [Cited on page 1.]

[79] V. V.P and S. Bhatnagar. Efficient energy management in smart grids with finite horizon Q-learning. *Sustainable Energy, Grids and Networks*, 38, 2024. [Cited on pages 7, 27, and 28.]

[80] C. Watkins. *Learning from Delayed Rewards*. PhD thesis, King's College, University of Cambridge, 05 1989. [Cited on pages 2 and 27.]

[81] C. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992. [Cited on pages 2, 21, 26, and 28.]

[82] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992. [Cited on page 21.]

[83] Z. Xie, Z. Sun, L. Jin, H. Ni, and T. Lyons. Learning spatial-semantic context with fully convolutional recurrent network for online handwritten chinese text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(8):1903–1917, 2018. [Cited on page 4.]